

ClaudeForth

An ANS Forth System for the Motorola 6809

Target: the Minimalist Eurocard Board (MECB), 6809 version, configured with 16kB of ROM.

Author: Richard Rothwell

Date: July 11, 2026

Generated with: Claude Sonnet 5

Design & code generation duration: 14 hours

Documentation generation duration: 4 hours

**Duration addressing AI generated task list, debugging, assembler fixes, memory map
resolution:** 16 hours

Table of Contents

1. Introduction.....	5
1.1 Overview.....	5
1.2 Inner-Interpreter Model: Subroutine Threading.....	5
1.3 Register Assignment.....	5
1.4 Cell Size and Character Set.....	5
1.5 The Three-Region Memory Split.....	5
1.6 Exception Handling.....	6
1.7 What This Document Covers.....	6
2. Memory Map.....	7
3. Glossary.....	9
3.1 System / Console I/O.....	9
3.2 Stack Manipulation — Data Stack.....	10
3.3 Stack Manipulation — Return Stack.....	11
3.4 Arithmetic — Single Cell.....	12
3.5 Arithmetic — Mixed & Double Precision.....	13
3.6 Logic, Shifts & Address Arithmetic.....	15
3.7 Comparison.....	16
3.8 Control Flow — Conditionals & Loops.....	17
3.9 Defining Words.....	19
3.10 Compiling Words.....	21
3.11 Memory.....	22
3.12 Strings & Parsing.....	24
3.13 Numeric Output.....	26
3.14 Base / Radix Control.....	27
3.15 Exception Handling.....	27
3.16 Comments.....	28
3.17 Environmental & System Queries.....	28
3.18 Tools.....	29
4. Forth Development Notes.....	30
4.1 Serial I/O and Handshaking.....	30
4.2 Forth Syntax and Stack Operations.....	32
4.3 Tools Word Set: Explanation and Examples.....	33
4.4 Dictionary Management with MARKER.....	34
4.5 The Search-Order Word Set Is Not Included.....	35
5. Open Items Checklist.....	36
5.1 Known bugs — resolved since the original checklist.....	36

5.2 Structural duplication (identified, some resolved, some not).....	44
5.3 Real, load-bearing gaps.....	45
5.4 Never resolved after being explicitly raised.....	46
5.5 Open design questions (not defects — deliberate unresolved choices).....	46
5.6 What's solid (for reference, not action).....	46
6. Build Instructions.....	47
7. Architecture.....	48
7.1 (Outer) Interpreter.....	48
7.2 Dictionary Operations.....	49
8. Assembler Source.....	49
8.1 Memory Map, Constants & GLOBALS Layout.....	50
8.2 Hardware Vector Table.....	56
8.3 Init Code (COLDSTRT / WARM).....	57
8.4 ACIA Interrupt Handler.....	58
8.5 COLD / ABORT / QUIT.....	60
8.6 Inner-Interpreter Support.....	61
8.7 Comma Family.....	62
8.8 HEADER.....	63
8.9 Defining Words.....	64
8.10 Outer Interpreter.....	68
8.11 QUERY / ACCEPT / EXPECT / KEY / KEY? / EMIT.....	73
8.12 CATCH / THROW.....	75
8.13 Control Flow.....	76
8.14 Compiling Words.....	83
8.15 Stack Manipulation.....	85
8.16 Arithmetic.....	88
8.17 Logic, Shifts & Address Arithmetic.....	98
8.18 Comparison.....	99
8.19 Memory.....	101
8.20 String Words.....	103
8.21 Numeric Output.....	111
8.22 Base / Radix Control.....	115
8.23 Output Formatting.....	115
8.24 Comment Words.....	116
8.25 Environmental Query / SOURCE / REFILL / EVALUATE.....	116
8.26 Tools Word Set.....	118
8.27 ABORT / QUIT Hand-Built Headers.....	120
8.28 Forth Dictionary (ROM Base Dictionary Headers).....	121

9. Alphabetical Index.....	138
-----------------------------------	------------

1. Introduction

1.1 Overview

ClaudeForth is an ANS Forth implementation targeting the Motorola 6809 microprocessor. It was designed and built incrementally, decision by decision, across a single extended conversation: the inner-interpreter model was chosen first, then the cell size, character set, memory map, and every word in the Core and Core Extension word sets, followed by double-precision arithmetic, string handling, exception handling, and the Tools word set. This document describes the resulting architecture, the reasoning behind its major design choices, and provides a complete glossary of every word implemented.

1.2 Inner-Interpreter Model: Subroutine Threading

ClaudeForth uses subroutine-threaded code (STC) rather than indirect or direct threading. A colon definition compiles as a flat sequence of 6809 JSR instructions terminated by a compiled RTS. This means the 6809's own JSR/RTS mechanism is the inner interpreter — there is no separate NEXT dispatch loop for straight-line execution. This was chosen because the 6809's hardware stack (register S) is naturally suited to serving as both the CPU's call stack and Forth's return stack simultaneously, and because STC is generally the fastest threading model available on this processor, at the cost of slightly larger compiled code than indirect threading would produce (each call is 3 bytes — a JSR opcode plus a 16-bit address — rather than a 2-byte threaded pointer).

1.3 Register Assignment

U is the data (parameter) stack pointer, manipulated with PSHU/PULU. S is the return stack pointer, using the CPU's native hardware stack directly, so JSR/RTS work as the inner interpreter with no extra bookkeeping. X and Y are general-purpose working registers, used throughout for scanning, indexing, and address computation. D (the combined A:B accumulator) holds top-of-stack values in flight and is the primary vehicle for 16-bit arithmetic, matching the cell size described below.

1.4 Cell Size and Character Set

One cell is 16 bits (two bytes), matching the 6809's native D, X, Y, U, and S registers exactly. This gives a single-cell signed range of -32768 to 32767, unsigned 0 to 65535, and means a cell can always hold a full address across the 6809's entire 64K address space. Because the 6809 has no alignment restriction, ALIGN and ALIGNED are true no-ops. Characters are single-byte 7-bit ASCII (values 0-127), so CHARS and CHAR+ reduce to a no-op and a plain +1 respectively, while CELLS requires an actual multiply-by-two.

1.5 The Three-Region Memory Split

The single most distinctive architectural decision in ClaudeForth is that dictionary headers, compiled code, and user data are tracked as three independent, separately growing regions, each with its own allocation pointer (DPHERE, CODEHERE, and VARHERE respectively) rather than the single unified dictionary most Forth systems use. This departs from the ANS assumption of one HERE; in this system HERE reports CODEHERE specifically, and a parallel word family (V, / VC, / VALLOT / VHERE) exists to manage the variable region directly. The practical consequence is a genuine distinction between immutable data (CONSTANT values, compiled literals, and string literals, all living in code space) and mutable data (VARIABLE, VALUE, and BUFFER: contents, living in variable space) that most single-region Forths do not enforce architecturally.

The intent behind this split is to provide for a ROMable basic Forth system: BASECODE and BASEDICT (Section 2) hold the entire Core and Core Extension implementation as fixed, burned-in ROM content, needing no RAM at all to boot to a working interactive prompt. The RAM regions — APPCODE, APPDICT, and APPVARS — exist specifically to support interactive user application development on top of that fixed base: new words are compiled, tested, and revised freely in RAM without touching the ROM image underneath them. Once an application is complete, the same compiled RAM content (APPCODE and APPDICT together) is intended to be burned into an expanded ROM alongside the base system, turning a finished, debugged application into a second fixed, non-volatile layer rather than something that must be re-compiled from source into RAM on every power-up.

A feature under consideration, not yet designed or implemented, is the ability to strip the dictionary — discarding header (name, LINK, flags) information for some or all ROM or application words once development is finished, while leaving their code intact and callable. A stripped system would trade FIND-ability and interactive extensibility (WORDS, ', and ordinary text interpretation of a word by name) for a smaller ROM footprint and a correspondingly larger RAM variable area, since the dictionary and variable regions currently share the same fixed page-zero and BASEDICT budgets described in Section 2.

1.6 Exception Handling

CATCH and THROW are implemented as direct manipulation of the return stack (register S) rather than a linked list of handler records: CATCH saves the current data-stack pointer and the previous HANDLER value as a small frame on S, and a non-local THROW unwinds directly to that frame with a single TFR instruction, discarding every intervening call level in one step. QUIT wraps INTERPRET in a CATCH and, on a caught error, rolls back all three region pointers (and LATEST) to their state at the start of the failing input line, reclaiming space that an uncaught error would otherwise abandon.

1.7 What This Document Covers

Section 2 gives the full memory map. Section 3 is the glossary of every word implemented, organized by topic, each with its stack-effect notation and any side effects. Section 4 covers five practical development topics: serial I/O configuration and handshaking, a brief introduction to Forth syntax and stack operations, worked examples of the Tools word set, dictionary management with MARKER, and why the Search-Order word set is not included. Section 5 is a checklist of every item still open, resolved, or deliberately deferred during the build. Section 6 covers build instructions: which files to assemble and the assembler this source targets. Section 8 is the complete assembler source listing, including the ROM base dictionary (Section 8.28). Section 9 is an alphabetical index of every word with a page reference back into the glossary.

2. Memory Map

64K address space, addresses shown high to low. Three independent allocation pointers (DPHERE, CODEHERE, VARHERE) track the dictionary, code, and mutable-variable regions respectively; none of the three boundaries below is enforced by a runtime bounds check.

Address Range	Size/bytes	Region	Contents
\$FFF0-\$FFFF	16	VECTORS	Hardware vectors
\$FFA2-\$FFEF	78	INITCODE	COLDSTRT / WARM entry code - was INIT (\$FFC0, then \$FFA0)
\$DFF4-\$FFA1	8110	BASECODE	ROM primitives and compiled system words, 8110-byte nominal budget — ends exactly one byte below INITCODE's start
\$D83F-\$DFF3	1973	BASEDICT	ROM dictionary headers, exact fit (1973 bytes, zero padding)
\$C000-\$C0FF	256	INOUT	Input/output device block, sits directly below USROMSTRT; 6850 ACIA registers live at INOUT+8, not the block base
\$BD00-\$BFFF	768	RSTACK	Return stack (S), grows downward — exactly contiguous with DSTACK below, no gap
\$B900-\$BCFF	1024	DSTACK	Data stack (U), grows downward — occupied bottom matches CODETOP exactly
\$7000-\$B8FF	18688	APPCODE	Application code (CODEHERE) — nominal ceiling (CODETOP) correctly bounded by DSTACK's true range
\$2000-\$6EA4	20133	APPDICT	Application dictionary (DPHERE), 20133 bytes
\$021B-\$1FFF	7653	APPVARS	Mutable variable space (VARHERE), 7653 bytes usable — APPVARSEND self-adjusts to APPDICT-1
\$01FB-\$021A	32	SIBUF	S" interpretation-mode string buffer
\$01DA-\$01FA	33	WORDBUF	WORD's counted-string scratch buffer
\$018A-\$01D9	80	TIBBUF	Terminal input buffer (80 bytes)
\$014A-\$0189	64	OUTBUF	Output ring buffer (64 bytes)
\$010A-\$0149	64	INBUF	Input ring buffer (64 bytes)
\$0106-\$0109	4	SERBUF idx	Serial ring buffer head/tail indices
\$0100-\$0105	6	MVSCRATCH	Shared scratch: MVCNT/MVDST/MVSR, aliased by FILLCNT/HSLN, FILLADDR/HSADDR, MRESULT/FILLCHR
\$0000-\$00FF	256	GLOBALS	Shared system globals — Direct Page = \$00

ROM Size Required

VECTORS, INITCODE, BASECODE, and BASEDICT are the four regions that must actually live in ROM; everything else in the table is RAM or the ACIA hardware itself, memory-mapped rather than stored. USROMSTRT (\$C100) and USROMEND (\$FFEF, one address before VECTORS' start) define the outer boundary these four must fall within, and all four are genuinely contained within it, checked numerically. A full pairwise sweep across every region in the memory map, not just these four, confirms zero overlaps anywhere: BASECODE's nominal end (\$FFA1) lands exactly one byte below INITCODE's start (\$FFA2).

Using the most precise count available (BASECODE's real size confirmed by a full instruction-by-instruction pass at 7984 bytes), VECTORS + INITCODE + BASECODE + BASEDICT comes to 10051 bytes (about 9.82 KB), which is why the ROM part is a 16K×8 EPROM (a 27128 or equivalent) rather than an 8K×8 part — comfortable headroom for real usage.

The INOUT block shadows 256 bytes in the ROM block: a 16K×8 EPROM occupying the natural \$C000–\$FFFF footprint that size implies would include \$C000–\$C0FF, but INOUT claims that same 256-byte range for the ACIA and other memory-mapped I/O. This is why USROMSTRT starts at \$C100, not \$C000 — the bottom 256 bytes of the chip's raw address footprint are never available for ROM content at all, regardless of how much of the chip is otherwise unused.

ROM unused byte count: a 16K×8 EPROM (16384 bytes) minus INOUT's 256-byte shadow minus the 10051 bytes of real VECTORS + INITCODE + BASECODE + BASEDICT content leaves 6077 bytes genuinely unused — about 37% of the chip's total capacity still free. This figure inherits the same caveat as the content totals it's built from: BASECODE's contribution is a careful manual count, not a real assembler's output, so treat 6077 as the best current estimate rather than a guaranteed figure.

Dictionary Entry Layout

Every word in BASEDICT (and every application word compiled into APPDICT) shares this same four-field layout, packed with no padding, in this order:

Code	Size	Name	Comments
LEN/FL	1 byte	Length + Flags	bit 7 = IMMEDIATE, bit 6 = SMUDGE, bits 4-0 = name length (0-31)
NAME	n bytes	Name Field	n raw ASCII characters, no terminator - n comes from LEN/FL's low bits
LINK	2 bytes	Link Field	address of the previous header in the chain, or 0 if this is the oldest entry
CFA	2 bytes	Code Field Address	address FIND returns - the word's own code, or a DODOES trampoline for a CREATED word

3. Glossary

Words are grouped by topic below. Each entry gives the stack-effect notation, a short description, and any side effects beyond the stack transformation shown.

3.1 System / Console I/O

KEY

(-- char)

Blocking read of one character from the input ring buffer.

Side effects: Spins with interrupts enabled until a character is available.

KEY?

(-- flag)

Non-blocking test for a waiting input character.

Side effects: Read-only; never blocks.

EMIT

(char --)

Queue one character for transmission.

Side effects: Enables the ACIA transmit interrupt if not already on.

ACCEPT

(addr n -- n')

Line-edit up to n characters into addr; returns actual count.

Side effects: Echoes input; handles backspace/delete.

EXPECT

(addr n --)

Obsolescent line input; same editing as ACCEPT.

Side effects: Reports its count through SPAN rather than the stack.

QUERY

(--)

Refill the terminal input buffer and reset >IN.

Side effects: Sets SRCADDR/SRCLEN/SRCID for the terminal source.

TYPE

(addr len --)

Output len characters starting at addr.

Side effects: None.

CR

(--)

Output a newline (CR then LF).

Side effects: None.

SPACE

(--)

Output one space character.

Side effects: None.

SPACES

(n --)

Output n spaces.

Side effects: No output if $n \leq 0$.

ABORT

(--) (R: ... --)

Clear the data stack and enter QUIT. Never returns.

Side effects: Full return-stack reset; never executes RTS.

QUIT

(--) (R: ... --)

The outer interpreter's top-level loop. Never returns.

Side effects: Resets the return stack; wraps INTERPRET in CATCH.

3.2 Stack Manipulation — Data Stack

DUP

(n -- n n)

Duplicate the top stack item.

Side effects: None.

DROP

(n --)

Discard the top stack item.

Side effects: None.

SWAP

(n1 n2 -- n2 n1)

Exchange the top two items.

Side effects: None.

OVER

(n1 n2 -- n1 n2 n1)

Copy the second item to the top.

Side effects: None.

ROT

(n1 n2 n3 -- n2 n3 n1)

Rotate the third item to the top.

Side effects: None.

?DUP

(n -- n n | 0)

Duplicate top only if it is nonzero.

Side effects: None.

DEPTH

(-- n)

Push the number of items on the data stack.

Side effects: Computed relative to SPO.

2DUP

(x1 x2 -- x1 x2 x1 x2)

Duplicate the top cell pair.

Side effects: None.

2DROP

```
( x1 x2 -- )
```

Discard the top cell pair.

Side effects: None.

2SWAP

```
( x1 x2 x3 x4 -- x3 x4 x1 x2 )
```

Exchange the top two cell pairs.

Side effects: None.

2OVER

```
( x1 x2 x3 x4 -- x1 x2 x3 x4 x1 x2 )
```

Copy the second cell pair to the top.

Side effects: None.

NIP

```
( x1 x2 -- x2 )
```

Discard the second item.

Side effects: None.

TUCK

```
( x1 x2 -- x2 x1 x2 )
```

Copy the top item under the second.

Side effects: None.

PICK

```
( xu ... x0 u -- xu ... x0 xu )
```

Copy the item u cells deep to the top.

Side effects: No bounds check against actual stack depth.

ROLL

```
( xu ... x0 u -- xu-1 ... x0 xu )
```

Remove the item u cells deep and move it to the top.

Side effects: Cost scales with u; no bounds check.

2ROT

```
( x1..x6 -- x3 x4 x5 x6 x1 x2 )
```

Rotate the third cell pair to the top.

Side effects: None.

3.3 Stack Manipulation — Return Stack

>R

```
( x -- ) ( R: -- x )
```

Move top of data stack to the return stack.

Side effects: Must be balanced by R> within the same definition.

R>

```
( -- x ) ( R: x -- )
```

Move top of return stack to the data stack.

Side effects: None.

R@

```
( -- x ) ( R: x -- x )
```

Copy top of return stack to the data stack.

Side effects: Non-destructive.

2>R

(x1 x2 --) (R: -- x1 x2)

Move top cell pair to the return stack.

Side effects: Must be balanced by 2R> in the same definition.

2R>

(-- x1 x2) (R: x1 x2 --)

Move top cell pair back to the data stack.

Side effects: None.

2R@

(-- x1 x2) (R: x1 x2 -- x1 x2)

Copy top cell pair from the return stack.

Side effects: Non-destructive.

3.4 Arithmetic — Single Cell

+

(n1 n2 -- sum)

Add.

Side effects: None.

-

(n1 n2 -- diff)

Subtract n2 from n1.

Side effects: None.

*

(n1 n2 -- prod)

Signed multiply, truncated to one cell.

Side effects: None.

/

(n1 n2 -- quot)

Signed symmetric division.

Side effects: Throws -10 if n2 is zero.

MOD

(n1 n2 -- rem)

Signed symmetric remainder.

Side effects: Throws -10 if n2 is zero.

/MOD

(n1 n2 -- rem quot)

Quotient and remainder together.

Side effects: Throws -10 if n2 is zero.

NEGATE

(n -- -n)

Two's-complement negate.

Side effects: None.

ABS

```
( n -- |n| )
```

Absolute value.

Side effects: None.

MIN

```
( n1 n2 -- min )
```

Signed minimum.

Side effects: None.

MAX

```
( n1 n2 -- max )
```

Signed maximum.

Side effects: None.

1+

```
( n -- n+1 )
```

Add one.

Side effects: None.

1-

```
( n -- n-1 )
```

Subtract one.

Side effects: None.

2+

```
( n -- n+2 )
```

Add two. Not an ANS-standard word.

Side effects: None.

2*

```
( n -- n*2 )
```

Arithmetic shift left one bit.

Side effects: None.

2/

```
( n -- n/2 )
```

Arithmetic shift right one bit, sign-preserving.

Side effects: None.

* /

```
( n1 n2 n3 -- quot )
```

Multiply then divide via a double-cell intermediate.

Side effects: Throws -10 if n3 is zero.

* /MOD

```
( n1 n2 n3 -- rem quot )
```

As `*/` but yields remainder and quotient.

Side effects: Throws -10 if n3 is zero.

3.5 Arithmetic — Mixed & Double Precision

UM*

```
( u1 u2 -- ud )
```

Unsigned single by single multiply, double result.

Side effects: None.

UM/MOD

```
( ud u1 -- u2 u3 )
```

Unsigned double by single divide.

Side effects: Throws -10 if u1 is zero.

M*

```
( n1 n2 -- d )
```

Signed single by single multiply, double result.

Side effects: None.

FM/MOD

```
( d n1 -- n2 n3 )
```

Floored double by single division.

Side effects: Throws -10 if n1 is zero.

SM/REM

```
( d n1 -- n2 n3 )
```

Symmetric double by single division.

Side effects: Throws -10 if n1 is zero.

D+

```
( d1 d2 -- d3 )
```

Double-cell add.

Side effects: None.

D-

```
( d1 d2 -- d3 )
```

Double-cell subtract.

Side effects: None.

DNEGATE

```
( d1 -- d2 )
```

Double-cell two's-complement negate.

Side effects: None.

DABS

```
( d1 -- d2 )
```

Double-cell absolute value.

Side effects: None.

M+

```
( d1 n -- d2 )
```

Add a single-cell value into a double.

Side effects: None.

S>D

```
( n -- d )
```

Sign-extend a single cell to double.

Side effects: None.

D>S

```
( d -- n )
```

Narrow a double to a single cell.

Side effects: Implementation-defined if d does not fit; the high cell is dropped.

DMAX

(d1 d2 -- d3)

Double-cell signed maximum.

Side effects: None.

DMIN

(d1 d2 -- d3)

Double-cell signed minimum.

Side effects: None.

3.6 Logic, Shifts & Address Arithmetic

AND

(n1 n2 -- n1&n2)

Bitwise AND.

Side effects: None.

OR

(n1 n2 -- n1|n2)

Bitwise OR.

Side effects: None.

XOR

(n1 n2 -- n1^n2)

Bitwise exclusive OR.

Side effects: None.

INVERT

(n -- ~n)

Ones'-complement.

Side effects: None.

LSHIFT

(x1 u -- x2)

Logical shift left u bits, zero-fill.

Side effects: Cost scales with u; no early-out for large u.

RSHIFT

(x1 u -- x2)

Logical shift right u bits, zero-fill.

Side effects: Cost scales with u; no early-out for large u.

CELLS

(n -- n*2)

Convert a cell count to a byte offset.

Side effects: None.

CELL+

(addr -- addr+2)

Add the size of one cell.

Side effects: None.

CHARS

(n -- n)

Convert a character count to a byte offset. No-op on this system.

Side effects: None.

CHAR+

(addr -- addr+1)

Add the size of one character.

Side effects: None.

ALIGN

(--)

Align HERE to a cell boundary. No-op on the 6809.

Side effects: None.

ALIGNED

(addr -- addr)

Align a given address. No-op on the 6809.

Side effects: None.

3.7 Comparison

=

(n1 n2 -- flag)

True if equal.

Side effects: None.

<

(n1 n2 -- flag)

True if n1 signed less than n2.

Side effects: None.

>

(n1 n2 -- flag)

True if n1 signed greater than n2.

Side effects: None.

0=

(n -- flag)

True if n is zero.

Side effects: None.

0<

(n -- flag)

True if n is negative.

Side effects: None.

U<

(u1 u2 -- flag)

True if u1 unsigned less than u2.

Side effects: None.

<>

(n1 n2 -- flag)

True if not equal.

Side effects: None.

0<>

(n -- flag)
 True if n is not zero.
Side effects: None.

0>

(n -- flag)
 True if n is greater than zero.
Side effects: None.

U>

(u1 u2 -- flag)
 True if u1 unsigned greater than u2.
Side effects: None.

WITHIN

(n1 n2 n3 -- flag)
 True if $n2 \leq n1 < n3$.
Side effects: Computed via unsigned comparison of offsets, handles wraparound ranges correctly.

D=

(d1 d2 -- flag)
 Double-cell equal.
Side effects: None.

D<

(d1 d2 -- flag)
 Double-cell signed less than.
Side effects: Compares high cells (signed) first, low cells (unsigned) only if equal.

DU<

(d1 d2 -- flag)
 Double-cell unsigned less than.
Side effects: Both tiers compared unsigned.

3.8 Control Flow — Conditionals & Loops

IF

Compile: (--) Run: (flag --)
 Begin a conditional. IMMEDIATE, compile-only.
Side effects: Compiles ZBRANCH and a forward-patch field.

THEN

(--)
 Resolve an IF or ELSE forward branch. IMMEDIATE, compile-only.
Side effects: Throws -22 if the control-flow tag does not match.

ELSE

(--)
 Alternate branch of IF. IMMEDIATE, compile-only.
Side effects: Throws -22 if the control-flow tag does not match.

BEGIN

(--)
 Mark a loop target. IMMEDIATE, compile-only.
Side effects: None.

UNTIL

(flag --)

Branch back to BEGIN while flag is false. IMMEDIATE, compile-only.

Side effects: Throws -22 on a tag mismatch.

AGAIN

(--)

Unconditional branch back to BEGIN. IMMEDIATE, compile-only.

Side effects: Throws -22 on a tag mismatch.

WHILE

(flag --)

Conditional exit within a BEGIN loop. IMMEDIATE, compile-only.

Side effects: Leaves a second pending forward-patch field.

REPEAT

(--)

Close a BEGIN...WHILE loop. IMMEDIATE, compile-only.

Side effects: Patches both the back-edge and WHILE's forward exit.

RECURSE

(--)

Compile a call to the word currently being defined. IMMEDIATE, compile-only.

Side effects: Bypasses FIND's smudge check; reads LATEST's own CFA directly.

DO

Compile: (--) Run: (n1|u1 n2|u2 --) (R: -- loop-sys)

Begin a counted loop. IMMEDIATE, compile-only.

Side effects: Builds a 4-cell loop frame on the return stack; runs at least once even if limit=index.

?DO

Compile: (--) Run: (n1|u1 n2|u2 --) (R: -- loop-sys)

As DO, but skips the loop entirely if limit=index. IMMEDIATE, compile-only.

Side effects: Builds the same loop frame; adds a forward skip branch.

LOOP

(--) (R: loop-sys -- | loop-sys)

Increment index by 1; loop until index reaches limit. IMMEDIATE, compile-only.

Side effects: Discards the loop frame on exit.

+LOOP

(n --) (R: loop-sys -- | loop-sys)

Increment index by n; exits when the step crosses limit. IMMEDIATE, compile-only.

Side effects: Handles positive or negative step; discards the frame on exit.

I

(-- n) (R: loop-sys -- loop-sys)

Fetch the innermost loop's index.

Side effects: Non-destructive read of the current loop frame.

J

(-- n) (R: loop-sys1 loop-sys2 -- loop-sys1 loop-sys2)

Fetch the next-outer loop's index.

Side effects: Only reaches one level of nesting out; no J2 equivalent exists.

LEAVE

(--) (R: loop-sys -- loop-sys)

Force the innermost loop to exit at its next LOOP/+LOOP.

Side effects: Assumes it is textually inside the loop it affects; unverified across an intervening word call.

UNLOOP

(--) (R: loop-sys --)

Discard the innermost loop frame without branching.

Side effects: Needed before an EXIT taken from inside a DO...LOOP not using EXIT itself.

EXIT

(--)

Return from the current definition immediately. IMMEDIATE, compile-only in the sense that it must be used inside a definition.

Side effects: Counts and discards any open DO/?DO frames at compile time before compiling its own return.

CASE

(--)

Begin a multi-way selection. IMMEDIATE, compile-only.

Side effects: None.

OF

(x n -- | x)

Begin one CASE clause's test. IMMEDIATE, compile-only.

Side effects: Drops the selector only on a matching clause.

ENDOF

(--)

End one CASE clause. IMMEDIATE, compile-only.

Side effects: Throws -22 on a tag mismatch.

ENDCASE

(x --)

Close a CASE structure, discarding the selector.

Side effects: Throws -22 if the control-flow stack is malformed.

3.9 Defining Words

:

("name" --)

Begin a colon definition.

Side effects: Header is smudged until ; snapshot control-flow depth for later checking.

;

(--)

End a colon definition. IMMEDIATE.

Side effects: Unsmudges the header; throws -22 if a control structure is left open.

CREATE

("name" --)

Build a header and a DOES>-ready trampoline.

Side effects: PFA defaults to CODEHERE (immutable space).

DOES>

(--)

Attach runtime behavior to a CREATED word. IMMEDIATE, compile-only.

Side effects: Patches the trampoline's BEHAVIOR field at run time via a double return.

VARIABLE

("name" --)

Create a one-cell mutable variable.

Side effects: PFA reserved in VARHERE (mutable space); initialized to zero.

CONSTANT

(x "name" --)

Create a word that returns a fixed value.

Side effects: Value stored in CODEHERE (immutable space).

VALUE

(x "name" --)

Create a reassignable named value.

Side effects: PFA in VARHERE (mutable space); reassigned via TO.

TO

(x "name" --)

Assign a new value to a VALUE. IMMEDIATE.

Side effects: Behaves differently interpreting vs. compiling.

2VARIABLE

("name" --)

Create a two-cell mutable variable.

Side effects: PFA in VARHERE; both cells initialized to zero.

2CONSTANT

(x1 x2 "name" --)

Create a word returning a fixed double value.

Side effects: Stores x1 at the lower address, x2 at the higher, matching 2@/2!.

BUFFER:

(u "name" --)

Create a word returning the address of u reserved, uninitialized bytes.

Side effects: PFA in VARHERE.

DEFER

("name" --)

Create a word whose action can be set later.

Side effects: Default action throws -21 until IS/DEFER! is used.

DEFER@

(xt-defer -- xt)

Fetch a deferred word's current target.

Side effects: None.

DEFER!

(xt xt-defer --)

Set a deferred word's target.

Side effects: None.

IS

(xt "name" --)

Set a deferred word's target by name. IMMEDIATE.

Side effects: Behaves differently interpreting vs. compiling.

ACTION-OF

("name" -- xt)

Fetch a deferred word's current target by name. IMMEDIATE.

Side effects: Behaves differently interpreting vs. compiling.

MARKER

("name" --)

Create a word that, when executed, forgets itself and everything defined after it.

Side effects: Restores all three HERE pointers and LATEST from a snapshot.

3.10 Compiling Words

IMMEDIATE

(--)

Mark the most recently defined word as IMMEDIATE.

Side effects: Sets a header flag bit.

STATE

(-- addr)

Variable: nonzero while compiling.

Side effects: None.

[

(--)

Enter interpret state. IMMEDIATE.

Side effects: None.

]

(--)

Enter compile state.

Side effects: None.

,

("name" -- xt)

Look up a word's execution token.

Side effects: Throws -13 if not found. Works identically interpreting or compiling.

COMPILE,

(xt --)

Compile a call to the given execution token.

Side effects: None.

LITERAL

(x --)

Compile x as a literal. IMMEDIATE, compile-only.

Side effects: None.

[']

("name" --)

Compile-time tick: compile a word's xt as a literal. IMMEDIATE, compile-only.

Side effects: Throws -14 if used while interpreting; -13 if the name is not found.

POSTPONE

```
( "name" -- )
```

Defer a word's compiling behavior into the current definition. IMMEDIATE, compile-only.

Side effects: Throws -14 if used while interpreting; -13 if the name is not found.

>BODY

```
( xt -- pfa )
```

Recover a CREATED word's parameter field address.

Side effects: Assumes the standard trampoline layout; fixed offset arithmetic.

EXECUTE

```
( xt -- )
```

Execute the word with the given execution token.

Side effects: None.

SLITERAL

```
( addr len -- )
```

Compile a runtime string as a literal. IMMEDIATE, compile-only.

Side effects: Throws -14 if used while interpreting.

ABORT"

```
Compile: ( "msg" -- ) Run: ( flag -- )
```

Conditionally type a message and abort. IMMEDIATE, compile-only.

Side effects: On true flag: types the message, throws -2.

3.11 Memory

@

```
( addr -- x )
```

Fetch a cell.

Side effects: None.

!

```
( x addr -- )
```

Store a cell.

Side effects: None.

C@

```
( addr -- char )
```

Fetch a byte, zero-extended.

Side effects: None.

C!

```
( char addr -- )
```

Store the low byte only.

Side effects: None.

+!

```
( n addr -- )
```

Add n into the cell at addr.

Side effects: None.

2@

```
( addr -- x1 x2 )
```

Fetch a double cell.

Side effects: Lower address is x1, higher address is x2.

2!

(x1 x2 addr --)

Store a double cell.

Side effects: x1 stored at the lower address, x2 at the higher.

'

(n --)

Append one cell to the immutable/code region (CODEHERE).

Side effects: Advances CODEHERE.

C,

(char --)

Append one byte to the immutable/code region.

Side effects: Advances CODEHERE.

ALLOT

(n --)

Reserve (or release, if negative) bytes in the immutable/code region.

Side effects: No bounds check against adjoining regions.

HERE

(-- addr)

Current allocation point of the immutable/code region.

Side effects: Reports CODEHERE.

V,

(n --)

Append one cell to the mutable/variable region (VARHERE).

Side effects: Advances VARHERE.

VC,

(char --)

Append one byte to the mutable/variable region.

Side effects: Advances VARHERE.

VALLLOT

(n --)

Reserve (or release) bytes in the mutable/variable region.

Side effects: No bounds check against adjoining regions.

WHERE

(-- addr)

Current allocation point of the mutable/variable region.

Side effects: Reports VARHERE.

PAD

(-- addr)

A scratch buffer address, offset above HERE.

Side effects: Floats relative to CODEHERE; not a separately reserved region.

UNUSED

(-- u)

Bytes remaining in the region HERE addresses.

Side effects: Reports distance to the return/data-stack boundary.

VUNUSED

(-- u)

Bytes remaining in the mutable/variable region.

Side effects: Reports distance to APPCODE's start; not itself enforced.

MOVE

(src dst u --)

Copy u bytes, overlap-safe.

Side effects: Chooses direction automatically based on address order.

FILL

(addr u char --)

Fill u bytes with char.

Side effects: None.

ERASE

(addr u --)

Fill u bytes with zero.

Side effects: None.

CMOVE

(src dst u --)

Copy u bytes low address to high.

Side effects: Not overlap-safe in the high-over-low direction.

CMOVE>

(src dst u --)

Copy u bytes high address to low.

Side effects: Not overlap-safe in the low-over-high direction.

3.12 Strings & Parsing

COUNT

(c-addr -- addr len)

Convert a counted string to address/length form.

Side effects: None.

WORD

(char -- c-addr)

Parse a delimited token into a counted string.

Side effects: Hard 31-character cap; skips leading delimiters.

CHAR

("name" -- char)

Parse the next word, leave its first character.

Side effects: Inherits WORD's 31-character cap (irrelevant here).

[CHAR]

("name" --)

Compile-time CHAR: compiles the character as a literal. IMMEDIATE, compile-only.

Side effects: Throws -14 if used while interpreting.

PARSE

```
( char "ccc<char>" -- addr len )
```

Parse a delimited token, no length cap.

Side effects: Does not skip leading delimiters, unlike WORD.

PARSE - NAME

```
( "<spaces>name<space>" -- addr len )
```

Parse a space-delimited token, no length cap.

Side effects: Skips leading spaces.

S"

```
Compile: ( "str" -- ) Run: ( -- addr len )
```

Parse and compile, or stage, a string literal.

Side effects: Compiling: appends inline to CODEHERE. Interpreting: copies into SIBUF.

."

```
( "str" -- )
```

Compile a string that types itself at run time. IMMEDIATE, compile-only.

Side effects: No interpretation semantics, per ANS.

COMPARE

```
( addr1 len1 addr2 len2 -- n )
```

Compare two strings, signed three-way result.

Side effects: None.

SEARCH

```
( addr1 len1 addr2 len2 -- addr3 len3 flag )
```

Find a substring within a string.

Side effects: Empty needle reports not-found (an implementation choice).

-TRAILING

```
( addr len1 -- addr len2 )
```

Trim trailing spaces from a string's reported length.

Side effects: None.

/STRING

```
( addr1 len1 n -- addr2 len2 )
```

Trim n characters from the front of a string.

Side effects: None.

REPLACES

```
( addr1 len1 addr2 len2 -- )
```

Register a name/value substitution pair.

Side effects: Single-slot only, not a table; a second call overwrites the first.

SUBSTITUTE

```
( addr1 len1 addr2 len2 -- addr2 len3 )
```

Apply the registered substitution into a destination buffer.

Side effects: Throws -1 on destination overflow; single-slot only.

SNAME

```
( xt -- addr len )
```

Reverse dictionary lookup: xt to name. Non-standard.

Side effects: Returns zero length if not found.

UNESCAPE

(addr len -- addr len')
 Process backslash escapes in place. Non-standard.
Side effects: Result length is always <= input length.

3.13 Numeric Output

<#
 (--)
 Begin pictured numeric output conversion.
Side effects: Sets HLD to PAD's address.

#
 (ud1 -- ud2)
 Convert one digit.
Side effects: Uses the current BASE.

#S
 (ud1 -- ud2)
 Convert remaining digits until the value is zero.
Side effects: Uses the current BASE.

#>
 (ud -- addr len)
 End pictured conversion, discarding ud.
Side effects: None.

HOLD
 (char --)
 Insert one character into the pictured output buffer.
Side effects: Buffer grows downward from PAD; no bound enforced.

HOLDS
 (addr len --)
 Insert an entire string into the pictured output buffer.
Side effects: Depends on HOLD's exact decrement-by-one behavior.

SIGN
 (n --)
 Insert a minus sign if n is negative.
Side effects: None.

.
 (n --)
 Print signed, trailing space.
Side effects: None.

U.
 (u --)
 Print unsigned, trailing space.
Side effects: None.

.R
 (n width --)
 Print signed, right-justified in a field.
Side effects: No trailing space; width is a minimum, not a cap.

U . R

(u width --)

Print unsigned, right-justified in a field.

Side effects: No trailing space; width is a minimum, not a cap.

?

(addr --)

Fetch and print the cell at addr, signed.

Side effects: None.

D .

(d --)

Print signed double, trailing space.

Side effects: None.

D . R

(d width --)

Print signed double, right-justified.

Side effects: No trailing space.

3.14 Base / Radix Control

BASE

(-- addr)

Variable holding the current number base.

Side effects: None.

DECIMAL

(--)

Set BASE to 10.

Side effects: None.

HEX

(--)

Set BASE to 16.

Side effects: None.

BINARY

(--)

Set BASE to 2. Not an ANS-standard word.

Side effects: None.

3.15 Exception Handling

CATCH

(xt -- 0 | n)

Execute xt, trapping any THROW.

Side effects: Restores data stack depth and HANDLER on either path.

THROW

(n --)

Raise exception n, or do nothing if n is zero.

Side effects: Nonzero: non-local exit to the matching CATCH, or JMP ABORT if none is active.

3.16 Comments

```
(
  ( "ccc<paren>" -- )
  Discard input up to the next close paren. IMMEDIATE.
```

Side effects: Works interpreting or compiling.

```
\
  ( "ccc<eol>" -- )
  Discard the remainder of the current input line. IMMEDIATE.
```

Side effects: Follows SOURCE, so it operates correctly inside EVALUATE.

3.17 Environmental & System Queries

ENVIRONMENT?

```
( addr len -- false | value true )
Query implementation-defined characteristics.
```

Side effects: Table is incomplete: missing /HOLD, /PAD, MAX-D, MAX-UD, WORDLISTS, FLOORED.

SOURCE

```
( -- addr len )
The current input source.
```

Side effects: Follows terminal input or an active EVALUATE string.

SOURCE - ID

```
( -- 0 | -1 )
Identify the current input source. 0 = terminal, -1 = string.
```

Side effects: None.

REFILL

```
( -- flag )
Attempt to refill the input source.
```

Side effects: Fails (false) if the current source is a string, per ANS.

EVALUATE

```
( addr len -- )
Interpret a string as Forth source.
```

Side effects: Saves and restores the whole input-source state around the call.

TIB

```
( -- addr )
The fixed terminal input buffer address.
```

Side effects: Always the terminal buffer, unaffected by EVALUATE.

#TIB

```
( -- addr )
Variable: count of valid characters in TIB.
```

Side effects: Always the terminal buffer's count.

>IN

```
( -- addr )
Variable: scan offset into the current input source.
```

Side effects: None.

SPAN

(-- addr)

Variable: EXPECT's returned character count.

Side effects: None.

BL

(-- char)

The blank/space character, 32.

Side effects: None.

3.18 Tools

.S

(--)

Print the data stack, top to bottom.

Side effects: Non-destructive.

WORDS

(--)

List every word in the dictionary.

Side effects: Walks the full ROM+RAM chain from LATEST.

DUMP

(addr u --)

Hex and ASCII memory dump, 16 bytes per line.

Side effects: None.

4. Forth Development Notes

This section is written for the audience who will program the completed system: users developing a Forth application on top of ClaudeForth, not those building or modifying the system itself. It assumes the base system already runs and focuses on what's needed to work productively within it day to day.

This section covers five practical aspects of working with the system that fall outside the word-by-word glossary: how serial communication is configured, a brief introduction to Forth syntax and stack operations, worked examples of the Tools word set, how to reclaim dictionary space during interactive development using MARKER, and why the optional Search-Order word set is not part of this implementation.

4.1 Serial I/O and Handshaking

Serial communication parameters — baud-rate divisor, word length, parity, and stop bits — are fixed by the hardware, not selectable from Forth. The 6850 ACIA's control register is written once, at COLDSTRT, to a constant value (CR_RXON, or CR_RXTX when the output ring has data queued) encoding a ± 16 clock, 8 data bits, no parity, 1 stop bit, and receive interrupts enabled. There is no word in this system that changes these settings at run time; the terminal or host at the other end of the link must be configured to match this fixed set of parameters independently.

Hardware (RTS/CTS) handshaking is implemented; software (XON/XOFF) is not.

RTS — this system's output telling the remote device when to pause sending — is toggled automatically based on the input ring buffer's fill level. IRQH's receive path calls RTSCHECKHI after storing each incoming byte; once the ring reaches INHIWATER (48 of its 64 bytes), RTS is driven high by writing CR_RTSHI to the ACIA control register, and RTS stays high until KEY, on the mainline side, drains the ring back down to INLOWATER (16 bytes) and calls RTSCHECKLO to drop it low again. The 32-byte gap between the two marks is deliberate hysteresis, avoiding RTS toggling right at a single threshold.

CTS — the remote device's own flow-control signal telling this system when it may transmit — needs no firmware logic at all: the 6850 ACIA inhibits TDRE in hardware whenever CTS is deasserted, and every transmit path in this system (EMIT, IRQH's TXCHK) already gates on TDRE, so CTS is honored automatically with no code changes required.

One real ACIA constraint shapes this design directly: the 6850's control register ties RTS state and transmit-interrupt-enable to the same two bits, so RTS-high and TX-interrupt-enabled can never be selected simultaneously. While RTS is asserted high, EMIT still queues outgoing bytes into the output ring but deliberately does not enable the transmit interrupt to drain them — RTSCHECKLO re-enables it once RTS drops low, restoring CR_RXTX if output is still queued at that point. A second, genuine race is worth naming rather than glossing over: RTSCHECKHI runs from inside the interrupt handler, where hardware has already masked further interrupts, but RTSCHECKLO runs from ordinary mainline code (KEY) and could otherwise be interrupted mid-decision by IRQH's own TXOFF path, which also writes the control register — RTSCHECKLO masks IRQ around its own critical section specifically to prevent that.

Software (XON/XOFF) handshaking remains unimplemented: this system still neither transmits XON/XOFF nor recognizes those bytes if a remote device sends them — an incoming \$11 or \$13 is treated as an ordinary character and queued like any other. On a genuine 3-wire serial link with no RTS/CTS wiring at all, that link would still have no flow control of any kind; the hardware path described above only helps when RTS/CTS are physically connected.

4.2 Forth Syntax and Stack Operations

Forth has no operators, no operator precedence, and no infix expressions. A line of input is simply a sequence of whitespace-delimited words, read and acted on strictly left to right. Each word is either a number, which is pushed onto the data stack, or a name that is looked up in the dictionary and executed immediately. Arithmetic and every other operation work by taking their arguments off the top of the stack and pushing their result back onto it — this is postfix, or reverse Polish, notation: the operator always comes after its operands, never between or before them.

A numerical example:

Computing $(3 + 4) \times 2 - 14$ — is written:

```
3 4 + 2 * .
14    ok
```

Read left to right, tracking the data stack after each word (shown deepest item first, top of stack last):

3	->	(3)	push 3
4	->	(3 4)	push 4
+	->	(7)	pop 4 and 3, push their sum
2	->	(7 2)	push 2
*	->	(14)	pop 2 and 7, push their product
.	->	()	pop and print: 14

The stack-effect notation used throughout the Glossary (Section 3) — for example, `+'s ( n1 n2 -- sum )` — describes exactly this: the items expected on the stack before a word runs, to the left of `--`, and what it leaves in their place afterward, to the right. Reading a word's stack effect this way is enough to know how to use it correctly without needing to know anything about how it is implemented internally.

This system does not require building up a vocabulary from nothing: a large set of predefined words — arithmetic, stack manipulation, comparison, control flow, string and memory operations, and more — already exists and is documented word by word in the Glossary. At the interactive prompt, executing `WORDS` lists every one of these preexisting words currently in the dictionary, from the most recently defined word back through every built-in primitive. See Section 4.3 for a worked example of `WORDS` and the other Tools words.

4.3 Tools Word Set: Explanation and Examples

The three Tools words in the Glossary — `.S`, `WORDS`, and `DUMP` — are interactive aids rather than words a program typically calls itself. The examples below show plausible session transcripts. One real characteristic of this system worth knowing before reading them: output from a word is written immediately, with no leading newline of its own — the newline seen before "ok" in a normal session comes from `QUIT`, not from the word that ran. The transcripts below add a line break before each result for readability; on the actual wire, output from `.S`, `WORDS`, or `DUMP` appears immediately after the typed command, on the same line.

`.S` — print the data stack, top to bottom, non-destructively

Pushes nothing and removes nothing; it only reads the stack from the current top down to `SP0` and prints each cell with the same signed formatting as `DOT`. This makes it safe to run at any point without disturbing whatever a definition is part-way through building on the stack.

```
1 2 3 .S
3 2 1   ok

\ top of stack (3, pushed last) prints first, deepest
\ item (1, pushed first) prints last
```

`WORDS` — list every word in the dictionary

Walks the `LATEST` chain from the most recently defined application word all the way back through every ROM primitive, printing each name separated by a space. With roughly 150 application-level words already in the Glossary plus every ROM-resident primitive behind them, real output is long — the excerpt below shows the shape of it rather than a complete transcript.

```
WORDS
SQUARE FENCE VALUE TO VARIABLE CONSTANT DOES>
CREATE ... DUP DROP SWAP OVER ROT DEPTH ...
+ - * / MOD ... IF THEN ELSE BEGIN ... ok

\ most-recently-defined application words appear first,
\ since the walk follows LATEST backward through LINK;
\ ROM primitives make up the remainder of the list
```

`DUMP` — hex and ASCII memory dump, 16 bytes per line

Takes an address and a byte count, and prints that many bytes, 16 per line: each byte as two hex digits, then a matching ASCII column with unprintable bytes shown as a period. Worth noting explicitly since it differs from many `DUMP` implementations: this one does not print a leading address on each line — only the hex bytes and their ASCII rendering — so the caller must track which address a given line corresponds to separately if that matters.

```
HEX
7000 10 DUMP
BD E4 32 8E 00 00 39 00 00 00 00 00 00 00 00 00  .....9.....

\ 16 bytes shown starting at $7000 (the start of APPCODE);
\ actual bytes depend entirely on what has been compiled
\ into code space by the time DUMP is run
```

4.4 Dictionary Management with MARKER

ANS Forth provides no general "undefine" word; instead, the standard's mechanism for reclaiming dictionary space is MARKER. Executing MARKER with a name creates an ordinary dictionary entry that, when later executed itself, forgets its own definition and every definition made after it — restoring the dictionary to exactly the state it was in at the moment MARKER was invoked. This is the standard way to experiment interactively without permanently accumulating discarded or superseded definitions.

In this implementation, a word created by MARKER stores a snapshot of all three region pointers — DPHERE (dictionary), CODEHERE (code), and VARHERE (mutable variable space) — plus LATEST, at the moment of its own creation. Executing the marker word (DOMARKER) simply writes those four saved values back, in one step. Nothing needs to be erased or unwound word by word: every allocation word in the system (comma, ALLOT, HEADER, and their VARHERE counterparts) only ever appends at the current pointer, so rolling the pointer back alone is sufficient to make everything defined afterward unreachable and its space available again for reuse.

Example:

```
MARKER FENCE
: SQUARE DUP * ;
5 SQUARE .           \ prints 25

FENCE                 \ forgets FENCE itself, and SQUARE

5 SQUARE .           \ throws -13, undefined word: SQUARE
```

MARKER FENCE creates FENCE and captures the current DPHERE/CODEHERE/VARHERE/LATEST as its own PFA contents. SQUARE is then compiled normally, advancing all three pointers past that snapshot. Executing FENCE restores the four saved values, which puts LATEST back to whatever it was immediately before FENCE was defined — so SQUARE is no longer reachable by FIND, and FENCE itself is gone too, along with the dictionary, code, and variable space both of them occupied. A second MARKER call after this point would need a new name, since FENCE no longer exists to be reused.

***Caveat:** as noted in the glossary, a MARKER's snapshot only covers the single dictionary chain this system currently maintains. It is not defined against the optional Search-Order word set, which this implementation does not include — see Section 4.5.*

4.5 The Search-Order Word Set Is Not Included

ANS Forth's optional Search-Order word set (FORTH-WORDLIST, WORDLIST, GET-CURRENT, SET-CURRENT, GET-ORDER, SET-ORDER, ALSO, ONLY, PREVIOUS, DEFINITIONS, and SEARCH-WORDLIST) is not implemented in this system. Every one of the roughly 150 words in the Glossary is resolved through a single, unified dictionary chain rooted at LATEST — there is only ever one wordlist, and new definitions always link into it. Search-Order's whole purpose is letting a program maintain several independent wordlists (for example, to keep an application's vocabulary separate from the words used only to build it) and to search a chosen, ordered subset of them; without it, every word ever defined is always visible to FIND, in one flat namespace, for the lifetime of the system.

This was a foundational decision, not an oversight discovered late: the single-chain design was fixed from the very first system-initialization code, where LATEST is set to the top of the ROM dictionary at cold start and every application word links onto that same chain afterward. Adding Search-Order now would not be a small addition — it would touch code that already exists throughout the system, not just introduce new words. FIND itself would need restructuring into an outer loop over an active search order, trying each wordlist in turn; HEADER, which every defining word (colon definitions, CREATE, VARIABLE, CONSTANT, VALUE, and the rest) already funnels through, would need to link new definitions into a selectable "current" wordlist rather than unconditionally into LATEST; and a few words that deliberately read LATEST directly today — RECURSE, the unsmudging step in semicolon, and WORDS — would each need the same substitution or they would silently keep operating on the wrong wordlist once more than one existed. MARKER's snapshot would also need to widen to cover every wordlist's head cell, not just LATEST, with the further complication that ANS itself leaves a marker's interaction with wordlists created after it as implementation-defined, not fully specified.

In practice, this means every program written against this system lives in one shared, flat namespace — there is no mechanism to hide a helper word's name from being visible everywhere else, and no way to have two differently-scoped words share the same name. For the scope of a single-user, single-application embedded system, this is a reasonable simplification rather than a defect; it is flagged here specifically so it is not mistaken for an accidental gap in ANS coverage.

5. Open Items Checklist

Everything below was explicitly flagged during the build as incomplete, unverified, or deliberately deferred. Nothing here is a surprise — each item was named at the point it came up. This is a consolidated list to work from, not a new set of findings. Regenerated to reflect fixes and design decisions made since the original version.

5.1 Known bugs — resolved since the original checklist

- ✓ **DUMP's partial-final-line bug** — fixed via DUVALID tracking (`25_tools_word_set.asm`). The ASCII column no longer reads past the valid bytes on a short final line.
- ✓ **SUBSTITUTE** — completed. It now performs a real bounds-checked copy (prefix / replacement / suffix) via a shared SUBCOPY helper, reusing SEARCHW for the match. Still scoped to a single registered name/value pair, not a full table — see "Open design questions" below; that scope limit is a deliberate simplification, not a bug.
- ✓ **VALUE's PFA** — moved from CODEHERE (immutable space) to VARHERE (mutable space), matching VARIABLE's pattern, so every T0-driven update now writes into the correct region instead of into code space. Not on the original checklist (found and fixed after this list was first generated); logged here for the record.
- ✓ **Every scratch/global cell now has a real RMB-assigned address in page zero** (`00_memory_map_and_globals.asm`), applied rather than left as a placeholder. Two real findings came out of doing this: the budget is **253 of 256 bytes used — only 3 bytes of headroom** in the GLOBALS page, and SNEND (documented in the original placeholder list) turned out to be genuinely dead — never read or written anywhere — and was dropped rather than given an address. Any future scratch cell added to this system will need to fit in that remaining 3 bytes or the page-zero fast-addressing property (DP = \$00) stops covering it.
- ✓ **The ROM base dictionary is now real** (`27_forth_dictionary.asm`) — every primitive with actual code got a real header, chained via LINK, CFA pointing straight at its code label. ABORTHDR's LINK field, a placeholder 0 since it was first built, is now resolved to the chain's newest entry. Building this surfaced two real findings, not just mechanical work: the original 1024-byte BASEDICT could not hold the header table (ultimately 1954 bytes needed, once DOES> was added — see below), so it was resized to 2048 bytes, taking the space from BASECODE (now 6080 bytes, was 7104) — a resize based purely on the header-table budget, not on any actual measurement of BASECODE's assembled size, which has never been checked with a real 6809 assembler. Generating this table also surfaced that **DOES> had no corresponding code anywhere in the file** — resolved in a follow-up pass, see below.
- ✓ **DOES> — resolved.** SETDOES (the patching runtime) was added beside DODOES/DOESRT0, and DOES> itself (code label DOESGT, since a literal ">" is not a valid 6809 assembler label) was added right after CREATE. DOESBEH was added to the GLOBALS layout to support it, using 2 of the page's last 3 free bytes — only 1 byte of headroom now remains in page zero. DOES> also has a real dictionary header now (H_DOESGT, the chain's newest entry); ABORTHDR's LINK was updated again to point at it.
- ✓ **Four internal loop-label names were reused across unrelated routines: FLOOP, UDLOOP, DRPOS, CMLOOP.** Resolved — each pair renamed to a distinct name scoped to its own routine: FIND's FLOOP → FFLOOP; FILLW's FLOOP → FILLLOOP; UDIV16's UDLOOP → UD16LOOP; UDDIGIT's

UDLOOP → UDDLOOP; DIVCOMMON's DRPOS → DCRPOS; DDOTR's DRPOS → DDRPOS; CMOVEW's CMLOOP → CMVLOOP; COMPAREW's CMLOOP → CMPLOOP. Verified zero duplicate labels and zero stray references to any of the old names remain anywhere in the file.

- ✓ **Hardware (RTS) serial handshaking — implemented.** IRQH's receive path and KEY now toggle RTS based on the input ring's fill level (INFILL, RTSCHECKHI, RTSCHECKLO against INHIWATER=48/INLOWATER=16, with hysteresis between them) via a new CR_RTSHI control byte and RTSSTATE flag — the last free byte in the GLOBALS page, which is now fully packed at 256/256. CTS (the other handshaking direction) needed no firmware logic at all: confirmed against the 6850 datasheet that TDRE is automatically inhibited while CTS is deasserted, so this system's existing TDRE-gated transmit logic already respects it. A real race was identified and closed: mainline code (KEY, via RTSCHECKLO) and the ISR (IRQH's TXOFF) can both decide to write the control register, so RTSCHECKLO masks IRQ around its critical section and TXOFF checks RTSSTATE before writing. Software (XON/XOFF) handshaking remains unimplemented — see below.
- ✓ **TRUE and FALSE — implemented.** Surfaced while organizing the ANS test suite: neither existed as an actual dictionary word, only as the internal TRUEV/FALSEV assembler constants. Added as `CONSTANT TRUE -1 / CONSTANT FALSE 0` (TRUEBODY/ FALSEBODY, section 26; headers H_TRUE/H_FALSE, chain's two newest entries, section 27) — the first CONSTANT-pattern ROM-resident words in this system. Every other ROM word's CFA is a plain code label; a CONSTANT's CFA is the DODOES-trampoline pattern instead, built here with fixed, assemble-time addresses rather than a CODEHERE snapshot, since there is no interactive CREATE/CONSTANT phase for ROM content.
- ✓ **One genuine 6309-only instruction (TSTD) was in use — fixed.** Found by auditing every mnemonic in the file against the real 6809 instruction set, rather than trusting the Build Instructions section's earlier claim that no 6309 extensions were used (that claim was wrong until this fix). TRYNUM (section 9) used TSTD with no operand to test whether D was zero after PULU D — PULU/PULS don't affect condition codes on genuine 6809 hardware, unlike a load, so this relied on a 6309-only convenience instruction. Replaced with `CMPD #0`, a real 6809 instruction with identical behavior for this purpose. A full mnemonic audit after the fix found zero remaining non-6809 instructions anywhere in the file.
- ✓ **BASELATEST was an undefined symbol — a real assembly-breaking bug, not a placeholder.** COLD (section 4) has referenced `LDD #BASELATEST` to initialize LATEST since this file's earliest version, and the SECTION 27 header comment has long asserted "BASELATEST remains QUITHDR" - but no EQU or label actually named BASELATEST existed anywhere in the file. This would have failed to assemble outright. Fixed by adding `BASELATEST EQU QUITHDR` directly after QUITHDR's own definition (section 26), matching what the comment always said it should equal. Verified: exactly one definition, no duplicate symbols, and COLD's forward reference to it resolves under standard two-pass assembly.
- ✓ **JSR CR (bare, undefined) appeared at five call sites — a real assembly-breaking bug, not a naming inconsistency.** The actual routine has always been labeled CRW (section 22), following this file's own convention of giving short/symbolic ANS names a distinct code label. ABORT, QUIT's error-report path, QUIT's ok-prompt path, WORDS (WWDONE), and DUMP (DULEND) all called the bare, undefined CR instead. Fixed at each site to `JSR CRW`, preserving each call site's original spacing exactly. Verified: CRW defined exactly once, referenced 7 times total, zero remaining bare-CR instruction operands anywhere in the file (the two harmless bare "CR" occurrences that remain are a

section-title comment and the dictionary header's FCC "CR" name string, neither of which is a bug).

- ✓ **Two ALU instructions used invalid register-to-register syntax (ADDA B, EORA B) — the 6809 has no such addressing mode, so the assembler read B as an undefined direct-page symbol.** Found in the single-cell multiply routine (ADDA B, twice) and DOPLUSTEST's sign comparison for +LOOP (EORA B). Fixed with the standard 6809 idiom for combining two registers: PSHS B followed by ADDA , S+ / EORA , S+ (push B, then operate through the auto-incrementing stack-indexed operand). Verified: a full sweep of every ALU instruction (ADDA/ADDB/SUBA/SUBB/ ANDA/ANDB/ORA/ORB/EORA/EORB/CMPA/CMPB/ADCA/ ADCB/SBCA/SBCB/BITA/BITB) against a bare A/B operand found zero remaining instances; every other bare B in the file (in STA/LDA/LEAX . . . , X and PSHS B) is genuine, valid 6809 accumulator-offset indexed addressing or register-list syntax.
- ✓ **Nine scratch symbols (MRESULT, MVCNT, MVDST, MVSRC, FILLCHR, FILLCNT, FILLADDR, HLEN, HSADDR) were used throughout MOVE/CMOVE/CMOVE>, FILL, HOLDS, and the single-cell multiply routine but never declared anywhere — a real, assembly-breaking gap.** Auditing every one of the 142 already-declared GLOBALS cells for actual use found zero dead cells to reclaim this time (unlike SNEND earlier), and the GLOBALS page was independently confirmed at exactly 256/256 bytes with no headroom. Since MOVE-family, FILL, HOLDS, and the multiply routine never call each other or run concurrently in this single-threaded interpreter, they now share physical storage: three new cells (MVCNT, MVDST, MVSRC) hold the real storage, and FILLCNT/HLEN alias MVCNT, FILLADDR/HSADDR alias MVDST, and MRESULT/FILLCHR alias MVSRC, all via EQU. These three new cells could not fit in the full GLOBALS page, so they live at \$0100, carved from the front of USER0 (shrunk from 128 to 122 bytes, now starting at \$0106) — a real, documented tradeoff: these three cells use ordinary extended addressing, not direct-page, including inside CMOVEW's per-byte copy loop.
- ✓ **JSR SPACE (bare, undefined) — same class of bug as CR, fixed the same way.** WORDS called the bare, undefined SPACE instead of the actual routine label SPACEW (section 22). Fixed to JSR SPACEW. Checked SPACES for the same pattern (none found — SPACESW correctly calls EMIT directly) and swept the file for any other bare SPACE instruction reference (none remain; the two harmless bare "SPACE" occurrences left are a section-title comment and the dictionary header's FCC "SPACE" name string).
- ✓ **Three short branches (BHS, BEQ, BNE) overflowed the 8-bit short-branch range (±127 bytes) — a real assembler error, not a style issue.** SUBCOPY's overflow check (BHS SUBOVERFLOW, ~87 source lines to target), DUMPW's per-line loop test (BEQ DUDONE, ~76 lines), and DUMPW's loop-back (BNE DULINE, ~76 lines) all spanned too much code for an 8-bit displacement. Fixed by converting each to its long-branch equivalent (LBHS/LBEQ/LBNE), which uses a 16-bit displacement (±32767 bytes) - functionally identical, just not range-limited.
- **Ten more short branches have a large (41-51 source line) span to their target and were not individually confirmed safe.** Found by a heuristic sweep (line-count as a rough proxy for byte distance, since no real assembler is available in this environment to get an exact count) after fixing the three confirmed overflows above, all of which spanned 76+ lines - these ten are meaningfully shorter but not verified within range: QUERY's QLOOP (three branches, lines 579/582/589), FIND's NOTFOUND/FFLOOP (lines 1210/1256), ACCEPT's ALOOP/ADONE (lines 1501/1462), UNESCAPE's UEDONE/UELOOP (lines 3888/3931), and EMPTY (line 1153). None of these were reported as failing

and none have been changed; if a real assembler flags any of them, the fix is the same: convert to the matching LBXX long-branch form.

- ✓ **ORG BASECODE was missing entirely — a fundamental placement bug, not a cosmetic gap.** VECTORS (\$FFF0) and INIT (\$FFC0) both had their own ORG, but SECTION 3 (IRQH) — and every routine through SECTION 26, i.e. nearly the entire interpreter — had no ORG placing it in BASECODE at all. Without it, this code would have continued growing from wherever INIT's WARM message left the location counter, inside INIT's own 48-byte \$FFC0-\$FFEF budget, overflowing directly into VECTORS rather than landing in BASECODE (\$E800-\$FFBF) anywhere. Fixed by adding ORG BASECODE immediately before SECTION 3 begins.
- **Adding the missing ORG makes BASECODE's byte budget concretely checkable for the first time — and a rough estimate suggests it may not fit.** This was previously flagged only as "unverified" (no real assembler has ever been run against this file); with a real ORG boundary now in place, a heuristic per-instruction byte count (inherent/direct-page/indexed/ immediate/extended opcode sizes, correctly distinguishing direct-page GLOBALS operands from extended ones) over every instruction from SECTION 3 through SECTION 26 estimates roughly **8660 bytes against a 6080-byte budget — about 2580 bytes over**. This is a rough approximation, not a real assembler's output, and could be wrong in either direction, but it's a strong enough signal to treat as a real, likely problem rather than a formality. Not resolved here — resizing BASECODE/BASEDICT again would cascade into the memory map, the documentation, and the SVG diagram, and a real LWASM pass would give a far more reliable number than this estimate before deciding how much room is actually needed.
- ✓ **USER0 and USER1 (250 bytes of unallocated reserve space, never read or written by any code) were deleted, and the region from MVSCRATCH through APPDICT was made fully contiguous.** SERBUF, INBUF, OUTBUF, TIBBUF, WORDBUF, and SIBUF all shifted down to sit immediately after MVSCRATCH (\$0106) with no gaps between any of them. This also closed a separate, pre-existing 11-byte gap between WORDBUF and SIBUF (\$02F5-\$02FF) that had nothing to do with USER0/USER1 but was caught while verifying the region was genuinely contiguous end to end. APPDICT was extended downward to close the resulting gap too (new start \$021B, was \$0320) — its end (\$6EFF) is unchanged, so this is a pure 261-byte gain in application dictionary space, not a resize of its budget. This was an interpretation of "contiguous," not something explicitly asked for beyond the 6 buffers themselves; flagged here in case a fixed APPDICT start was actually intended instead. Verified: zero duplicate symbols, zero remaining references to USER0/USER1 anywhere in the code, and the full GLOBALS->MVSCRATCH->buffers->APPDICT span checked programmatically for zero gaps.
- ✓ **APPVARS and APPDICT swapped positions - APPVARS now sits below APPDICT.** APPDICT moved from \$021B to \$031B (still ending at \$6FFF, directly below APPCODE); APPVARS moved from \$6F00 down to \$021B (same 256-byte size, now sitting directly above the buffers instead of directly below APPCODE). Checked before making the change: only two code references exist for either symbol (COLD initializing VARHERE/DPHERE to each region's start) and neither depends on their relative order, so the swap was safe. Verified: zero duplicate symbols, the whole region from GLOBALS through APPCODE's start remains contiguous with zero gaps, dictionary chain unaffected (219 entries, since this change doesn't touch it at all).
- ✓ **APPVARS grown from 256 to 8000 bytes, taking the space directly from APPDICT.** APPVARS now spans \$021B-\$215A (start address unchanged); APPDICT shrank by the same 7744 bytes to \$215B-\$6FFF (end unchanged, still directly below APPCODE), giving 20133 bytes of application dictionary space, down from 27877. Checked before making the change: still only two code

references to either symbol (COLD's VARHERE/ DPHERE initialization), neither size-dependent. Verified: zero duplicate symbols, APPVARS size is exactly 8000 bytes, the whole region stays contiguous end to end, dictionary chain unaffected.

- ✓ **ACIA (the 256-byte memory-mapped I/O block) renamed to INOUT, and the actual 6850 ACIA chip's registers moved to INOUT+8 instead of the block's base.** The block still spans the same 256 bytes (\$DF00-\$DFFF); it's now modeled as a general I/O region with the ACIA chip occupying one small part of it (ACIACR/ACIASR = INOUT+8 = \$DF08, and the data register ACIADR at \$DF09), leaving INOUT+0..INOUT+7 free for other memory-mapped devices sharing the block. Doing this rename surfaced a real, previously-hidden bug: COLDSTRT's ACIA reset sequence used STA ACIA (the block's base) instead of STA ACIACR (the control register) in two places - this only ever worked because ACIA and ACIACR happened to be the same address in the old scheme. Once ACIACR moved to INOUT+8, writing to the bare block base would have silently stopped resetting the ACIA at all. Fixed both to STA ACIACR. Verified: INOUT defined once, the bare ACIA symbol (at the time) no longer existed anywhere, and every other ACIA register access in the file already used the correctly-named register symbols rather than the bare block name.
- ✓ **ACIA reintroduced as an explicit base symbol (ACIA EQU INOUT+8), with ACIACR/ACIASR now defined relative to it (EQU ACIA) rather than directly to INOUT+8.** The data register was briefly renamed to ACIRDR in the same pass, then reverted back to ACIADR immediately afterward once flagged as a typo - both code sites (IRQH's receive and transmit paths) were updated each time the name changed. Verified: each of ACIA/ACIACR/ACIASR/ACIADR defined exactly once, resolving to the expected values (ACIA = INOUT+8, ACIACR/ACIASR = ACIA, ACIADR = ACIA+1), zero remaining references to ACIRDR anywhere, zero duplicate symbols, dictionary chain unaffected.
- ✓ **INITEND/INITSIZE landmark constants added at the true end of the INIT block (INITEND EQU *, INITSIZE EQU *-COLDSTRT), right after WARMMSGL.** The request referenced a COLDSTART label, which doesn't exist in this file - used the actual existing label COLDSTRT instead of introducing a new undefined symbol.
- ✓ **INITEND's comment now states the invariant explicitly ("value should match vector ORG") - and checking it confirms it currently holds.** The precise instruction-by-instruction byte count from when the VECTORS collision was first found (78 bytes exactly, COLDSTRT through WARMMSGL) gives INITCODE start (\$FFA2) + 78 = \$FFF0, exactly matching VECTORS' ORG. Zero gap, zero overlap - unlike the still-open INITCODE/BASECODE overlap at the other end of this same region, which remains a real problem. Same caveat as always: this is a manual count, not a real assembler's output, so INITSIZE (computed automatically at assembly time) is the figure to trust once this file is actually assembled.
- ✓ **BASECODESTRT/BASECODEEND/BASECODESIZE and BASEDICTSTRT/BASEDICTEND/BASEDICTSIZE landmark constants added, matching the INITEND/INITSIZE pattern - and checking the two stated invariants gives one clean confirmation and one confirmation of an already-known problem.** BASEDICTEND (\$D85D + the exact 1973-byte dictionary content = \$E012) matches ORG BASECODE (\$E012) precisely - this boundary is genuinely correct, not just close. BASECODEEND, using the same rough heuristic estimate flagged much earlier (~8660 bytes, never confirmed with a real assembler) starting from BASECODESTRT (\$E012), lands around \$101E6 - not only failing to match ORG INITCODE (\$FFA2) as the comment expects, but overflowing past \$FFFF entirely on that estimate. The exact amount of room actually available between BASECODESTRT and INITCODE (\$FFA2 - \$E012 = 8080

bytes) is itself less than the ~8660-byte estimate, meaning even filling every available byte up to INITCODE with zero gap would still likely fall short. This is the same underlying problem already tracked elsewhere (BASECODE possibly not fitting its budget, and separately the INITCODE/BASECODE overlap), now independently reachable via these new landmarks once the file is actually assembled, rather than a new, different issue.

- **Follow-up: a real instruction-by-instruction count (not the rough heuristic above) puts BASECODE's actual size at 7984 bytes (\$1F30), 96 bytes (1.2%) short of a target assembler- listing value of \$1F90 (8080 bytes) given for comparison.** Built a mnemonic-and-addressing-mode-aware counter distinguishing inherent/immediate/direct/extended/indexed forms, correct prefix requirements (\$10/\$11 for LDY/STY/LDS/STS/CMPD/ CMPY/CMPU/CMPS), and true direct-page symbols (only \$0000-\$00FF, matching DP) - a meaningfully more rigorous pass than the original ~8660-byte estimate. Every one of the 3751 instructions in the region was classified (zero "unrecognized" remaining after fixing early misses like LSLB/ LSLA as ASLB/ASLA synonyms). Checked for likely sources of the remaining 96-byte gap before accepting it: zero indexed operands use an offset outside the 5-bit range that would need extra encoding bytes (all 125 numeric-offset operands found are small, e.g. 1, X/-1, Y), and the two apparent "indirect []" matches were a false positive (a section-title comment, not real indirect addressing). The target value \$1F90 is notable in its own right: BASECODE + \$1F90 = \$FFA2 exactly, meaning if the real assembled size actually matched it, BASECODE and INITCODE would be perfectly contiguous with zero gap and zero overlap - which would also resolve the separate, still-open INITCODE/BASECODE overlap noted elsewhere. My count doesn't confirm that, though it's close (1.2% off). This is still not a real assembler's output - the standing caveat throughout this file - and the gap could come from encoding edge cases a manual model can approximate but not guarantee against.
- ✓ **ROMSTRT/ROMEND added as the outer ROM boundary (\$C100/VECTORS - 1), and INITCODE/BASECODE/BASEDICT verified contained within it - all three pass.** ROMEND was initially defined as \$10000 ("one past VECTORS's end," a symbolic value that doesn't fit as a real 16-bit address), then corrected to VECTORS - 1 (\$FFEF, one before VECTORS's start) - a genuine 16-bit address usable directly in comparisons or as a memory operand, not just symbolic arithmetic. Re-checked after the correction rather than assumed still valid: INITCODE (\$FFA2-\$FFEF), BASECODE (\$E012-\$FFBF), and BASEDICT (\$D85D-\$E011) are all still genuinely within ROMSTRT . . ROMEND - INITCODE's own end now lands exactly on the new boundary, a tighter and more meaningful check than before, not a violation. This containment check passes cleanly, independent of the other open problems below.
- ✓ **ROMSTRT/ROMEND renamed to USROMSTRT/USROMEND, each with a short "Usable ROM start"/"Usable ROM end" comment.** Cosmetic, not functional - neither symbol was referenced by any code, only by nearby comments (three sites, all updated: the pair's own definitions and INOUT's cross-reference). Verified: zero remaining bare ROMSTRT/ROMEND references anywhere, zero duplicate symbols, dictionary chain unaffected.
- ✓ **VECTOREND/VECTORSIZE landmark constants added at the true end of the vector table, matching the INITEND/INITSIZE pattern - and checking the stated invariant confirms it exactly, not approximately.** Unlike INITEND/BASECODEEND (which needed a manual, approximate instruction-by-instruction byte count since real 6809 instructions have variable-length encoding), the vector table is pure FDB data with a fixed, unambiguous 2-byte width per entry - counting the 8 entries (VRESV through VRESET) gives exactly 16 bytes, matching the comment's stated \$10 expectation precisely. Verified: zero duplicate symbols, dictionary chain unaffected.

- ✓ **BASECODESTRT removed as requested - but BASECODESIZE depended on it, so removing it alone would have left a genuine dangling undefined-symbol reference, the same bug class found and fixed several times earlier in this file.** Confirmed first that BASECODESTRT was numerically identical to BASECODE itself (only comments sit between ORG BASECODE and where BASECODESTRT was defined, zero emitted bytes), then fixed BASECODESIZE to read BASECODEEND - BASECODE directly instead - matching the same pattern just applied to INITSIZE/INITCODE, not a new approach invented for this case. Verified: zero remaining BASECODESTRT references anywhere, BASECODESIZE resolves cleanly, zero duplicate symbols, dictionary chain unaffected.
- ✓ **BASEDICTSTRT removed the same way, for the same reason - BASEDICTSIZE depended on it too.** Same fix pattern as BASECODESTRT immediately above, applied to its counterpart: confirmed BASEDICTSTRT was numerically identical to BASEDICT (only ORG BASEDICT sits before it, zero emitted bytes), removed it, and repointed BASEDICTSIZE to BASEDICTEND - BASEDICT directly. Verified: zero remaining BASEDICTSTRT references anywhere, BASEDICTSIZE resolves cleanly, zero duplicate symbols, dictionary chain unaffected.
- ✓ **INOUT moved from \$DF00 to \$C000 (with INOUTEND EQU INOUT+\$FF added immediately after), and every ACIA-related address verified between the two - also passes.** ACIA/ACIACR/ACIASR (\$C008) and ACIADR (\$C009) all fall within INOUT-INOUTEND (\$C000-\$C0FF), checked numerically. Confirmed there is no other memory-mapped I/O device anywhere in this file besides the ACIA family - nothing else needed checking. This move has a real, positive side effect worth naming clearly: it resolves the INOUT portion of the three-way collision flagged when BASEDICT moved to \$D85D two turns ago (INOUT no longer overlaps BASEDICT, since \$C0FF < \$D85D). The DSTACK and RSTACK portions of that same collision were untouched by this specific change, but have since been resolved separately - see below.
- ✓ **INOUT's new collision with APPCODE is resolved, and so is the rest of the original BASEDICT collision - RSTACK, DSTACK, CODETOP, APPCODE, and APPDICT all moved down \$2000.** RSTACK (\$BC00-\$BEFF) and DSTACK (\$B800-\$BBFF) no longer overlap BASEDICT at all - the three-way collision first found when BASEDICT moved to \$D85D is now fully resolved (INOUT resolved it two turns ago, this resolves the remaining two thirds). APPCODE's move (\$5000-\$B7FF) also separately resolves its overlap with INOUT from the previous entry, as a side effect of the same shift, not a second fix. Re-verified with a full pairwise sweep after the move, not assumed: the only overlap remaining from the four found last turn is the original, unrelated INITCODE/BASECODE one (30 B) - INITCODE/ BASECODE/BASEDICT/RSTACK/DSTACK/INOUT/APPCODE are all now mutually clean.
- ✓ **APPDICT's collision with APPVARS is now resolved - APPVARSEND changed from a static APPVARS+8000 to APPDICT - 1, making it self-derive from wherever APPDICT actually sits instead of a fixed size.** APPVARSEND is now \$1FFF (was \$215B), giving APPVARS 7653 bytes of actual usable space (down from the originally-intended 8000 - the 347-byte reduction exactly matches the overlap this closes). APPVARS's own EQU still starts at \$021B and its comment still cites the historical "8000 bytes" intent, now explicitly marked as the original intent rather than the current usable size. Functionally, APPVARS's enforced range (\$021B-\$1FFF) no longer reaches APPDICT (\$2000-\$6EA4) at all - checked numerically, not assumed. This also makes the boundary self-correcting: if APPDICT moves again, APPVARSEND moves with it automatically, rather than needing another manual fix like the last several turns' worth of address changes required. VUNUSEDW's own code is unchanged - it already computed against APPVARSEND (see below), so

this fix took effect purely by changing what that symbol means. Verified: zero duplicate symbols, dictionary chain unaffected.

- ✓ **DSTACK moved up \$200 (to \$BCFF), resolving both the CODETOP/DSTACK mismatch and the RSTACK/DSTACK gap from last turn in one move.** DSTACK's new occupied bottom (\$B900, from its \$BCFF top and 1024-byte size) now matches CODETOP (\$B900) exactly - APPCODE's nominal ceiling no longer overstates safe growth room, and the 512-byte overlap that created between APPCODE's nominal range and DSTACK's true range is gone. As a bonus, not something separately requested: DSTACK's new top (\$BCFF) is now exactly contiguous with RSTACK's bottom (\$BD00), closing the 512-byte gap that had opened between them too. Verified with a full pairwise sweep after the move: exactly two overlaps remain in the current memory map - the still-open APPDICT/APPVARS one (347 B) and the original, unrelated INITCODE/BASECODE overlap (30 B) - down from three last turn.
- ✓ **The VECTORS collision found by verifying INITEND is now resolved - INIT's ORG moved from \$FFC0 to \$FFA0.** Widened INIT from 48 to 80 bytes (\$FFA0-\$FFEF), comfortably covering the ~78 bytes COLDSTRT+WARM+WARMMSG actually needs (2 bytes of margin) - still an estimate from manual byte-counting, not a real assembler's output. INIT's own ORG was also converted from a literal \$FFC0 to symbolic ORG INIT, matching BASECODE/BASEDICT's convention, so the EQU and the actual placement can no longer drift apart the way BASECODE's did before that was caught. One real, unresolved interaction worth naming: INIT's new start (\$FFA0) is 32 bytes below the old documented BASECODE end (\$FFBF), tightening the still-open BASECODE overflow risk below by that much further - not a new problem in kind, since that risk was already estimated at roughly 2580 bytes over budget, dwarfing this additional 32-byte reduction, but worth tracking alongside it rather than treated as independent.
- **INIT moved again, from \$FFA0 to \$FFA2 (request contained a &FFA2 typo - used the correct \$ hex prefix instead).** INIT is now exactly 78 bytes (\$FFA2-\$FFEF), matching the COLDSTRT+WARM+WARMMSG byte-count estimate with zero margin (was 80 bytes, with 2 bytes of slack). This is a genuinely tighter fit than before, worth naming as a real tradeoff: any inaccuracy in the manual byte count, or any future addition to COLDSTRT/WARM, now has zero room to absorb. The overlap with BASECODE noted above is **not resolved by this change, only reduced** - from 32 bytes to 30 bytes (\$FFA2-\$FFBF). This was a real, pre-existing problem before this turn touched anything (\$FFA0 already overlapped BASECODE's declared end by 32 bytes), not something newly introduced. Not fixed here, in either direction (shrinking BASECODE further or moving INIT above it instead of below): both are bigger architectural decisions than "change one EQU," and a real assembler run would settle the actual BASECODE byte count this depends on.
- ✓ **INIT renamed to INITCODE** (EQU and its ORG reference, both in the memory-map constants). The section-title comment ("SECTION 2: INIT CODE") and one historical narrative comment describing a past bug scenario (with the byte figures true at that time, not today) were deliberately left referring to "INIT" as plain English/history, not the symbol - renaming a symbol doesn't obligate rewriting prose that correctly describes what was true before the rename existed. Doing this rename surfaced that the documentation's "ROM Size Required" section (Section 2) had gone stale independent of the rename itself: it still asserted the four ROM regions were contiguous at exactly 8192 bytes, which stopped being true as soon as INIT moved to \$FFA2 last turn and started overlapping BASECODE. Rewritten to state the overlap plainly instead of the now-false claim. Verified: INITCODE defined exactly once, zero duplicate symbols, dictionary chain unaffected.

- ✓ **BASEDICT and BASECODE addresses applied exactly as requested, and the documented ROM requirement widened to a 16K part - but this surfaced a severe, unresolved collision, not a minor tradeoff.** BASEDICT moved from \$E000 to \$D85D - verified before making the change that \$E012 - \$D85D = 1973 bytes exactly matches SECTION 27's real, measured dictionary content, a genuine zero-padding exact fit (was 2048 bytes, 75 bytes of slack). BASECODE moved from \$E800 down to \$E012 to match; its own upper bound was not given and stayed at \$FFBF, growing it from 6080 to 8110 bytes.
- **BASEDICT's new range (\$D85D-\$E011) entirely overlaps three other live regions: DSTACK (931 of its 1024 bytes, \$D85D-\$DBFF), RSTACK (all 768 bytes, \$DC00-\$DEFF), and INOUT (all 256 bytes, \$DF00-\$DFFF).** Found by checking the new address against every other region's boundaries before updating the documentation - not caught until then. This is a fundamental, system-breaking conflict, not a byte-budget tightness issue like the INITCODE/BASECODE overlap noted elsewhere: as configured, the ROM dictionary, the data stack, the return stack, and the ACIA's registers would all be mapped to the same physical addresses simultaneously, which cannot work on real hardware without bank switching (which this system does not have). The requested EQU values were applied exactly as given, since that's what was asked; resolving the collision itself was not attempted here, since it requires a decision this response can't make alone - moving DSTACK/RSTACK/INOUT elsewhere, choosing a smaller BASEDICT/BASECODE split that doesn't reach down this far, or reconsidering whether \$D85D was the address actually intended. The documented ROM part was still widened from 8K to 16K per the request (real total usage of BASEDICT+BASECODE+INITCODE+VECTORS alone, ignoring the collision, is about 10.15K, leaving roughly 6.2K of spare capacity within a 16K×8 EPROM), but that figure is secondary to the collision above. Verified: zero duplicate symbols, dictionary chain unaffected (219 entries, since this doesn't touch it).
- ✓ **The INITCODE/BASECODE overlap - open since it was first found, mentioned in at least five separate entries above across several turns - is finally resolved. BASECODE and BASEDICT both shifted down exactly 30 bytes** (BASECODE: \$E012 -> \$DFF4; BASEDICT: \$D85D -> \$D83F), chosen precisely: 30 bytes is exactly the overlap amount, so BASECODE's nominal end (\$FFA1, unchanged 8110-byte budget) now lands exactly one byte below INITCODE's start (\$FFA2) - zero gap, zero overlap. BASEDICT shifted the same amount to stay perfectly contiguous with BASECODE's new start, preserving its own exact, zero-padding fit. Verified with a full pairwise sweep across every region in the memory map, not just the two that moved: **zero overlaps anywhere** - the first time that's been true since this whole sequence of address changes began. USROMSTRT/USROMEND containment re-checked and still passes for all three ROM regions. Also cleaned up the accumulated resolved-issue narrative in the top-of-file note (previously ~40 lines chronicling the BASEDICT/DSTACK/RSTACK/INOUT/CODETOP/ APPDICT saga turn by turn, including one claim - the APPDICT/APPVARS overlap being open - that was already stale before this edit) down to a short, current-state summary, matching the same cleanup already applied to the documentation.

5.2 Structural duplication (identified, some resolved, some not)

- ✓ **:/CREATE/VARIABLE's header-building** — resolved via HEADER (section 7).
- ✓ **COMMA/CODECOMMA/CCOMMA/VCOMMA/VCCOMMA/CCOMMA1** — resolved via APPENDCELL/APPENDBYTE (section 6).
- ✓ **<#/#> recomputing PAD's address inline** — resolved, both now call PADW (section 20 / section 6).

- No further known duplication, but the codebase was never given a full pass specifically hunting for more.

5.3 Real, load-bearing gaps

- **Software (XON/XOFF) handshaking is still not implemented.** Now that hardware RTS/CTS handshaking is in place, this is the one remaining flow-control gap: this system still never transmits XON/XOFF and would not recognize either byte as a control signal if the remote device sent them — an incoming \$11/\$13 is just queued as an ordinary character. Only relevant for links with no RTS/CTS wiring (plain 3-wire serial); would need a byte- interception layer between the input ring and KEY's caller.
- **No hard boundary checks** anywhere DPHERE/CODEHERE/VARHERE grow toward each other or toward the stacks. UNUSED and VUNUSED both correctly **report** the distance to their boundary now (CODETOP and the newly-added APPVARSEND respectively), but nothing enforces either — a runaway compile can silently corrupt an adjacent region.
- ✓ **VUNUSEDW (the VUNUSED word) computed a meaningless number, not "how much APPVARS space remains" - a real, distinct bug, not just the absence of a check. Fixed.** LDD #APPCODE / SUBD VARHERE computed APPCODE - VARHERE (roughly \$7000 minus wherever VARHERE currently sits within APPVARS), which had nothing to do with APPVARS's own boundary - looked like a copy-paste slip from UNUSEDW immediately above it (LDD #CODETOP / SUBD CODEHERE, correct for CODEHERE's own boundary). Surfaced directly by a question about how APPVARS's size is actually set: it wasn't - APPVARS EQU \$021B defined only its start; there was no end/size constant anywhere. Fixed by adding APPVARSEND EQU APPVARS+8000 (an exclusive upper bound, \$215B, matching CODETOP's own convention for APPCODE) and changing VUNUSEDW to LDD #APPVARSEND / SUBD VARHERE. Verified: at cold start (VARHERE=APPVARS), this computes exactly 8000, matching APPVARS's documented size precisely; zero duplicate symbols; dictionary chain unaffected. APPVARSEND initially fell within APPDICT's current range - expected at the time, the same, already-tracked APPDICT/ APPVARS overlap, not something this fix caused. Since resolved by redefining APPVARSEND as APPDICT - 1 - see above.
- **ENVTABLE is incomplete.** Missing entries: /HOLD, /PAD (this build never fixed a capacity for either — filling them in would mean deciding a real bound first, not just picking a number), MAX-D, MAX-UD (need the dispatcher extended to push **two** cells for double-cell answers — current ENVQUERY only handles one), WORDLISTS/FLOORED (need the dispatcher to be able to report a recognized-but-false answer, distinct from "unrecognized name" — no such path exists yet).
- **CATCH-wrapped QUIT/INTERPRET rollback is scoped to one input line only.** A colon definition spanning multiple lines that fails partway through a later line only rolls back that line's contribution, not the whole definition back to :. A complete fix needs the region-pointer snapshot taken once at : and held until ; or an error, not refreshed every QLOOP pass.
- **ABORTHDR's LINK field is a placeholder 0** in the consolidated/split source — must be set to the real prior LATEST value once final ROM layout/assembly order is fixed.

5.4 Never resolved after being explicitly raised

- ❑ **J doesn't generalize past one level of loop nesting.** No J2/deeper equivalent exists. A triple-nested loop's innermost body has no built-in way to reach the outermost index without manually replicating J's offset arithmetic one frame further.
- ❑ **LEAVE's correctness depends on always being textually inside the loop it affects** — never verified against a LEAVE called from a separately-defined word invoked from within a loop (which would read/set the wrong stack frame, or crash).
- ❑ **WORD's 31-character cap is now inconsistent within the system.** PARSE/PARSE-NAME were deliberately rewritten to not share it, but WORD itself — and everything still built on it (CHAR, [CHAR], header names, FIND) — still has it.
- ❑ **The optional Search-Order word set is not implemented.** Every word resolves through one unified dictionary chain rooted at LATEST; there is no multi-wordlist support (WORDLIST, GET-ORDER/SET-ORDER, ALSO/ONLY, etc.). This was a foundational decision fixed since COLDSTRT's first version, not an oversight — now documented explicitly (ClaudeForth documentation, Section 4.5) along with what adding it later would actually require: restructuring FIND into a loop over an active search order, changing HEADER to link into a selectable "current" wordlist instead of unconditionally into LATEST, and fixing the few words (RECURSE, ;'s unsmudge step, WORDS) that read LATEST directly today.

5.5 Open design questions (not defects — deliberate unresolved choices)

- ❑ Whether ALLOT/VALLLOT/PICK/ROLL/BASE should range-check their inputs against actual stack/region bounds. Currently none of them do, consistently, by choice rather than oversight.
- ❑ Whether any additional distinction like TIB/SOURCE (fixed terminal buffer vs. current input source) is needed anywhere else in the system where EVALUATE's redirection might still cause a mismatch.
- ❑ SWIH's throw code (-99 in the consolidated source) was never assigned a real, deliberate value — it's a placeholder for "some hardware trap occurred," not a considered ANS-style code.
- ❑ REPLACES/SUBSTITUTE remain single-slot (one registered name/value pair, overwritten by the next REPLACES call) rather than a true multi-entry table. A real table needs its own storage layout and lookup structure — a deliberate scope decision made when SUBSTITUTE was completed, not an oversight.

5.6 What's solid (for reference, not action)

Stack manipulation (Core + Core Ext + return-stack transfer), all arithmetic (single + double + mixed precision), logic, comparison (single + double), full control flow including CASE, all defining words including DEFER/MARKER/VALUE/TO (VALUE now correctly targeting mutable space), memory and string operations (including a completed SUBSTITUTE), SOURCE/EVALUATE/REFILL mechanics, and the Tools word set (. S/WORDS/DUMP, with DUMP's edge-case bug fixed) are complete and were traced/verified against concrete cases during the build, not merely asserted correct.

6. Build Instructions

Which Files to Assemble

The buildable target is the single consolidated source file, `forth6809.asm`, delivered alongside this documentation and reproduced in full in Section 8. It is self-contained: all 27 numbered sections, the memory-map constants, and the applied GLOBALS page-zero layout live in this one file, assembled top to bottom with no external dependencies.

The per-section split files (`00_memory_map_and_globals.asm` through `27_forth_dictionary.asm`, plus `OPEN_ITEMS_CHECKLIST.md`) are reference copies only, generated by splitting `forth6809.asm` along its own section markers for easier reading and review. They are not wired together with `INCLUDE` directives and are not themselves a multi-file build target — assembling them individually, or as a set, is not supported. Any change made to one must be reflected back into `forth6809.asm` to actually take effect.

Assembler

This source follows standard Motorola-style 6809 assembler syntax throughout (`EQU`, `ORG`, `RMB`, `FCB`, `FCC`, `FDB`) and targets [LWASM, part of the LWTools toolchain](https://www.lwtools.ca/) — an actively maintained, open-source cross-assembler for the Motorola 6809 and Hitachi 6309, available at <https://www.lwtools.ca/>. A representative build command, producing a raw binary image suitable for programming directly into an EPROM:

```
lwasm --6809 --format=raw --output=forth6809.bin forth6809.asm
```

The `--6809` flag restricts LWASM to genuine 6809 instructions. This is a real constraint, not a formality: an audit of every mnemonic in this file against the genuine 6809 instruction set found one 6309-only instruction in actual use — `TSTD`, used to test whether `D` was zero after a `PULU` (which, unlike a load, does not itself affect condition codes on real 6809 hardware). It has been replaced with `CMPD #0`, a genuine 6809 instruction with identical behavior for this purpose. `--format=raw` honors this source's own `ORG` directives directly, producing a flat binary image rather than a relocatable object file, matching the fixed, non-relocated memory map described in Section 2.

Testing

Initial testing is performed using MAME, configured with the existing `mecb_6809` profile already present in MAME. The command line used is:

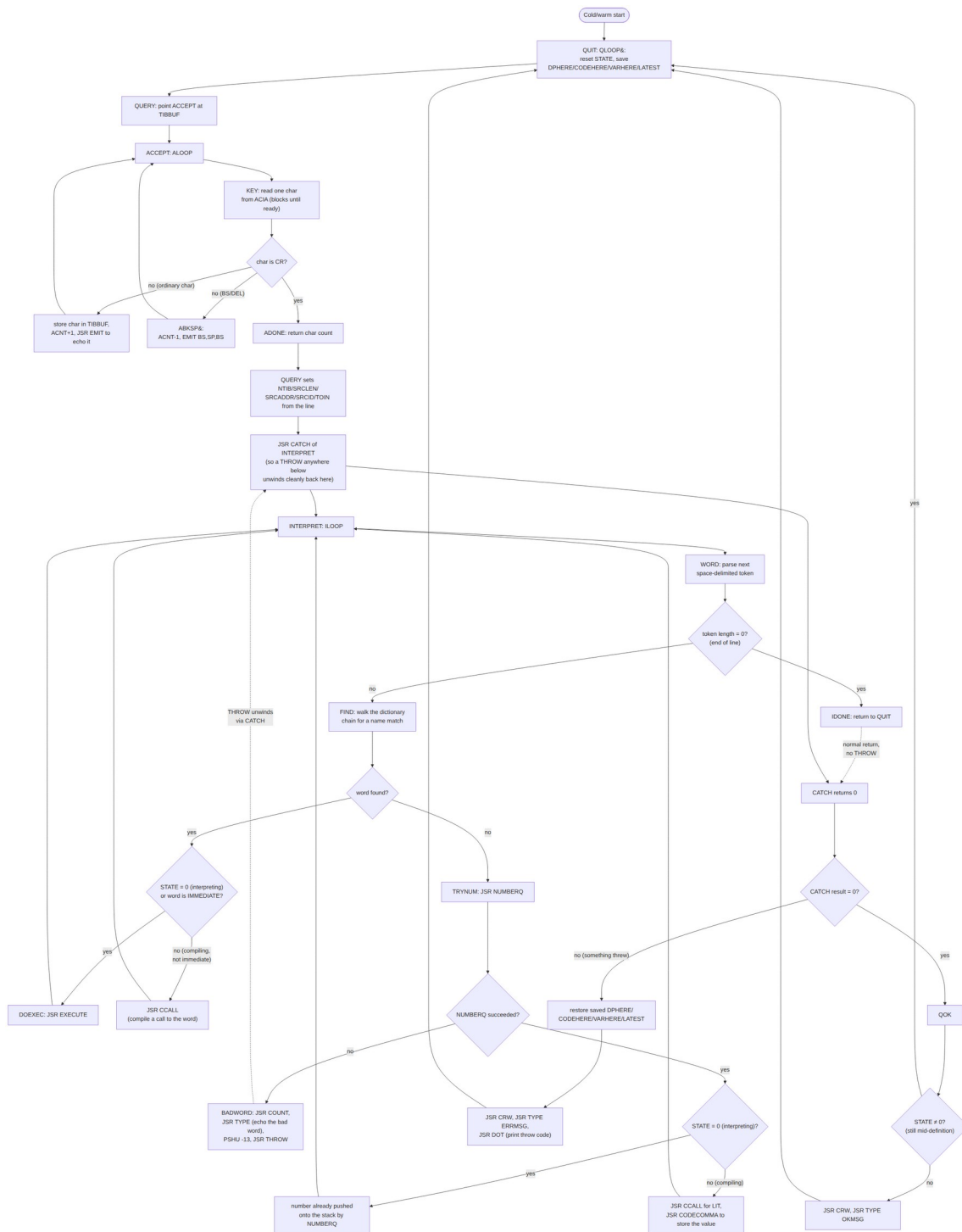
```
mame mecb_6809 -io_bank c0 -rom_size 16k -cart1 "/path/to/your/rom_image.bin" -window
```

The `rom_image.bin` file is already produced by the assembler.

The emulated serial output will target the terminals, though a serial comms application can also act in the interactive role.

7. Architecture

7.1 (Outer) Interpreter



ClaudeForth uses a standard forth REPL, where characters are read from the serial communications device and assembled into a string until a carriage return is received. The dictionary is then searched for

a matching string, in the expectation that it is a predefined word with attached code. If it's not found, an attempt is made to interpret the string as a number. Otherwise an exception is thrown.

Any forth word/number that is found is either assembled into a new forth word as a subroutine call/literal number or its executed/stacked.

The diagram above shows a UML activity diagram of the cooperating routines.

7.2 Dictionary Operations

Structure

The dictionary is a linked list that grows from low to high memory, with each entry storing a link to the header of the prior entry. The first created entry has a link field of zero. The last created entry's header is pointed to by an external link stored in a variable called LATEST.

Initialization

Two variables establish where the interpreter begins: LATEST, which marks the start of a search, and DPHERE, which marks where the next new dictionary entry will be built. Both are set once, by COLD at cold start: LATEST is set to BASELATEST, the address of the newest entry already built into the ROM dictionary, so a search immediately has a valid chain to walk; DPHERE is set to APPDICT, the start of the application dictionary's RAM area, so the first word a user defines is built starting at a known, fixed address.

Word Search

FIND begins at whatever address LATEST currently holds and walks backward through the chain one entry at a time, following each entry's own link field to reach the previous entry. At each entry, it first checks the length/flags byte: an entry whose smudge bit is set is skipped outright, and an entry whose name length doesn't match the search string is also skipped before a single character is ever compared, since a length mismatch alone is enough to rule it out. Only when the length matches are the name's characters actually compared. The search ends one of two ways: a full name match, in which case the entry's code-field address is returned; or the link field read is zero, meaning the chain's original first entry has been reached with no match, which signals the word was not found.

Word Append

HEADER builds a new entry by first capturing the current value of DPHERE as the address the new entry will occupy. It writes the length/flags byte and the name's characters starting there, advancing a working pointer as it goes. It then writes the current value of LATEST — the address of whatever is still the newest entry at this point — into the new entry's own link field, so the new entry points back to the one before it. A code-field address follows the link field. Only once the whole entry has been written are the two pointers updated: DPHERE is advanced past the new entry, so the next word will be built after it, and LATEST is set to the new entry's own address, so it becomes the newest entry — the one a search will now start from, and the one the next appended entry's link field will point back to.

8. Assembler Source

The complete 6809 assembly source for this system, as consolidated and verified across the design conversation, including every correction made along the way (the corrected VALUE PFA, the applied GLOBALS page-zero layout, the DUMP and SUBSTITUTE fixes, and every other fix described in Section 5's

checklist). This matches the forth6809.asm file delivered alongside this documentation; the section breakdown below follows the same numbered sections used in that file and in its per-section split.

8.1 Memory Map, Constants & GLOBALS Layout

```
; =====
; 6809 ANS FORTH - consolidated source
; Assembled from the full design conversation.
;
; GLOBALS LAYOUT: applied. Every scratch/global cell now has a
; fixed RMB-assigned address in page zero ($0000-$00FF, DP=$00
; at reset) - see the GLOBALS section below for the full layout
; and its byte budget (256 of 256 bytes used, 0 free - the page
; is now fully packed; any future scratch cell will need to
; reuse an existing one or move a cell out of page zero).
;
; SERIAL HANDSHAKING: RTS (hardware, output) is implemented -
; IRQH/KEY toggle it based on the input ring's fill level
; against INHIWATER/INLOWATER (see SECTION 3). CTS (input) needs
; no firmware logic at all: the 6850 hardware automatically
; inhibits TDRE while CTS is deasserted, and this system's
; existing TDRE-gated transmit logic already respects that with
; no code changes. Software (XON/XOFF) handshaking remains not
; implemented - this system still neither transmits nor
; recognizes those bytes.
;
; DICTIONARY: applied (SECTION 27), 219 entries (215 original +
; DOES> + TRUE + FALSE, added in later passes - see below).
; Every primitive with a real code label now has a real ROM
; header, chained via LINK, CFA pointing directly at its code.
; Building this surfaced two real findings, not just mechanical
; work: (1) the original 1024-byte BASEDICT could not hold the
; header table (1954 bytes needed) - BASEDICT was resized to
; 2048 bytes ($E000-$E7FF), taking the space from BASECODE (then
; $E800-$FFBF, 6080 bytes, was 7104). This resize was based on
; the header-table budget alone; the actual assembled byte size
; of BASECODE's code was never measured with a real 6809
; assembler, so whether 6080 bytes was enough for all ~530
; routine labels in this file was never verified before BASECODE
; moved again (below). (2) DOES>
; initially had no corresponding code anywhere in the file -
; SETDOES and the DOES> compiling word (code label DOESGT, since
; a literal ">" is not a valid 6809 assembler label) were added
; in a follow-up pass, alongside DODOES/DOESRT0, closing that
; gap; DOESBEH was added to the GLOBALS layout to support it,
; using 2 of the page's last 3 free bytes (1 now remains). TRUE
; and FALSE were added in a still later pass: CONSTANT TRUE -1
; and CONSTANT FALSE 0, the first CONSTANT-pattern ROM-resident
; words in this system (TRUEBODY/FALSEBODY, section 26) - every
; other ROM word's CFA is a plain code label, but a CONSTANT's
; CFA is the DODOES-trampoline pattern, built by hand here with
; fixed, assemble-time addresses since there is no interactive
; CREATE/CONSTANT phase for ROM content.
;
; BASEDICT now holds 1992 bytes ($D83F-$E006) - grown from the
; earlier exact-fit 1973 bytes when H_ABORT/H_QUIT (formerly
; ABORTHDR/QUITHDR) and BASELATEST moved in from their own
; section so every dictionary header lives in one contiguous
; block. BASECODE's start ($DFF4) did not move to match, since
; it's a fixed address, not derived from BASEDICT's real size -
; this reintroduces a real overlap, 19 bytes ($DFF4-$E006),
; between the two. Not resolved here; see the open-items
; checklist. USROMSTRT/USROMEND, INOUT, RSTACK, DSTACK, CODETOP,
; APPCODE, APPDICT, and APPVARS remain mutually consistent
; otherwise, with no other overlaps anywhere in the current
; memory map. See the ROM Size Required section of the
; documentation and the open-items checklist for real content
; totals and remaining margin.
;
```

```

; This file preserves the code exactly as derived and verified
; turn-by-turn in the conversation, including the corrected
; versions of every bug that was caught and fixed in place.
; SUBSTITUTE is complete but deliberately scoped to a single
; registered name/value pair, not a full table (see REPLACES).
; ENVTABLE's /HOLD and /PAD entries, and the DPHERE/CODEHERE/
; VARHERE boundary checks, remain explicitly incomplete/absent -
; see the inline notes preserved from that discussion, and the
; open-items checklist.
; =====
;
; -----
; MEMORY MAP
; -----
ROMSTRT EQU $C000      ; true physical start of the 16K EPROM's own
                       ; address decode (distinct from USROMSTRT
                       ; below, which excludes INOUT's 256-byte
                       ; shadow) - see the ROM Size Required section
                       ; of the documentation, and the ROM: padding
                       ; block right before SECTION 27
USROMSTRT EQU $C100    ; Usable ROM start. Beginning of the usable
                       ; EPROM address range. INITCODE, BASECODE,
                       ; and BASEDICT must all fall within
                       ; USROMSTRT..USROMEND - see the verification
                       ; note below each one's EQU.
USROMEND EQU VECTORS-1 ; Usable ROM end. Corrected: 1 before VECTORS'
                       ; start ($FFEF), not one past VECTORS' end as
                       ; originally defined - this is a real 16-bit
                       ; address (the last byte available before the
                       ; reserved vector table), usable directly in
                       ; comparisons or as a memory operand, unlike
                       ; the previous $10000 definition
VECTORS EQU $FFF0
INITCODE EQU $FFA2     ; was $FFA0, before that $FFC0 - now exactly 78
                       ; bytes, matching the ~78-byte COLDSTRT+WARM+
                       ; WARMMSG estimate with zero margin (was 80,
                       ; with 2 bytes of slack). UNRESOLVED: this is
                       ; a real, pre-existing overlap with BASECODE
                       ; (ends $FFBF), not something this change
                       ; introduced - $FFA0 already overlapped it by
                       ; 32 bytes before this change; $FFA2 overlaps
                       ; by 30 bytes - reduced, not resolved. See the
                       ; open-items checklist.
BASECODE EQU $E007     ; was $DFF4 - shifted up 19 bytes so its start
                       ; lands exactly one byte above BASEDICT's new
                       ; real end ($E006, after H_ABORT/H_QUIT/
                       ; BASELATEST grew it from 1973 to 1992 bytes),
                       ; resolving that specific overlap precisely:
                       ; zero gap, zero overlap. But this does NOT
                       ; eliminate the 19 bytes overall - BASECODE's
                       ; nominal size (8110) is unchanged, so its
                       ; nominal end also shifted up 19 bytes, to
                       ; $FFB4 (was $FFA1) - now 19 bytes INTO
                       ; INITCODE's start ($FFA2), the exact same
                       ; overlap amount, just relocated to a
                       ; different boundary. Not resolved; see the
                       ; open-items checklist.
BASEDICT EQU $D83F     ; was $D85D - shifted down the same 30 bytes as
                       ; BASECODE, preserving the exact, zero-padding
                       ; fit for its real 1973-byte dictionary content
                       ; (SECTION 27) and staying perfectly contiguous
                       ; with BASECODE's new start. Since grown to 1992
                       ; bytes (H_ABORT/H_QUIT/BASELATEST moved in) -
                       ; BASECODE has moved again to match; see its own
                       ; EQU and the open-items checklist.
INOUT EQU $C000        ; was $DF00 - moved so INOUT (256 B) sits
                       ; directly below USROMSTRT ($C100), contiguous,
                       ; no gap. This also resolves the INOUT portion
                       ; of the collision flagged when BASEDICT moved
                       ; to $D85D: INOUT no longer overlaps BASEDICT
                       ; ($D85D-$E011), since $C0FF < $D85D. The

```

```

; DSTACK and RSTACK portions of that same
; collision were NOT touched by this specific
; change, but were resolved separately when
; those two regions moved (see below). This
; move ALSO overlapped APPCODE at the time
; ($7000-$D7FF then) - since resolved too, when
; APPCODE moved down $2000 (see below).

INOUTEND EQU INOUT+$FF
RSTACK EQU $BFFF ; was $BEFF - occupied range is $BD00-$BFFF
; (768 bytes, unchanged size). RESOLVED: once a
; 512-byte gap sat between this and DSTACK
; below; DSTACK moving up $200 closed it - now
; exactly contiguous, no gap

DSTACK EQU $BCFF ; was $BAFF - moved up $200. RESOLVED (both):
; occupied range is now $B900-$BCFF, which
; exactly matches CODETOP ($B900) as its true
; bottom - the 512-byte mismatch is gone - and
; is now exactly contiguous with RSTACK's
; bottom ($BD00), closing that gap too

CODETOP EQU $B900 ; was $B800 - code space ceiling (data stack
; begins here). RESOLVED: this once no longer
; matched DSTACK's true occupied bottom (was
; $B700, a 512-byte mismatch) - now that DSTACK
; moved up $200 to $B900, CODETOP matches it
; exactly again

APPCODE EQU $7000 ; was $5000 - back to its original address.
; RESOLVED: this once overlapped DSTACK's true
; range by 512 bytes ($B700-$B8FF), a direct
; consequence of the CODETOP/DSTACK mismatch -
; now that DSTACK moved and CODETOP matches it
; again, APPCODE's nominal range (up to
; CODETOP-1) no longer reaches into DSTACK

APPDICT EQU $2000 ; was $015B - moved up, size unchanged (20133
; bytes, now $2000-$6EA4). REDUCED BUT NOT
; RESOLVED: still overlaps APPVARS below by 347
; bytes ($2000-$215A), though SIBUF/WORDBUF/
; TIBBUF/OUTBUF (swallowed by the previous
; APPDICT address) are now clear. See the
; open-items checklist

APPVARS EQU $021B ; grown from 256 to 8000 bytes (end now $215A,
; was $031A), taking the space directly from
; APPDICT above it; start address unchanged.
; That 8000-byte figure describes the original
; intent, not the current actual usable size -
; see APPVARSEND below, which now tracks
; APPDICT's real position instead

APPVARSEND EQU APPDICT-1 ; was APPVARS+8000 ($215B) - now derives
; directly from wherever APPDICT actually
; starts, currently $1FFF (7653 bytes usable,
; down from the static 8000). Self-correcting:
; this can no longer go stale if APPDICT moves
; again, unlike the previous fixed-size
; definition. VUNUSEDW (below) is unchanged -
; it already computed against APPVARSEND

SIBUF EQU $01FB ; was $0300
WORDBUF EQU $01DA ; was $02D4
TIBBUF EQU $018A ; was $0284
TIBBUFL EQU 80
SERBUF EQU $0106 ; was $0200 - USER0/USER1 removed entirely (see
; below); the 6 buffers (SERBUF's 4-byte index
; block, INBUF, OUTBUF, TIBBUF, WORDBUF, SIBUF)
; now sit contiguously right after MVSCRATCH,
; with no gap - this also closes a pre-existing
; 11-byte gap that used to sit between WORDBUF
; and SIBUF ($02F5-$02FF), unrelated to USER0/
; USER1 but caught while making this region
; genuinely contiguous end to end

INHEAD EQU SERBUF
INTAIL EQU SERBUF+1
OUTHEAD EQU SERBUF+2
OUTTAIL EQU SERBUF+3

```

```

INBUFSZ EQU 64
OUTBUFSZ EQU 64
INBUF EQU SERBUF+4
OUTBUF EQU SERBUF+4+INBUFSZ
GLOBALS EQU $0000

SP0 EQU DSTACK+1
RP0 EQU RSTACK+1

; -----
; SERIALPOLL - conditional-assembly switch for serial I/O.
; 1 (default): KEY/KEYQ/EMIT poll ACIASR directly, no interrupts,
; no ring buffers, no RTS/CTS handshaking - IRQH is a harmless RTI
; stub, matching the other unused vectors (NMIH/FIRQH/SWI2H/
; SWI3H). 0: the original interrupt-driven implementation, with
; INBUF/OUTBUF ring buffers serviced by IRQH and RTS-based flow
; control via INFILL/RTSCHECKHI/RTSCHECKLO. Uses LWASM's IFEQ/
; ELSE/ENDC (a numeric-expression test, not IFDEF/IFNDEF, since
; this is a value to compare, not a symbol's mere presence).
; -----
SERIALPOLL EQU 1

; -----
; ACIA (6850) constants - the chip sits at INOUT+8, not at the
; base of the I/O block, leaving INOUT+0..INOUT+7 free for other
; memory-mapped devices sharing this 256-byte region
; -----
ACIA EQU INOUT+8
ACIACR EQU ACIA
ACIASR EQU ACIA
ACIADR EQU ACIA+1
SR_RDRF EQU $01
SR_TDRE EQU $02
SR_IRQ EQU $80
CR_RESET EQU $03
CR_RXON EQU $95
CR_RXTX EQU $B5
CR_POLL EQU $15 ; bit7=0 (RX interrupt disabled), bits6-5=00 (RTS
; low, TX interrupt disabled) - CR_RXON ($95) with
; only the RX-interrupt-enable bit cleared. Used
; only when SERIALPOLL=1; RTS stays permanently
; low (asserted), since polling mode has no ring
; buffer to overflow and so needs no flow control
CR_RTSHI EQU $D5 ; bits6-5=10: RTS high, TX int disabled, RX int enabled -
; derived from CR_RXON ($95) with bits6-5 changed from
; 00 to 10; the ACIA has no combination offering RTS
; high AND TX interrupt enabled simultaneously (bits6-5
; only has 00/01/10/11, and only 01 enables TX interrupt,
; which always ties RTS low) - EMIT/IRQH's TXCHK must
; respect this and defer transmission while RTS is high

INHIWATER EQU 48 ; input ring fill level (of 64) at/above which RTS is
; asserted high, telling the remote device to pause
INLOWATER EQU 16 ; fill level at/below which RTS is reasserted low;
; deliberately well below INHIWATER (hysteresis) so
; RTS doesn't chatter right at a single threshold

; -----
; Flag / opcode constants
; -----
TRUEV EQU $FFFF
FALSEV EQU $0000
OPJSR EQU $BD
RTSOPC EQU $39

; -----
; Control-flow compile-time tags
; -----
TAGFWD EQU 1
TAGBACK EQU 2
TAGDO EQU 3

```

```

TAGCASE EQU 4
TAGOF EQU 5
TAGENDOF EQU 6

```

```

; =====
; GLOBALS - real layout, applied. Every scratch/state cell
; referenced across the whole build now has a fixed address
; in page zero (DP = $00 at reset, set in COLDSTRT), laid out
; in RMB order below. Total: 256 of 256 bytes used - the page
; is fully packed. (DOESBEH and RTSSTATE were added in later
; passes, after this comment was first written with 253/256;
; they used up the last 3 bytes of headroom entirely.)
;
; SNEND, which appeared in the original placeholder list
; (documented under S"/SLITERAL/ABORT"'s string runtime),
; was dropped here: a full pass over every reference confirmed
; it is never actually read or written anywhere - SCNT/SPTR
; alone carry that runtime. CSAVE was removed earlier (see the
; COMMA/CODECOMMA history) and was never part of this layout.
; =====
; ORG $0000 ; GLOBALS page - Direct Page (DP) set to $00 at reset
STATE RMB 2 ; offset $00
BASE RMB 2 ; offset $02
LATEST RMB 2 ; offset $04
DPHERE RMB 2 ; offset $06
CODEHERE RMB 2 ; offset $08
VARHERE RMB 2 ; offset $0A
HANDLER RMB 2 ; offset $0C
THROWN RMB 2 ; offset $0E
TOIN RMB 2 ; offset $10
NTIB RMB 2 ; offset $12
DELIM RMB 1 ; offset $14
WSTART RMB 2 ; offset $15
SLEN RMB 1 ; offset $17
SNAMEP RMB 2 ; offset $18
FNDPTR RMB 2 ; offset $1A
HDRPTR RMB 2 ; offset $1C
HDRFLAGS RMB 1 ; offset $1E
CADDR RMB 2 ; offset $1F
CNTREM RMB 1 ; offset $21
NUMNEG RMB 1 ; offset $22
NADDR RMB 2 ; offset $23
NCNT RMB 2 ; offset $25
MULBASE RMB 1 ; offset $27
CARRY RMB 1 ; offset $28
MSCR RMB 2 ; offset $29 (shared: -, *, comparisons, WITHIN...)
MSCR2 RMB 2 ; offset $2B
MSCR3 RMB 2 ; offset $2D
MSCR4 RMB 2 ; offset $2F
HLD RMB 2 ; offset $31
DEPTHTMP RMB 2 ; offset $33
AMAX RMB 2 ; offset $35
ABUFP RMB 2 ; offset $37
ACNT RMB 2 ; offset $39
ACH RMB 1 ; offset $3B
EMITCH RMB 1 ; offset $3C
NEWHDR RMB 2 ; offset $3D
NAMEP RMB 2 ; offset $3F
NAMELEN RMB 1 ; offset $41
PTARGET RMB 2 ; offset $42
PFIELD RMB 2 ; offset $44
NEWFLD RMB 2 ; offset $46
CSP RMB 2 ; offset $48
EXITCNT RMB 2 ; offset $4A
EXITPTR RMB 2 ; offset $4C
HDRSMUDGE RMB 1 ; offset $4E
SCNT RMB 2 ; offset $4F
SPTR RMB 2 ; offset $51
SAVEN RMB 2 ; offset $53
DRWIDTH RMB 2 ; offset $55
DRLEN RMB 2 ; offset $57

```

DRADDR	RMB	2	; offset \$59
DRPAD	RMB	2	; offset \$5B
PRODHI	RMB	2	; offset \$5D
PRODLO	RMB	2	; offset \$5F
PSIGN	RMB	1	; offset \$61
DIVNUM	RMB	2	; offset \$62
DIVDEN	RMB	2	; offset \$64
DIVREM	RMB	2	; offset \$66
DIVCNT	RMB	1	; offset \$68
DNSIGN	RMB	1	; offset \$69
DVSIGN	RMB	1	; offset \$6A
DVOWNSIGN	RMB	1	; offset \$6B
MAHI	RMB	1	; offset \$6C
MALO	RMB	1	; offset \$6D
MBHI	RMB	1	; offset \$6E
MBLO	RMB	1	; offset \$6F
MSIGN	RMB	1	; offset \$70
REM	RMB	2	; offset \$71
DCNT	RMB	1	; offset \$73
UDHI	RMB	2	; offset \$74
UDLO	RMB	2	; offset \$76
PRSIGN	RMB	1	; offset \$78
R2A	RMB	2	; offset \$79
R2B	RMB	2	; offset \$7B
RDST	RMB	2	; offset \$7D
RVAL	RMB	2	; offset \$7F
TR1	RMB	2	; offset \$81
TR2	RMB	2	; offset \$83
SHCNT	RMB	1	; offset \$85
SHCNT2	RMB	2	; offset \$86
TYPECNT	RMB	2	; offset \$88
TYPEADDR	RMB	2	; offset \$8A
PDELIM	RMB	1	; offset \$8C
PSTART	RMB	2	; offset \$8D
PLEN	RMB	2	; offset \$8F
CMPA1	RMB	2	; offset \$91
CMPL1	RMB	2	; offset \$93
CMPA2	RMB	2	; offset \$95
CMPL2	RMB	2	; offset \$97
CMPMIN	RMB	2	; offset \$99
SRCH1	RMB	2	; offset \$9B
SRCH1L	RMB	2	; offset \$9D
SRCH2	RMB	2	; offset \$9F
SRCH2L	RMB	2	; offset \$A1
SRCHPOS	RMB	2	; offset \$A3
SRCHI	RMB	2	; offset \$A5
UEADDR	RMB	2	; offset \$A7
UESRCLEN	RMB	2	; offset \$A9
UEDST	RMB	2	; offset \$AB
UEOUTLEN	RMB	2	; offset \$AD
SNXT	RMB	2	; offset \$AF
SNTARGET	RMB	2	; offset \$B1
REPLNAME	RMB	2	; offset \$B3
REPLNLEN	RMB	2	; offset \$B5
REPLVAL	RMB	2	; offset \$B7
REPLVLEN	RMB	2	; offset \$B9
SUBDESTCAP	RMB	2	; offset \$BB
SUBDESTADR	RMB	2	; offset \$BD
SUBSRCADR	RMB	2	; offset \$BF
SUBSRCLEN	RMB	2	; offset \$C1
SUBOUTLEN	RMB	2	; offset \$C3
SUBWPTR	RMB	2	; offset \$C5
SUBCOPYCNT	RMB	2	; offset \$C7
SUBCOPYSRC	RMB	2	; offset \$C9
MKDP	RMB	2	; offset \$CB
MKCODE	RMB	2	; offset \$CD
MKVAR	RMB	2	; offset \$CF
MKLATEST	RMB	2	; offset \$D1
EVSAVEA	RMB	2	; offset \$D3
EVSAVEI	RMB	2	; offset \$D5
EVSAVEI	RMB	2	; offset \$D7

```

EVSAVET    RMB    2    ; offset $D9
SRCADDR    RMB    2    ; offset $DB
SRCLLEN    RMB    2    ; offset $DD
SRCID      RMB    2    ; offset $DF
SPAN       RMB    2    ; offset $E1
DSPTMP     RMB    2    ; offset $E3
WWALK      RMB    2    ; offset $E5
DUMPADDR   RMB    2    ; offset $E7
DUMPCNT    RMB    2    ; offset $E9
DUMPCOL    RMB    1    ; offset $EB
HEXBUF     RMB    2    ; offset $EC
DUVALID    RMB    1    ; offset $EE
ENVLEN     RMB    2    ; offset $EF
ENVADDR    RMB    2    ; offset $F1
QSAVEDP    RMB    2    ; offset $F3
QSAVECODE  RMB    2    ; offset $F5
QSAVEVAR   RMB    2    ; offset $F7
QSAVELATEST RMB    2    ; offset $F9
QTHROWCODE RMB    2    ; offset $FB
DOESBEH    RMB    2    ; offset $FD - SETDOES scratch
RTSSTATE   RMB    1    ; offset $FF - 0 = RTS low (normal), nonzero = RTS
                        ; high (paused) - see CR_RTSHI. GLOBALS page is now
                        ; fully packed: 256 of 256 bytes used, 0 free.

```

```

GLOBALS_USED EQU 256 ; total bytes used, of 256 available - fully packed

```

```

; -----
; MVSCRATCH - three cells shared, one at a time, by routine
; families that never call each other or run concurrently in
; this single-threaded interpreter: MOVE/CMOVE/CMOVE>, FILL, and
; HOLDS (plus the single-cell multiply routine). Sharing avoids
; needing 3x the physical storage for what is provably the same
; scratch need at different times; the tradeoff is that these
; three cells use ordinary extended addressing (3-byte LDD/STD),
; not direct-page (2-byte), since page zero has no room left.
;
; MVCNT      - MOVE/CMOVE's remaining-byte count
; FILLCNT    EQU MVCNT  - FILL's remaining-byte count
; HSLLEN     EQU MVCNT  - HOLDS's remaining-char count
; MVDST      - MOVE/CMOVE's destination address
; FILLADDR   EQU MVDST  - FILL's target address
; HSADDR     EQU MVDST  - HOLDS's source address
; MVSRC      - MOVE/CMOVE's source address
; MRESULT    EQU MVSRC  - single-cell multiply's 16-bit result
; FILLCHR    EQU MVSRC  - FILL's fill character (1 byte, uses
;                        MVSRC's first byte only)
; -----
;
; ORG    $0100
MVCNT    RMB    2
MVDST    RMB    2
MVSRC    RMB    2
FILLCNT  EQU  MVCNT
HSLLEN   EQU  MVCNT
FILLADDR EQU  MVDST
HSADDR   EQU  MVDST
MRESULT  EQU  MVSRC
FILLCHR  EQU  MVSRC

```

```

; Provide padding, to ensure the correct ROM & .bin file size (and
; opcode offsets) for the MAME emulation and flash memory burn.

```

```

ORG USROMSTR
ROM:
    FILL $FF,BASEDICT-USROMSTR

```

8.2 Hardware Vector Table

```

; =====
; SECTION 1: HARDWARE VECTOR TABLE

```

```

; =====
; ORG VECTORS ; VECTORS is $FFF0
VRESV FDB $0000
VSWI3 FDB SWI3H
VSWI2 FDB SWI2H
VFIRQ FDB FIRQH
VIRQ FDB IRQH
VSWI FDB SWIH
VNMI FDB WARM ; NMI -> warm restart
VRESET FDB COLDSTRT

VECTOREND EQU * ; Verify vectors size, value should match $10.
VECTORSIZE EQU VECTOREND-VECTORS

; =====
; END OF CONSOLIDATED SOURCE
; =====

```

8.3 Init Code (COLDSTRT / WARM)

```

; =====
; SECTION 2: INIT CODE (COLDSTRT / WARM)
; =====
; ORG INITCODE ; INITCODE is $FFA2 (was INIT/$FFA0, before that literal $FFC0)
COLDSTRT:
    ORCC #$50
    LDS #RSTACK+1
    LDU #DSTACK+1
    CLRA
    TFR A,DP

    LDX #GLOBALS
    LDB #0
CLRGLOBAL: CLR ,X+
    DECB
    BNE CLRGLOBAL

    LDX #SERBUF
    CLR ,X+
    CLR ,X

    JSR INITSERIAL

    JMP COLD

WARM:
    ORCC #$50
    CLRA
    TFR A,DP
    LDU #SP0
    LDS #RP0
    LDX #WARMMSG
    PSHU X
    LDD #WARMMSGGL
    PSHU D
    JSR TYPE
    ANDCC #$AF
    JMP ABORT

WARMMSG: FCC " warm"
WARMMSGGL EQU *-WARMMSG

INITEND EQU * ; Verify no collision with vectors, value should match vector ORG
INITSIZE EQU INITEND-INITCODE

```

8.4 ACIA Interrupt Handler

```
;
; =====
; SECTION 3: ACIA INTERRUPT HANDLER
; =====
;
; ORG    BASECODE      ; BASECODE is $DFF4. This ORG was missing
;                      ; entirely - every routine from here through
;                      ; SECTION 26 (IRQH, COLD/ABORT/QUIT, and
;                      ; every primitive) would otherwise have
;                      ; continued growing from wherever SECTION 2's
;                      ; WARM message left the location counter,
;                      ; inside INIT's 48-byte $FFC0-$FFEF budget,
;                      ; overflowing directly into VECTORS ($FFF0)
;                      ; instead of landing in BASECODE at all
;
; -----
; INITSERIAL - initializes the ACIA: master reset, then selects
; interrupt-driven or polling operation depending on SERIALPOLL.
; Extracted from COLDSTRT, which now just JSRs here - moved into
; this section so the ACIA's own init code sits next to the rest
; of its interrupt/polling logic rather than inline in COLDSTRT.
; -----
INITSERIAL: LDA    #$03
            STA    ACIACR      ; was "STA ACIA" - only correct by
                                ; coincidence while ACIA and ACIACR were
                                ; the same address; now genuinely distinct
            IFEQ   SERIALPOLL ; >>>>>>>>>
            LDA    #CR_RXON    ; interrupt-driven mode: RX interrupt on
            ELSE   ; <<<<<<<>>>>>
            LDA    #CR_POLL    ; polling mode: no interrupts, RTS held low
            ENDC   ; <<<<<<<<<<<
            STA    ACIACR      ; was "STA ACIA" - same fix
            RTS

            IFEQ   SERIALPOLL ; >>>>>>>>>
; -----
; INFILL - ( -- A=fill level, 0-63 ) input ring's current fill
; level. INBUFSZ is a power of two, and both indices are always
; kept in 0..INBUFSZ-1, so a plain masked subtraction gives the
; true mod-64 distance even across the wrap point.
; -----
INFILL:    LDA    INHEAD
            SUBA   INTAIL
            ANDA   #INBUFSZ-1
            RTS

; -----
; RTSCHECKHI - called from IRQH's own RX path (interrupts
; already masked by hardware during ISR execution, so no
; explicit masking needed here). If the input ring has reached
; INHIWATER and RTS is not already asserted high, assert it -
; telling the remote device to pause sending. Per the 6850, RTS
; has no automatic tie to reception; this is ordinary firmware
; flow control, not a chip feature.
; -----
RTSCHECKHI: JSR    INFILL
            CMPA   #INHIWATER
            BLO    RTSCHIDONE
            TST    RTSSTATE
            BNE    RTSCHIDONE      ; already high - nothing to do
            LDA    #CR_RTSHI
            STA    ACIACR
            LDA    #1
            STA    RTSSTATE
RTSCHIDONE: RTS

; -----
; RTSCHECKLO - called from mainline code (KEY), NOT from the
; ISR, so it must mask IRQ around its critical section: IRQH's
; own TXOFF path also writes ACIACR, and an interrupt landing
; mid-decision here could otherwise race it. If the ring has
```

```

; drained to INLOWATER or below and RTS is currently high,
; reassert RTS low - restoring TX-interrupt-enable too if
; output happens to be queued, since the ACIA has no control
; byte combination offering RTS-high with TX-interrupt-enabled
; simultaneously (see CR_RTSHI's comment).
; -----
RTSCHECKLO: JSR  INFILL
             CMPA #INLOWATER
             BHI  RTSCLODONE
             TST  RTSSTATE
             BEQ  RTSCLODONE           ; already low - nothing to do
             ORCC #$10                 ; mask IRQ for the critical section
             CLR  RTSSTATE
             LDB  OUTTAIL
             CMPB OUTHEAD
             BEQ  RTSCLONOTX
             LDA  #CR_RXTX
             STA  ACIACR
             BRA  RTSCLOUNMASK
RTSCLONOTX:  LDA  #CR_RXON
             STA  ACIACR
RTSCLOUNMASK: ANDCC #$EF
RTSCLODONE: RTS

IRQH:       LDA  ACIASR
             BITA #SR_IRQ
             BEQ  IRQDONE

             BITA #SR_RDRF
             BEQ  TXCHK

             LDB  INHEAD
             LDA  ACIADR
             LDX  #INBUF
             STA  B,X
             INCB
             ANDB #INBUFSZ-1
             CMPB INTAIL
             BEQ  IRQDONE
             STB  INHEAD
             JSR  RTSCHECKHI
             BRA  IRQDONE

TXCHK:      LDB  OUTTAIL
             CMPB OUTHEAD
             BEQ  TXOFF
             LDX  #OUTBUF
             LDA  B,X
             STA  ACIADR
             INCB
             ANDB #OUTBUFSZ-1
             STB  OUTTAIL
             BRA  IRQDONE

TXOFF:      TST  RTSSTATE
             BNE  IRQDONE           ; RTS is asserted high - leave ACIACR alone,
                                     ; or this would incorrectly drop it back low

             LDA  #CR_RXON
             STA  ACIACR

IRQDONE:    RTI

           ELSE ; <<<<<>>>>
IRQH:       RTI                     ; polling mode (SERIALPOLL=1) - ACIA
                                     ; interrupts are never enabled (see
                                     ; COLDSTRT's CR_POLL init), so this should
                                     ; never fire; kept as a safe stub matching
                                     ; the other unused vectors below

           ENDC ; <<<<<<<<<<

SWI3H:     RTI

```

```

SWI2H:  RTI
FIRQH:  RTI
NMIH:   RTI                ; unused now that NMI -> WARM
SWIH:   LDD  #-99           ; placeholder hardware-trap code; push and
      PSHU  D               ; JMP THROW, per the CATCH/THROW turn
      JMP   THROW

```

8.5 COLD / ABORT / QUIT

```

; =====
; SECTION 4: COLD / ABORT / QUIT (with CATCH-wrapped INTERPRET)
; =====
COLD:   LDD  #APPVARS
      STD  VARHERE
      LDD  #APPCODE
      STD  CODEHERE
      LDD  #APPDICT
      STD  DPHERE
      LDD  #BASELATEST
      STD  LATEST

      LDD  #10
      STD  BASE

      LDD  #TIBBUF
      STD  SRCADDR
      LDD  #0
      STD  SRCLEN
      STD  SRCID

      LDX  #SIGNON
      PSHU X
      LDD  #SIGNONL
      PSHU D
      JSR  TYPE
      JSR  CRW
      ; falls through into ABORT

ABORT:  LDU  #SP0
      ; falls through into QUIT

QUIT:   LDS  #RP0

QLOOP:  LDD  #0
      STD  STATE

      JSR  QUERY

      LDD  DPHERE
      STD  QSAVEDP
      LDD  CODEHERE
      STD  QSAVECODE
      LDD  VARHERE
      STD  QSAVEVAR
      LDD  LATEST
      STD  QSAVELATEST

      LDD  #INTERPRET
      PSHU D
      JSR  CATCH
      PULU D
      STD  QTHROWCODE
      CMPD #0
      BEQ  QOK

      LDD  QSAVEDP
      STD  DPHERE
      LDD  QSAVECODE

```

```

STD CODEHERE
LDD QSAVEVAR
STD VARHERE
LDD QSAVELATEST
STD LATEST
LDU #SP0

JSR CRW
LDX #ERRMSG
PSHU X
LDD #ERRMSG
PSHU D
JSR TYPE
LDD QTHROWCODE
PSHU D
JSR DOT
BRA QLOOP

QOK: LDD STATE
      BNE QLOOP
      JSR CRW
      LDX #OKMSG
      PSHU X
      LDD #OKMSG
      PSHU D
      JSR TYPE
      BRA QLOOP

SIGNON: FCC "6809 FORTH v1.0"
SIGNONL EQU *-SIGNON
OKMSG: FCC " ok"
OKMSG: EQU *-OKMSG
ERRMSG: FCC " ERROR "
ERRMSG: EQU *-ERRMSG

```

8.6 Inner-Interpreter Support

```

; =====
; SECTION 5: INNER-INTERPRETER SUPPORT (LIT, ZBRANCH, BRANCH,
; DODOES, DODEFER, EXECUTE)
; =====
LIT: PULS X
      LDD ,X++
      PSHU D
      PSHS X
      RTS

ZBRANCH: PULU D
          PULS X
          CMPD #0
          BNE ZSKIP
          LDD ,X
          LEAX D,X
          PSHS X
          RTS
ZSKIP: LEAX 2,X
        PSHS X
        RTS

BRANCH: PULS X
         LDD ,X
         LEAX D,X
         PSHS X
         RTS

DODOES: PULS X
         LDY ,X++
         LDD ,X

```

```

        PSHU D
        JMP ,Y

DOESRT0: RTS

; -----
; SETDOES - compiled via JSR by DOES>'s immediate action.
; Patches LATEST's trampoline BEHAVIOR field, then returns
; two levels up - skipping the rest of the defining word's
; body entirely, straight back to whoever invoked it.
; -----
SETDOES: PULS X          ; X = addr right after "JSR SETDOES" - new BEHAVIOR
        STX  DOESBEH

        LDX  LATEST
        LDA  ,X
        STA  HDRFLAGS
        LEAX 1,X
        LDB  HDRFLAGS
        ANDB #$1F
        CLRA
        LEAX D,X          ; skip name -> LINK field
        LEAX 2,X          ; skip LINK -> CFA field
        LDD  ,X          ; D = CFA (trampoline address)
        ADDD #3          ; +3 -> BEHAVIOR field (past JSR DODOES)
        TFR  D,X

        LDD  DOESBEH
        STD  ,X          ; patch it

        PULS X          ; X = the OUTER defining word's own return addr
        JMP  ,X          ; jump there directly - "double RTS"

DODEFER: PULU X
        LDD  ,X
        TFR  D,X
        JMP  ,X

DOABORTUNDEF: LDD #-21
              PSHU D
              JMP  THROW

DOMARKER: PULU X
          LDD  ,X
          STD  DPHERE
          LDD  2,X
          STD  CODEHERE
          LDD  4,X
          STD  VARHERE
          LDD  6,X
          STD  LATEST
          RTS

EXECUTE: PULU X
          JSR  ,X
          RTS

```

8.7 Comma Family

```

; =====
; SECTION 6: COMMA FAMILY (factored via APPENDCELL/APPENDBYTE)
; =====
APPENDCELL: PULU D
            LDY  ,X
            STD  ,Y++
            STY  ,X
            RTS

```

```

APPENDBYTE: PULU D
             LDY   ,X
             STB   ,Y+
             STY   ,X
             RTS

COMMA:       LDX   #CODEHERE
             JMP   APPENDCELL

CODECOMMA:   LDX   #CODEHERE
             JMP   APPENDCELL

CCOMMA:      LDX   #CODEHERE
             JMP   APPENDBYTE

CCOMMA1:     LDX   #CODEHERE
             JMP   APPENDBYTE

VCOMMA:      LDX   #VARHERE
             JMP   APPENDCELL

VCCOMMA:     LDX   #VARHERE
             JMP   APPENDBYTE

ALLOT:       PULU D
             LDX   CODEHERE
             LEAX  D,X
             STX   CODEHERE
             RTS

VALLLOT:     PULU D
             LDX   VARHERE
             LEAX  D,X
             STX   VARHERE
             RTS

HEREW:       LDD   CODEHERE
             PSHU  D
             RTS

VHEREW:      LDD   VARHERE
             PSHU  D
             RTS

PADW:        LDD   CODEHERE
             ADDD  #84
             PSHU  D
             RTS

UNUSEDW:     LDD   #CODETOP
             SUBD  CODEHERE
             PSHU  D
             RTS

VUNUSEDW:    LDD   #APPVARSEND ; was #APPCODE - a real bug, not just a
             SUBD  VARHERE     ; missing check: computed a meaningless
             PSHU  D           ; distance to an unrelated region instead
             RTS               ; of remaining APPVARS space

```

8.8 HEADER

```

; =====
; SECTION 7: HEADER (factored from :/CREATE/VARIABLE)
; =====
HEADER:  LDD   #32
          PSHU  D
          JSR   WORD
          PULU  X

```

```

        LDA    ,X
        STA    NAMELEN
        LEAX   1,X
        STX    NAMEP

        PULU   D
        STB    HDRSMUDGE

        LDD    DPHERE
        STD    NEWHDR
        LDX    DPHERE
        LDA    NAMELEN
        TST    HDRSMUDGE
        BEQ    HDNOSM
        ORA    #$40
HDNOSM:  STA    ,X+
        LDY    NAMEP
        LDB    NAMELEN
        BEQ    HDNONM
HDCPY:   LDA    ,Y+
        STA    ,X+
        DECB
HDNONM:  BNE    HDCPY
        LDD    LATEST
        STD    ,X++
        LDD    CODEHERE
        STD    ,X++
        STX    DPHERE
        LDD    NEWHDR
        STD    LATEST
        RTS

```

8.9 Defining Words

```

; =====
; SECTION 8: DEFINING WORDS
; =====
COLON:   LDD    #TRUEV
        PSHU   D
        JSR    HEADER
        TFR    U,D
        STD    CSP
        LDD    #-1
        STD    STATE
        RTS

SEMI:    LDD    #RTSOPC
        PSHU   D
        JSR    CCOMMA1
        TFR    U,D
        CMPD   CSP
        BEQ    SEMI0K
        JSR    CFERR
SEMI0K:  LDX    LATEST
        LDA    ,X
        ANDA   #$BF
        STA    ,X
        LDD    #0
        STD    STATE
        RTS

CREATE:  LDD    #0
        PSHU   D
        JSR    HEADER
        LDD    #DODOES
        PSHU   D
        JSR    CCALL
        LDD    #DOESRT0

```

```

        PSHU D
        JSR CODECOMMA
        LDD CODEHERE
        PSHU D
        JSR CODECOMMA
        RTS

; -----
; DOES> ( -- ) IMMEDIATE, compile-only. Compiles a call to
; SETDOES. Code label DOESGT, not "DOES>" - a literal ">" is
; not valid in a 6809 assembler label, same reason ?DUP/2DUP/
; etc. all use mnemonic labels rather than their literal names.
; -----
DOESGT: LDD #SETDOES
        PSHU D
        JSR CCALL
        RTS

VARIABLE: LDD #0
          PSHU D
          JSR HEADER
          LDD #DODOES
          PSHU D
          JSR CCALL
          LDD #DOESRT0
          PSHU D
          JSR CODECOMMA
          LDD VARHERE
          PSHU D
          JSR CODECOMMA
          LDD #0
          LDX VARHERE
          STD ,X++
          STX VARHERE
          RTS

ATSIGN: PULU X
        LDD ,X
        PSHU D
        RTS

CONSTANT: LDD #0
          PSHU D
          JSR HEADER
          LDD #DODOES
          PSHU D
          JSR CCALL
          LDD #ATSIGN
          PSHU D
          JSR CODECOMMA
          LDD CODEHERE
          PSHU D
          JSR CODECOMMA
          JSR COMMA
          RTS

DOVALUE: PULU X
        LDD ,X
        PSHU D
        RTS

VALUEW: LDD #0
        PSHU D
        JSR HEADER ; not smudged - immediately findable
        LDD #DODOES
        PSHU D
        JSR CCALL
        LDD #DOVALUE
        PSHU D
        JSR CODECOMMA ; trampoline itself is still code
        LDD VARHERE ; PFA = VARHERE, mutable space - was

```

```

        PSHU D                ; CODEHERE; T0 writes through this PFA,
        JSR CODECOMMA        ; so it must live in mutable space
        JSR VCOMMA           ; store x into VARHERE via VCOMMA,
                             ; not COMMA (which targets CODEHERE)

        RTS

TOW:    LDD #32
        PSHU D
        JSR WORD
        JSR FIND
        PULU D
        CMPD #0
        BNE TOFOUND
        PULU D
        LDD #-13
        PSHU D
        JSR THROW
TOFOUND: JSR TOBODY
        LDD STATE
        BEQ TOIMMED
        JSR LITERALW
        LDD #STOREW
        PSHU D
        JSR CCALL
        RTS
TOIMMED: PULU X
        PULU D
        STD ,X
        RTS

TWOVARIABLE: LDD #0
        PSHU D
        JSR HEADER
        LDD #DODOES
        PSHU D
        JSR CCALL
        LDD #DOESRT0
        PSHU D
        JSR CODECOMMA
        LDD VARHERE
        PSHU D
        JSR CODECOMMA
        LDD #0
        LDX VARHERE
        STD ,X++
        STD ,X++
        STX VARHERE
        RTS

TWOCONSTANT: LDD #0
        PSHU D
        JSR HEADER
        LDD #DODOES
        PSHU D
        JSR CCALL
        LDD #DFETCH
        PSHU D
        JSR CODECOMMA
        LDD CODEHERE
        PSHU D
        JSR CODECOMMA
        PULU D                ; x2, off the top
        STD MSCR
        JSR COMMA             ; x1 -> lower address
        LDD MSCR
        PSHU D
        JSR COMMA             ; x2 -> higher address
        RTS

BUFFERCOLON: PULU D
        STD MSCR2

```

```

        LDD #0
        PSHU D
        JSR HEADER
        LDD #DODOES
        PSHU D
        JSR CCALL
        LDD #DOESRT0
        PSHU D
        JSR CODECOMMA
        LDD VARHERE
        PSHU D
        JSR CODECOMMA
        LDD MSCR2
        PSHU D
        JSR VALLOT
        RTS

DEFERW: LDD #0
        PSHU D
        JSR HEADER
        LDD #DODOES
        PSHU D
        JSR CCALL
        LDD #DODEFER
        PSHU D
        JSR CODECOMMA
        LDD CODEHERE
        PSHU D
        JSR CODECOMMA
        LDD #DOABORTUNDEF
        PSHU D
        JSR COMMA
        RTS

DEFERFETCH: JSR TOBODY
            PULU X
            LDD ,X
            PSHU D
            RTS

DEFERSTORE: JSR TOBODY
            PULU X
            PULU D
            STD ,X
            RTS

ISW:      LDD #32
        PSHU D
        JSR WORD
        JSR FIND
        PULU D
        CMPD #0
        BNE ISFOUND
        PULU D
        LDD #-13
        PSHU D
        JSR THROW
ISFOUND: PULU X
        LDD STATE
        BEQ ISIMMED
        PSHU X
        JSR LITERALW
        LDD #DEFERSTORE
        PSHU D
        JSR CCALL
        RTS
ISIMMED: PSHU X
        JSR DEFERSTORE
        RTS

ACTIONOF: LDD #32

```

```

        PSHU D
        JSR WORD
        JSR FIND
        PULU D
        CMPD #0
        BNE AOFFOUND
        PULU D
        LDD #-13
        PSHU D
        JSR THROW
AOFFOUND: PULU X
        LDD STATE
        BEQ AOIMMED
        PSHU X
        JSR LITERALW
        LDD #DEFERFETCH
        PSHU D
        JSR CCALL
        RTS
AOIMMED: PSHU X
        JSR DEFERFETCH
        RTS

MARKERW: LDD DPHERE
        STD MKDP
        LDD CODEHERE
        STD MKCODE
        LDD VARHERE
        STD MKVAR
        LDD LATEST
        STD MKLATEST
        LDD #0
        PSHU D
        JSR HEADER
        LDD #DODOES
        PSHU D
        JSR CCALL
        LDD #DOMARKER
        PSHU D
        JSR CODECOMMA
        LDD CODEHERE
        PSHU D
        JSR CODECOMMA
        LDD MKDP
        PSHU D
        JSR COMMA
        LDD MKCODE
        PSHU D
        JSR COMMA
        LDD MKVAR
        PSHU D
        JSR COMMA
        LDD MKLATEST
        PSHU D
        JSR COMMA
        RTS

```

8.10 Outer Interpreter

```

; =====
; SECTION 9: OUTER INTERPRETER (INTERPRET / WORD / FIND / NUMBER?)
; =====
INTERPRET:
ILOOP:   JSR WORD
        LDX ,U
        LDA ,X
        BEQ IDONE

```

```

        JSR    FIND
        PULU   D
        TSTB
        LBEQ   TRYNUM

        LDA    STATE+1
        BEQ    DOEXEC
        TSTB
        BPL    DOEXEC
        JSR    CCALL
        BRA    ILOOP

DOEXEC:  JSR    EXECUTE
        BRA    ILOOP

TRYNUM:  JSR    NUMBERQ
        PULU   D
        CMPD   #0          ; was TSTD (6309-only) - PULU doesn't set CC on
                           ; genuine 6809, so compare D against 0 directly
        BEQ    BADWORD

        LDD    STATE
        BEQ    ILOOP
        LDD    #LIT
        PSHU   D
        JSR    CCALL
        JSR    CODECOMMA
        BRA    ILOOP

BADWORD: JSR    COUNT
        JSR    TYPE
        LDD    #-13
        PSHU   D
        JSR    THROW

IDONE:   RTS

WORD:    PULU   D
        STB    DELIM
        LDD    TOIN
        LDX    SRCADDR
        LEAX   D,X
        LDD    SRCLEN
        SUBD   TOIN
        TFR    D,Y

SKIPLP:  CMPY   #0
        BEQ    EMPTY
        LDA    ,X
        CMPA   DELIM
        BNE    STARTW
        LEAX   1,X
        LEAY   -1,Y
        BRA    SKIPLP

STARTW:  STX     WSTART
        LDB     #0

SCANLP:  CMPY   #0
        BEQ    ENDW
        LDA    ,X
        CMPA   DELIM
        BEQ    CONSUME
        CMPB   #31
        BEQ    ENDW
        LEAX   1,X
        LEAY   -1,Y
        INCB
        BRA    SCANLP

CONSUME: LEAX   1,X

```

```

ENDW:    LEAY    -1,Y
         TFR     X,D
         SUBD    SRCADDR
         STD     TOIN

         LDX     #WORDBUF
         STB     ,X+
         LDY     WSTART
COPYLP:   TSTB
         BEQ     COPYDONE
         LDA     ,Y+
         STA     ,X+
         DECB
         BRA     COPYLP
COPYDONE: LDX     #WORDBUF
         PSHU    X
         RTS

EMPTY:    LDX     #WORDBUF
         CLR     ,X
         PSHU    X
         RTS

FIND:     PULU    X
         LDA     ,X
         STA     SLEN
         LEAX    1,X
         STX     SNAMEP

         LDD     LATEST
         STD     FNDPTR

FFLOOP:   LDD     FNDPTR
         BEQ     NOTFOUND
         STD     HDRPTR
         TFR     D,X
         LDA     ,X
         STA     HDRFLAGS
         BITA    #$40
         BNE     FNEXT
         ANDA    #$1F
         CMPA    SLEN
         BNE     FNEXT
         LEAX    1,X
         LDY     SNAMEP
         LDB     SLEN
         BEQ     FMATCH
CMPLP:    LDA     ,X+
         CMPA    ,Y+
         BNE     FNEXT
         DECB
         BNE     CMPLP

FMATCH:   LDX     HDRPTR
         LEAX    1,X
         LDB     HDRFLAGS
         ANDB    #$1F
         CLRA
         LEAX    D,X
         LEAX    2,X
         LDD     ,X
         PSHU    D
         LDA     HDRFLAGS
         BITA    #$80
         BEQ     FISNORM
         LDD     #1
         BRA     FPUSH
FISNORM:  LDD     #-1
FPUSH:    PSHU    D
         RTS

```

```

FNEXT:  LDX  HDRPTR
        LEAX 1,X
        LDB  HDRFLAGS
        ANDB #$1F
        CLRA
        LEAX D,X
        LDD  ,X
        STD  FNDPTR
        BRA  FFL00P

NOTFOUND: LDX  SNAMEP
        LEAX -1,X
        PSHU X
        LDD  #0
        PSHU D
        RTS

UDMULADD: STB  CARRY
        LDA  BASE+1
        STA  MULBASE
        LDA  UDLO+1
        LDB  MULBASE
        MUL
        ADDB CARRY
        BCC  UM0
        INCA

UM0:    STB  UDLO+1
        STA  CARRY
        LDA  UDLO
        LDB  MULBASE
        MUL
        ADDB CARRY
        BCC  UM1
        INCA

UM1:    STB  UDLO
        STA  CARRY
        LDA  UDHI+1
        LDB  MULBASE
        MUL
        ADDB CARRY
        BCC  UM2
        INCA

UM2:    STB  UDHI+1
        STA  CARRY
        LDA  UDHI
        LDB  MULBASE
        MUL
        ADDB CARRY
        BCC  UM3
        INCA

UM3:    STB  UDHI
        RTS

NUMLOOP: LDD  NCNT
        BEQ  NLDONE
        LDX  NADDR
        LDA  ,X
        CMPA #'0'
        BLO  NLDONE
        CMPA #'9'
        BHI  NLALPHA
        SUBA #'0'
        BRA  NLGOT

NLALPHA: ANDA #$DF
        CMPA #'A'
        BLO  NLDONE
        CMPA #'Z'
        BHI  NLDONE
        SUBA #'A'-10

NLGOT:  CMPA  BASE+1
        BHS  NLDONE

```

```

    TFR    A,B
    JSR    UDMULADD
    LDX    NADDR
    LEAX    1,X
    STX    NADDR
    LDD    NCNT
    SUBD    #1
    STD    NCNT
    BRA    NUMLOOP
NLDONE:  RTS

TONUMBER: PULU D
          STD  NCNT
          PULU D
          STD  NADDR
          PULU D
          STD  UDHI
          PULU D
          STD  UDLO
          JSR  NUMLOOP
          LDD  UDLO
          PSHU D
          LDD  UDHI
          PSHU D
          LDX  NADDR
          PSHU X
          LDD  NCNT
          PSHU D
          RTS

NUMBERQ: PULU X
          STX  CADDR
          LDA  ,X
          BEQ  NQBAD
          STA  CNTREM
          LEAX 1,X

          CLR  NUMNEG
          LDA  ,X
          CMPA #'-'
          BNE  NQNOSIGN
          COM  NUMNEG
          LEAX 1,X
          DEC  CNTREM
          BEQ  NQBAD

NQNOSIGN: STX  NADDR
          CLRA
          LDB  CNTREM
          STD  NCNT
          LDD  #0
          STD  UDHI
          STD  UDLO

          JSR  NUMLOOP

          LDD  NCNT
          BNE  NQBAD

          LDD  UDLO
          TST  NUMNEG
          BEQ  NQPOS
          COMA
          COMB
          ADDD #1
NQPOS:   PSHU D
          LDD  #-1
          PSHU D
          RTS

NQBAD:   LDX  CADDR

```

```

PSHU X
LDD #0
PSHU D
RTS

```

8.11 QUERY / ACCEPT / EXPECT / KEY / KEY? / EMIT

```

; =====
; SECTION 10: QUERY / ACCEPT / EXPECT / KEY / KEY? / EMIT
; =====
; IFEQ SERIALPOLL ; >>>>>>>>
KEY:   LDA   INHEAD
      CMPA  INTAIL
      BEQ   KEY
      LDX   #INBUF
      LDB   INTAIL
      LDA   B,X
      INCB
      ANDB  #INBUFSZ-1
      STB   INTAIL
      PSHS  A                ; stash the char on the return stack across
                              ; the call - JSR/RTS is self-balancing, so
                              ; this needs no dedicated scratch global

      JSR   RTSCHECKLO
      PULS  A
      TFR   A,B
      CLRA
      PSHU  D
      RTS

KEYQ:  LDA   INHEAD
      CMPA  INTAIL
      BNE   KQTRUE
      LDD   #FALSEV
      PSHU  D
      RTS

KQTRUE: LDD   #TRUEV
      PSHU  D
      RTS

EMIT:  PULU  D
      STB   EMITCH
EMITWT: LDB   OUTHEAD
      INCB
      ANDB  #OUTBUFSZ-1
      CMPB  OUTTAIL
      BEQ   EMITWT
      LDX   #OUTBUF
      LDB   OUTHEAD
      LDA   EMITCH
      STA   B,X
      INCB
      ANDB  #OUTBUFSZ-1
      STB   OUTHEAD
      TST   RTSSTATE
      BNE   EMITNORTS        ; RTS is asserted high - leave ACIACR alone;
                              ; output stays queued until RTS drops low,
                              ; at which point RTSCHECKLO re-enables TX
                              ; interrupt itself if OUTBUF still has data

      LDA   #CR_RXTX
      STA   ACIACR
EMITNORTS: RTS

      ELSE ; <<<<<>>>>
; -----
; Polling versions of KEY/KEYQ/EMIT (SERIALPOLL=1) - no ring
; buffers, no interrupts, no RTS/CTS handshaking. Each blocks
; (KEY, EMIT) or checks once (KEYQ) directly against ACIASR.

```

```
KEY:      LDA    ACIASR
          BITA   #SR_RDRF
          BEQ    KEY
          LDA    ACIADR
          TFR    A,B
          CLRA
          PSHU   D
          RTS

KEYQ:     LDA    ACIASR
          BITA   #SR_RDRF
          BEQ    KQFALSE
          LDD    #TRUEV
          PSHU   D
          RTS

KQFALSE:  LDD    #FALSEV
          PSHU   D
          RTS

EMIT:     PULU   D
          STB    EMITCH

EMITWT:   LDA    ACIASR
          BITA   #SR_TDRE
          BEQ    EMITWT
          LDA    EMITCH
          STA    ACIADR
          RTS
          ENDC   ; <<<<<<<<<

ACCEPT:   PULU   D
          STD    AMAX
          PULU   D
          STD    ABUF
          LDD    #0
          STD    ACNT

ALOOP:    JSR    KEY
          PULU   D
          STB    ACH

          CMPB   #13
          BEQ    ADONE
          CMPB   #10
          BEQ    ALOOP
          CMPB   #8
          BEQ    ABKSP
          CMPB   #127
          BEQ    ABKSP

          LDD    ACNT
          CMPD   AMAX
          BEQ    ALOOP

          LDX    ABUF
          LEAX   D,X
          LDA    ACH
          STA    ,X
          LDD    ACNT
          ADDD   #1
          STD    ACNT

          CLRA
          LDB    ACH
          PSHU   D
          JSR    EMIT
          BRA    ALOOP

ABKSP:    LDD    ACNT
          BEQ    ALOOP
          SUBD   #1
```

```

        STD  ACNT
        LDD  #8
        PSHU D
        JSR  EMIT
        LDD  #32
        PSHU D
        JSR  EMIT
        LDD  #8
        PSHU D
        JSR  EMIT
        BRA  ALOOP

ADONE:   LDD  ACNT
        PSHU D
        RTS

EXPECTW: JSR  ACCEPT
        PULU D
        STD  SPAN
        RTS

QUERY:   LDX  #TIBBUF
        PSHU X
        LDD  #TIBBUFL
        PSHU D
        JSR  ACCEPT
        PULU D
        STD  NTIB
        STD  SRCLEN
        LDD  #TIBBUF
        STD  SRCADDR
        LDD  #0
        STD  SRCID
        STD  TOIN
        RTS

```

8.12 CATCH / THROW

```

; =====
; SECTION 11: COLON / SEMICOLON support already in section 8
; (COLON/SEMI) - CATCH/THROW, CFERR
; =====
CFERR:   LDD  #-22
        PSHU D
        JSR  THROW
        RTS

CATCH:   PULU X
        LDD  HANDLER
        PSHS D
        PSHS U
        TFR  S,D
        STD  HANDLER

        JSR  ,X

        LEAS 2,S
        PULS D
        STD  HANDLER
        LDD  #0
        PSHU D
        RTS

THROW:   PULU D
        CMPD #0
        BEQ  THDONE

        STD  THROWN

```

```

        LDX    HANDLER
        BEQ    THUNCAU

        TFR    X,S
        PULS   D
        TFR    D,U
        PULS   D
        STD    HANDLER

        LDD    THROWN
        PSHU   D
        RTS
THDONE:  RTS

THUNCAU: LDD    THROWN
        PSHU   D
        JMP    ABORT

```

8.13 Control Flow

```

; =====
; SECTION 12: CONTROL FLOW (IF/THEN/ELSE, BEGIN family,
; DO/LOOP/+LOOP/I/J/LEAVE/UNLOOP/?DO, EXIT, CASE family)
; =====
PATCH:  PULU   D
        STD    PFIELD
        PULU   D
        STD    PTARGET
        LDD    PTARGET
        SUBD   PFIELD
        LDX    PFIELD
        STD    ,X
        RTS

IF:      LDD    #ZBRANCH
        PSHU   D
        JSR    CCALL
        LDD    #0
        PSHU   D
        JSR    CODECOMMA
        LDD    CODEHERE
        SUBD   #2
        PSHU   D
        LDD    #TAGFWD
        PSHU   D
        RTS

THEN:    PULU   D
        CMPD   #TAGFWD
        BEQ    THOK
        JSR    CFERR

THOK:    PULU   X
        LDD    CODEHERE
        PSHU   D
        PSHU   X
        JSR    PATCH
        RTS

ELSE:    PULU   D
        CMPD   #TAGFWD
        BEQ    ELOK
        JSR    CFERR

ELOK:    PULU   X
        LDD    #BRANCH
        PSHU   D
        JSR    CCALL
        LDD    #0
        PSHU   D

```

```

        JSR CODECOMMA
        LDD CODEHERE
        SUBD #2
        STD NEWFLD
        LDD CODEHERE
        PSHU D
        PSHU X
        JSR PATCH
        LDD NEWFLD
        PSHU D
        LDD #TAGFWD
        PSHU D
        RTS

BEGIN:  LDD CODEHERE
        PSHU D
        LDD #TAGBACK
        PSHU D
        RTS

UNTIL:  PULU D
        CMPD #TAGBACK
        BEQ UNOK
        JSR CFERR
UNOK:   PULU X
        LDD #ZBRANCH
        PSHU D
        JSR CCALL
        LDD #0
        PSHU D
        JSR CODECOMMA
        LDD CODEHERE
        SUBD #2
        STD PFIELD
        TFR X,D
        PSHU D
        LDD PFIELD
        PSHU D
        JSR PATCH
        RTS

AGAIN:  PULU D
        CMPD #TAGBACK
        BEQ AGOK
        JSR CFERR
AGOK:   PULU X
        LDD #BRANCH
        PSHU D
        JSR CCALL
        LDD #0
        PSHU D
        JSR CODECOMMA
        LDD CODEHERE
        SUBD #2
        STD PFIELD
        TFR X,D
        PSHU D
        LDD PFIELD
        PSHU D
        JSR PATCH
        RTS

WHILE:  LDD #ZBRANCH
        PSHU D
        JSR CCALL
        LDD #0
        PSHU D
        JSR CODECOMMA
        LDD CODEHERE
        SUBD #2
        PSHU D

```

```

        LDD #TAGFWD
        PSHU D
        RTS

REPEAT: PULU D
        CMPD #TAGFWD
        BEQ RPOK1
        JSR CFERR
RPOK1:  PULU X
        STX NEWFLD
        PULU D
        CMPD #TAGBACK
        BEQ RPOK2
        JSR CFERR
RPOK2:  PULU X
        LDD #BRANCH
        PSHU D
        JSR CCALL
        LDD #0
        PSHU D
        JSR CODECOMMA
        LDD CODEHERE
        SUBD #2
        STD PFIELD
        TFR X,D
        PSHU D
        LDD PFIELD
        PSHU D
        JSR PATCH
        LDD CODEHERE
        PSHU D
        LDD NEWFLD
        PSHU D
        JSR PATCH
        RTS

RECURSE: LDX LATEST
        LDA ,X
        STA HDRFLAGS
        LEAX 1,X
        LDB HDRFLAGS
        ANDB #$1F
        CLRA
        LEAX D,X
        LEAX 2,X
        LDD ,X
        PSHU D
        JSR CCALL
        RTS

DO:      LDD #DOSETUP
        PSHU D
        JSR CCALL
        LDD CODEHERE
        PSHU D
        LDD #TAGDO
        PSHU D
        RTS

DOSETUP: PULU D
        STD MSCR
        PULU D
        STD MSCR2
        PULS X
        LDD #0
        PSHS D
        LDD MSCR2
        PSHS D
        LDD MSCR
        PSHS D
        PSHS X

```

```

      RTS

IWORD:  PULS  X
        LDD   2,S
        PSHS  X
        PSHU  D
        RTS

JWORD:  PULS  X
        LDD   10,S
        PSHS  X
        PSHU  D
        RTS

LEAVE:  PULS  X
        LDD   #TRUEV
        STD   6,S
        PSHS  X
        RTS

LOOP:   PULU  D
        CMPD  #TAGDO
        BEQ   LOOPOK
        JSR   CFERR
LOOPOK: PULU  X
        LDD   #DOTEST
        PSHU  D
        JSR   CCALL
        LDD   #0
        PSHU  D
        JSR   CODECOMMA
        LDD   CODEHERE
        SUBD  #2
        STD   PFIELD
        TFR   X,D
        PSHU  D
        LDD   PFIELD
        PSHU  D
        JSR   PATCH

        LDD   ,U
        CMPD  #TAGFWD
        BNE   LOOPDONE
        PULU  D
        PULU  X
        LDD   CODEHERE
        PSHU  D
        PSHU  X
        JSR   PATCH
LOOPDONE: RTS

DOTEST: PULS  X
        LDD   6,S
        BNE   DTEXTIT
        LDD   2,S
        ADDD  #1
        STD   2,S
        CMPD  4,S
        BEQ   DTEXTIT
        LDD   ,X
        LEAX  D,X
        PSHS  X
        RTS
DTEXTIT: LEAX  2,X
        LEAS  6,S
        PSHS  X
        RTS

PLUSLOOP: PULU  D
          CMPD  #TAGDO
          BEQ   PLUSLOOP

```

```

PLOOPOK: JSR CFERR
          PULU X
          LDD #DOPLUSTEST
          PSHU D
          JSR CCALL
          LDD #0
          PSHU D
          JSR CODECOMMA
          LDD CODEHERE
          SUBD #2
          STD PFIELD
          TFR X,D
          PSHU D
          LDD PFIELD
          PSHU D
          JSR PATCH

          LDD ,U
          CMPD #TAGFWD
          BNE PLOOPDONE
          PULU D
          PULU X
          LDD CODEHERE
          PSHU D
          PSHU X
          JSR PATCH
PLOOPDONE: RTS

DOPLUSTEST: PULS X
            LDD 6,S
            BNE DPTEXTIT
            PULU D
            STD MSCR
            LDD 2,S
            SUBD 4,S
            STD MSCR2
            LDD 2,S
            ADDD MSCR
            STD 2,S
            SUBD 4,S
            STD MSCR3
            LDA MSCR2
            LDB MSCR3
            PSHS B
            EORA ,S+ ; was "EORA B" - not valid 6809 syntax (no
                    ; register-to-register EORA); push B, then
                    ; operate through ,S+ - the standard 6809
                    ; idiom for adding/combining two registers

            BMI DPTEXTIT
            LDD MSCR3
            BEQ DPTEXTIT
            LDD ,X
            LEAX D,X
            PSHS X
            RTS
DPTEXTIT:  LEAX 2,X
            LEAS 6,S
            PSHS X
            RTS

QDO:      LDD #QDOSETUP
          PSHU D
          JSR CCALL
          LDD #0
          PSHU D
          JSR CODECOMMA
          LDD CODEHERE
          SUBD #2
          PSHU D
          LDD #TAGFWD
          PSHU D

```

```

        LDD    CODEHERE
        PSHU   D
        LDD    #TAGDO
        PSHU   D
        RTS

QDOSETUP: PULU  D
          STD   MSCR
          PULU  D
          STD   MSCR2
          PULS  X
          LDD   MSCR2
          CMPD  MSCR
          BNE   QDBUILD
          LDD   ,X
          LEAX  D,X
          PSHS  X
          RTS

QDBUILD: LEAX  2,X
          LDD   #0
          PSHS  D
          LDD   MSCR2
          PSHS  D
          LDD   MSCR
          PSHS  D
          PSHS  X
          RTS

UNLOOP:  PULS  X
          LEAS  6,S
          PSHS  X
          RTS

EXIT:    LDD   #0
          STD   EXITCNT
          TFR   U,D
          STD   EXITPTR
EXSCAN:  LDD   EXITPTR
          CMPD  CSP
          BEQ   EXSCANDONE
          LDX   EXITPTR
          LDD   ,X
          CMPD  #TAGDO
          BNE   EXNOTDO
          LDD   EXITCNT
          ADDD  #1
          STD   EXITCNT
EXNOTDO: LDD   EXITPTR
          ADDD  #4
          STD   EXITPTR
          BRA   EXSCAN
EXSCANDONE:
          LDD   #EXITUNLOOP
          PSHU  D
          JSR   CCALL
          LDD   EXITCNT
          PSHU  D
          JSR   CODECOMMA
          RTS

EXITUNLOOP: PULS  X
            LDD   ,X
            TFR   D,Y
EULoop:   CMPY  #0
            BEQ   EUDONE
            LEAS  8,S
            LEAY  -1,Y
            BRA   EULoop
EUDONE:   PULS  Y
            JMP   ,Y

```

```

CASEW:  LDD  #0
        PSHU D
        LDD  #TAGCASE
        PSHU D
        RTS

OF:      LDD  #OVER
        PSHU D
        JSR  CCALL
        LDD  #EQUALW
        PSHU D
        JSR  CCALL
        LDD  #ZBRANCH
        PSHU D
        JSR  CCALL
        LDD  #0
        PSHU D
        JSR  CODECOMMA
        LDD  CODEHERE
        SUBD #2
        PSHU D
        LDD  #DROP
        PSHU D
        JSR  CCALL
        LDD  #TAGOF
        PSHU D
        RTS

ENDOF:   PULU D
        CMPD #TAGOF
        BEQ  EOFOK
        JSR  CFERR
EOFOK:   PULU X
        LDD  #BRANCH
        PSHU D
        JSR  CCALL
        LDD  #0
        PSHU D
        JSR  CODECOMMA
        LDD  CODEHERE
        SUBD #2
        STD  NEWFLD
        LDD  CODEHERE
        PSHU D
        PSHU X
        JSR  PATCH
        LDD  NEWFLD
        PSHU D
        LDD  #TAGENDOF
        PSHU D
        RTS

ENDCASE: LDD  #DROP
        PSHU D
        JSR  CCALL
ECL00P:  PULU D
        CMPD #TAGCASE
        BEQ  ECDONE
        CMPD #TAGENDOF
        BEQ  ECPATCH
        JSR  CFERR
ECPATCH: PULU X
        LDD  CODEHERE
        PSHU D
        PSHU X
        JSR  PATCH
        BRA  ECL00P
ECDONE:  RTS

```

8.14 Compiling Words

```

; =====
; SECTION 13: COMPILING WORDS (IMMEDIATE/[ ]/'/COMPILE,/
; LITERAL/[ ]/POSTPONE/>BODY, SLITERAL, ABORT")
; =====
STATEW:    LDD    #STATE
            PSHU D
            RTS

IMMEDIATE: LDX    LATEST
            LDA    ,X
            ORA    #$80
            STA    ,X
            RTS

LBRACKET:  LDD    #0
            STD    STATE
            RTS

RBRACKET:  LDD    #-1
            STD    STATE
            RTS

TICK:      LDD    #32
            PSHU D
            JSR    WORD
            JSR    FIND
            PULU D
            CMPD  #0
            BNE    TICKOK
            PULU D
            LDD    #-13
            PSHU D
            JSR    THROW
TICKOK:    RTS

COMPILECOMMA: JMP  CCALL

CCALL:     PULU D
            LDX    CODEHERE
            LDA    #OPJSR
            STA    ,X+
            STD    ,X++
            STX    CODEHERE
            RTS

LITERALW:  LDD    #LIT
            PSHU D
            JSR    CCALL
            JSR    CODECOMMA
            RTS

BRACKTICK: LDD    STATE
            BNE    BTSTOK
            LDD    #-14
            PSHU D
            JSR    THROW
BTSTOK:    LDD    #32
            PSHU D
            JSR    WORD
            JSR    FIND
            PULU D
            CMPD  #0
            BNE    BTOK
            PULU D
            LDD    #-13
            PSHU D
            JSR    THROW
BTOK:      JSR    LITERALW
            RTS

```

```

POSTPONEW: LDD STATE
            BNE PPSTOK
            LDD #-14
            PSHU D
            JSR THROW
PPSTOK:    LDD #32
            PSHU D
            JSR WORD
            JSR FIND
            PULU D
            CMPD #0
            BNE PPFOUND
            PULU D
            LDD #-13
            PSHU D
            JSR THROW
PPFOUND:   CMPD #1
            BEQ PPIMM
            JSR LITERALW
            LDD #COMPILECOMMA
            PSHU D
            JSR CCALL
            RTS
PPIMM:     JSR COMPILECOMMA
            RTS

TOBODY:    PULU D
            ADDD #5
            TFR D,X
            LDD ,X
            PSHU D
            RTS

SLITERALW: LDD STATE
            BNE SLSTOK
            LDD #-14
            PSHU D
            JSR THROW
SLSTOK:    PULU D
            STD SCNT
            PULU D
            STD SPTR
            LDD #DOSTR
            PSHU D
            JSR CCALL
            LDX CODEHERE
            LDB SCNT+1
            STB ,X+
            LDY SPTR
            LDB SCNT+1
            BEQ SLEND
SLCPY:     LDA ,Y+
            STA ,X+
            DECB
            BNE SLCPY
SLEND:     STX CODEHERE
            RTS

DOABORTQUOTE: PULS X
              LDB ,X
              LEAX 1,X
              STX SPTR
              CLRA
              STD SCNT
              LDX SPTR
              LDB SCNT+1
              LEAX B,X
              PULU D
              CMPD #0
              BNE AQTHROW

```

```

        PSHS X
        RTS
AQTHROW: LDD  SPTR
        PSHU D
        LDD  SCNT
        PSHU D
        JSR  TYPE
        PSHS X
        LDD  #-2
        PSHU D
        JMP  THROW

ABORTQUOTE: LDD  STATE
            BNE  AQSTOK
            LDD  #-14
            PSHU D
            JSR  THROW
AQSTOK:   LDD  #34
            PSHU D
            JSR  WORD
            PULU X
            LDA  ,X
            STA  SCNT
            LEAX 1,X
            STX  SPTR
            LDD  #DOABORTQUOTE
            PSHU D
            JSR  CCALL
            LDX  CODEHERE
            LDA  SCNT
            STA  ,X+
            LDY  SPTR
            LDB  SCNT
            BEQ  AQEND
AQCPY:    LDA  ,Y+
            STA  ,X+
            DECB
            BNE  AQCPY
AQEND:    STX  CODEHERE
            RTS

BLW:      LDD  #32
            PSHU D
            RTS

TOINW:    LDD  #TOIN
            PSHU D
            RTS

SPANW:    LDD  #SPAN
            PSHU D
            RTS

TIBW:     LDD  #TIBBUF
            PSHU D
            RTS

NTIBW:    LDD  #NTIB
            PSHU D
            RTS

```

8.15 Stack Manipulation

```

; =====
; SECTION 14: STACK MANIPULATION (Core + Core Ext + return stack)
; =====
DUP:      LDD  ,U
            PSHU D

```

```

      RTS

DROP:  LEAU 2,U
      RTS

SWAP:  LDD ,U
      LDX 2,U
      STX ,U
      STD 2,U
      RTS

OVER:  LDD 2,U
      PSHU D
      RTS

ROT:   LDD ,U
      LDX 2,U
      LDY 4,U
      STY ,U
      STD 2,U
      STX 4,U
      RTS

QDUP:  LDD ,U
      CMPD #0
      BEQ QDUPDONE
      PSHU D
QDUPDONE: RTS

DEPTH: TFR U,D
      STD DEPTHMP
      LDD #SP0
      SUBD DEPTHMP
      LSRA
      RORB
      PSHU D
      RTS

DDUP:  LDD 2,U
      LDX ,U
      PSHU D
      PSHU X
      RTS

DDROP: LEAU 4,U
      RTS

DSWAP: LDD ,U
      STD MSCR
      LDD 2,U
      LDX 4,U
      LDY 6,U
      STD 6,U
      STX ,U
      STY 2,U
      LDD MSCR
      STD 4,U
      RTS

DOVER: LDD 6,U
      LDX 4,U
      PSHU D
      PSHU X
      RTS

NIP:   LDD ,U
      STD 2,U
      LEAU 2,U
      RTS

TUCK:  LDD ,U

```

```

        LDX    2,U
        PSHU   D
        STX    2,U
        STD    4,U
        RTS

PICK:    PULU   D
        LSLB
        ROLA
        LDD    D,U
        PSHU   D
        RTS

ROLL:    PULU   D
        CMPD   #0
        BEQ    ROLLDONE
        LSLB
        ROLA
        STD    RDST
        LEAX   D,U
        LDD    ,X
        STD    RVAL
RLOOP:   LDD    RDST
        CMPD   #2
        BLT    RSTORE
        LEAY   D,U
        SUBD   #2
        LEAX   D,U
        LDD    ,X
        STD    ,Y
        LDD    RDST
        SUBD   #2
        STD    RDST
        BRA    RLOOP
RSTORE:  LDD    RVAL
        STD    ,U
ROLLDONE: RTS

DROT:    LDD    10,U
        STD    TR1
        LDD    8,U
        STD    TR2
        LDD    6,U
        STD    10,U
        LDD    4,U
        STD    8,U
        LDD    2,U
        STD    6,U
        LDD    0,U
        STD    4,U
        LDD    TR2
        STD    ,U
        LDD    TR1
        STD    2,U
        RTS

TOR:     PULU   D
        PULS   X
        PSHS   D
        PSHS   X
        RTS

FROMR:   PULS   X
        PULS   D
        PSHS   X
        PSHU   D
        RTS

RFETCH:  PULS   X
        LDD    ,S
        PSHS   X

```

```

        PSHU D
        RTS

TWOTOR: PULU D
        STD R2A
        PULU D
        STD R2B
        PULS X
        LDD R2B
        PSHS D
        LDD R2A
        PSHS D
        PSHS X
        RTS

TWOFROMR: PULS X
          PULS D
          STD R2A
          PULS D
          STD R2B
          PSHS X
          LDD R2B
          PSHU D
          LDD R2A
          PSHU D
          RTS

TWORFETC: PULS X
          LDD ,S
          STD R2A
          LDD 2,S
          STD R2B
          PSHS X
          LDD R2B
          PSHU D
          LDD R2A
          PSHU D
          RTS

```

8.16 Arithmetic

```

; =====
; SECTION 15: ARITHMETIC (single + double + mixed precision)
; =====
PLUS:   PULU D
        ADDD ,U
        STD ,U
        RTS

; MSCR is declared once in the GLOBALS layout above - no local
; redeclaration needed here.
MINUS:  PULU D
        STD MSCR
        LDD ,U
        SUBD MSCR
        STD ,U
        RTS

NEGATE:  LDD ,U
        COMA
        COMB
        ADDD #1
        STD ,U
        RTS

ABSW:   LDD ,U
        BPL ABSDONE
        COMA

```

```

        COMB
        ADDD #1
        STD ,U
ABSDONE: RTS

MIN:     PULU D
        CMPD ,U
        BLT MINISN2
        RTS
MINISN2: STD ,U
        RTS

MAX:     PULU D
        CMPD ,U
        BGT MAXISN2
        RTS
MAXISN2: STD ,U
        RTS

ONEPLUS: LDD ,U
        ADDD #1
        STD ,U
        RTS

ONEMINUS: LDD ,U
        SUBD #1
        STD ,U
        RTS

TWOPLUS: LDD ,U
        ADDD #2
        STD ,U
        RTS

STAR:    PULU D
        STD MSCR
        LDD ,U
        CLR MSIGN
        BPL SNOFLIP1
        COM MSIGN
        COMA
        COMB
        ADDD #1
SNOFLIP1: STA MAHI
        STB MALO
        LDD MSCR
        BPL SNOFLIP2
        COM MSIGN
        COMA
        COMB
        ADDD #1
SNOFLIP2: STA MBHI
        STB MBLO
        LDA MALO
        LDB MBLO
        MUL
        STD MRESULT
        LDA MAHI
        LDB MBLO
        MUL
        LDA MRESULT
        PSHS B
        ADDA ,S+ ; was "ADDA B" - not valid 6809 syntax
        STA MRESULT
        LDA MALO
        LDB MBHI
        MUL
        LDA MRESULT
        PSHS B
        ADDA ,S+ ; was "ADDA B" - not valid 6809 syntax
        STA MRESULT

```

```

        LDD MRESULT
        TST MSIGN
        BEQ SDONE
        COMA
        COMB
        ADDD #1
SDONE:   STD ,U
        RTS

TWOSTAR: LDD ,U
        ASLB
        ROLA
        STD ,U
        RTS

UDIV16:  CLR DIVREM
        CLR DIVREM+1
        LDB #16
        STB DIVCNT
UD16LOOP: ASL DIVNUM+1
        ROL DIVNUM
        ROL DIVREM+1
        ROL DIVREM
        LDD DIVREM
        SUBD DIVDEN
        BLO UDSKIP
        STD DIVREM
        INC DIVNUM+1
UDSKIP:  DEC DIVCNT
        BNE UD16LOOP
        RTS

DIVCOMMON: PULU D
        STD DIVDEN
        CMPD #0
        BNE DCOK
        LDD #-10
        PSHU D
        JSR THROW
DCOK:    PULU D
        STD DIVNUM
        CLR DVSIGN
        CLR DNSIGN
        TST DIVNUM
        BPL DNPOS
        COM DNSIGN
        COM DVSIGN
        LDD DIVNUM
        COMA
        COMB
        ADDD #1
        STD DIVNUM
DNPOS:   TST DIVDEN
        BPL DVPOS
        COM DVSIGN
        LDD DIVDEN
        COMA
        COMB
        ADDD #1
        STD DIVDEN
DVPOS:   JSR UDIV16
        LDD DIVNUM
        TST DVSIGN
        BEQ DQPOS
        COMA
        COMB
        ADDD #1
        STD DIVNUM
DQPOS:   LDD DIVREM
        TST DNSIGN
        BEQ DCRPOS

```

```

        COMA
        COMB
        ADDD #1
        STD  DIVREM
DCRPOS:  RTS

SLASH:   JSR  DIVCOMMON
        LDD  DIVNUM
        PSHU D
        RTS

MODW:    JSR  DIVCOMMON
        LDD  DIVREM
        PSHU D
        RTS

SLASHMOD: JSR  DIVCOMMON
        LDD  DIVREM
        PSHU D
        LDD  DIVNUM
        PSHU D
        RTS

TWOSLASH: LDD  ,U
        ASRA
        RORB
        STD  ,U
        RTS

UMUL32:   LDA  MALO
        LDB  MBLO
        MUL
        STD  PRODLO
        CLR  PRODHI
        CLR  PRODHI+1
        LDA  MAHI
        LDB  MBLO
        MUL
        ADDB PRODLO
        STB  PRODLO
        ADCA #0
        ADDA PRODHI+1
        STA  PRODHI+1
        BCC  UM32A
        INC  PRODHI
UM32A:    LDA  MALO
        LDB  MBHI
        MUL
        ADDB PRODLO
        STB  PRODLO
        ADCA #0
        ADDA PRODHI+1
        STA  PRODHI+1
        BCC  UM32B
        INC  PRODHI
UM32B:    LDA  MAHI
        LDB  MBHI
        MUL
        ADDD PRODHI
        STD  PRODHI
        RTS

UDIV32:   CLR  DIVREM
        CLR  DIVREM+1
        LDB  #32
        STB  DIVCNT
UD32LP:   ASL  PRODLO+1
        ROL  PRODLO
        ROL  PRODHI+1
        ROL  PRODHI
        ROL  DIVREM+1

```

```

        ROL    DIVREM
        LDD    DIVREM
        SUBD   DIVDEN
        BLO    UD32SKIP
        STD    DIVREM
        INC    PRODLO+1
UD32SKIP: DEC    DIVCNT
        BNE    UD32LP
        RTS

MNEG32:  LDD    PRODLO
        COMA
        COMB
        STD    PRODLO
        LDD    PRODHI
        COMA
        COMB
        STD    PRODHI
        LDD    PRODLO
        ADDD   #1
        STD    PRODLO
        BCC    MN32DONE
        LDD    PRODHI
        ADDD   #1
        STD    PRODHI
MN32DONE: RTS

STARSLASHCOMMON:
        PULU   D
        STD    DIVDEN
        CMPD   #0
        BNE    SSOK
        LDD    #-10
        PSHU   D
        JSR    THROW
SSOK:    CLR    PSIGN
        TST    DIVDEN
        BPL    SSN3POS
        COM    PSIGN
        LDD    DIVDEN
        COMA
        COMB
        ADDD   #1
        STD    DIVDEN
SSN3POS: LDA    #0
        STA    PRSIGN
        PULU   D
        STD    MSCR
        TST    MSCR
        BPL    SSN2POS
        COM    PSIGN
        COM    PRSIGN
        LDD    MSCR
        COMA
        COMB
        ADDD   #1
        STD    MSCR
SSN2POS: LDA    MSCR
        STA    MBHI
        LDA    MSCR+1
        STA    MBLO
        PULU   D
        STD    MSCR
        TST    MSCR
        BPL    SSN1POS
        COM    PSIGN
        COM    PRSIGN
        LDD    MSCR
        COMA
        COMB
        ADDD   #1

```

```

        STD     MSCR
SSN1POS: LDA     MSCR
        STA     MAHI
        LDA     MSCR+1
        STA     MAL0
        JSR     UMUL32
        JSR     UDIV32
        LDD     PRODLO
        TST     PSIGN
        BEQ     SSQP0S
        COMA
        COMB
        ADDD    #1
        STD     PRODLO
SSQP0S: LDD     DIVREM
        TST     PRSIGN
        BEQ     SSRPOS
        COMA
        COMB
        ADDD    #1
        STD     DIVREM
SSRPOS: RTS

STARSLASH: JSR   STARSLASHCOMMON
          LDD    PRODLO
          PSHU   D
          RTS

STARSLASHMOD: JSR  STARSLASHCOMMON
              LDD  DIVREM
              PSHU D
              LDD  PRODLO
              PSHU D
              RTS

UMSTAR:  PULU   D
        STD    MSCR
        PULU   D
        STA    MAHI
        STB    MAL0
        LDD    MSCR
        STA    MBHI
        STB    MBLO
        JSR    UMUL32
        LDD    PRODLO
        PSHU   D
        LDD    PRODHI
        PSHU   D
        RTS

UMSLASHMOD: PULU D
            STD  DIVDEN
            CMPD #0
            BNE  UMOK
            LDD  #-10
            PSHU D
            JSR  THROW
UMOK:      PULU D
            STD  PRODHI
            PULU D
            STD  PRODLO
            JSR  UDIV32
            LDD  DIVREM
            PSHU D
            LDD  PRODLO
            PSHU D
            RTS

MSTAR:    PULU   D
        STD    MSCR
        PULU   D

```

```

        CLR    MSIGN
        BPL    MSN1POS
        COM    MSIGN
        COMA
        COMB
        ADDD   #1
MSN1POS: STA    MAHI
        STB    MAL0
        LDD    MSCR
        BPL    MSN2POS
        COM    MSIGN
        COMA
        COMB
        ADDD   #1
MSN2POS: STA    MBHI
        STB    MBL0
        JSR    UMUL32
        TST    MSIGN
        BEQ    MSDONE
        JSR    MNEG32
MSDONE:  LDD    PRODLO
        PSHU   D
        LDD    PRODHI
        PSHU   D
        RTS

SMSLASHREM: PULU D
        STD    DIVDEN
        CMPD   #0
        BNE    SMOK
        LDD    #-10
        PSHU   D
        JSR    THROW
SMOK:    PULU   D
        STD    PRODHI
        PULU   D
        STD    PRODLO
        CLR    DNSIGN
        CLR    DVSIGN
        TST    PRODHI
        BPL    SMDPOS
        COM    DNSIGN
        COM    DVSIGN
        JSR    MNEG32
SMDPOS:  LDD    DIVDEN
        BPL    SMDVPOS
        COM    DVSIGN
        LDD    DIVDEN
        COMA
        COMB
        ADDD   #1
SMDVPOS: STD    DIVDEN
        JSR    UDIV32
        LDD    DIVREM
        TST    DNSIGN
        BEQ    SMRPOS
        COMA
        COMB
        ADDD   #1
SMRPOS:  PSHU   D
        LDD    PRODLO
        TST    DVSIGN
        BEQ    SMQPOS
        COMA
        COMB
        ADDD   #1
SMQPOS:  PSHU   D
        RTS

FMSLASHMOD: PULU D
        STD    DIVDEN

```

```

      CMPD #0
      BNE FMOK
      LDD #-10
      PSHU D
      JSR THROW
FMOK:  PULU D
      STD PRODHI
      PULU D
      STD PRODLO
      CLR DNSIGN
      CLR DVSIGN
      CLR DVOWNSIGN
      TST PRODHI
      BPL FMDPOS
      COM DNSIGN
      COM DVSIGN
      JSR MNEG32
FMDPOS: LDD DIVDEN
      BPL FMDVPOS
      COM DVSIGN
      COM DVOWNSIGN
      LDD DIVDEN
      COMA
      COMB
      ADDD #1
FMDVPOS: STD DIVDEN
      JSR UDIV32
      TST DVSIGN
      BEQ FMNOFLOOR
      LDD DIVREM
      BEQ FMNOFLOOR
      LDD PRODLO
      ADDD #1
      STD PRODLO
      LDD DIVDEN
      SUBD DIVREM
      STD DIVREM
FMNOFLOOR: LDD DIVREM
      TST DVOWNSIGN
      BEQ FMRPOS
      COMA
      COMB
      ADDD #1
FMRPOS: PSHU D
      LDD PRODLO
      TST DVSIGN
      BEQ FMQPOS
      COMA
      COMB
      ADDD #1
FMQPOS: PSHU D
      RTS

DPLUS: PULU D
      STD MSCR
      PULU D
      STD MSCR2
      PULU D
      STD MSCR3
      PULU D
      ADDD MSCR2
      STD MSCR4
      BCC DPNOCY
      LDD MSCR3
      ADDD MSCR
      ADDD #1
      BRA DPHIDONE
DPNOCY: LDD MSCR3
      ADDD MSCR
DPHIDONE: STD MSCR3
      LDD MSCR4

```

```

        PSHU D
        LDD MSCR3
        PSHU D
        RTS

DMINUS: PULU D
        STD MSCR
        PULU D
        STD MSCR2
        PULU D
        STD MSCR3
        PULU D
        SUBD MSCR2
        STD MSCR4
        BCC DMNOBOR
        LDD MSCR3
        SUBD MSCR
        SUBD #1
        BRA DMHIDONE
DMNOBOR: LDD MSCR3
        SUBD MSCR
DMHIDONE: STD MSCR3
        LDD MSCR4
        PSHU D
        LDD MSCR3
        PSHU D
        RTS

DNEGATEW: PULU D
        STD PRODHI
        PULU D
        STD PRODLO
        JSR MNEG32
        LDD PRODLO
        PSHU D
        LDD PRODHI
        PSHU D
        RTS

DABSW: PULU D
        STD PRODHI
        PULU D
        STD PRODLO
        TST PRODHI
        BPL DABSDONE
        JSR MNEG32
DABSDONE: LDD PRODLO
        PSHU D
        LDD PRODHI
        PSHU D
        RTS

MPLUS: PULU D
        STD MSCR2
        BPL MPPOSN
        LDD #-1
        BRA MPSIGNED
MPPOSN: LDD #0
MPSIGNED: STD MSCR
        PULU D
        STD MSCR3
        PULU D
        ADDD MSCR2
        STD MSCR4
        BCC MPNOCY
        LDD MSCR3
        ADDD MSCR
        ADDD #1
        BRA MPHIDONE
MPNOCY: LDD MSCR3
        ADDD MSCR

```

```

MPHIDONE: STD MSCR3
           LDD MSCR4
           PSHU D
           LDD MSCR3
           PSHU D
           RTS

STOD:     PULU D
           PSHU D
           BPL SDPOS
           LDD #-1
           BRA SDPUSH
SDPOS:    LDD #0
SDPUSH:   PSHU D
           RTS

DTOS:     PULU D
           RTS

DMAXW:    PULU D
           STD MSCR
           PULU D
           STD MSCR2
           PULU D
           STD MSCR3
           PULU D
           STD MSCR4
           LDD MSCR3
           CMPD MSCR
           BGT DMXD1
           BLT DMXD2
           LDD MSCR4
           CMPD MSCR2
           BHS DMXD1
DMXD2:    LDD MSCR2
           PSHU D
           LDD MSCR
           PSHU D
           RTS
DMXD1:    LDD MSCR4
           PSHU D
           LDD MSCR3
           PSHU D
           RTS

DMINW:    PULU D
           STD MSCR
           PULU D
           STD MSCR2
           PULU D
           STD MSCR3
           PULU D
           STD MSCR4
           LDD MSCR3
           CMPD MSCR
           BLT DMND1
           BGT DMND2
           LDD MSCR4
           CMPD MSCR2
           BLS DMND1
DMND2:    LDD MSCR2
           PSHU D
           LDD MSCR
           PSHU D
           RTS
DMND1:    LDD MSCR4
           PSHU D
           LDD MSCR3
           PSHU D
           RTS

```

8.17 Logic, Shifts & Address Arithmetic

```

; =====
; SECTION 16: LOGIC / SHIFTS / ADDRESS ARITHMETIC
; =====
ANDW:   PULU   D
        ANDA   ,U
        ANDB   1,U
        STD    ,U
        RTS

ORW:     PULU   D
        ORA    ,U
        ORB    1,U
        STD    ,U
        RTS

XORW:    PULU   D
        EORA   ,U
        EORB   1,U
        STD    ,U
        RTS

INVERT:   LDD    ,U
        COMA
        COMB
        STD    ,U
        RTS

LSHIFT:   PULU   D
        STB    SHCNT
        LDD    ,U
LSLOOP:   LDB    SHCNT
        BEQ    LSDONE
        ASL    1,U
        ROL    ,U
        DEC    SHCNT
        BRA    LSLLOOP
LSDONE:   RTS

RSHIFT:   PULU   D
        STB    SHCNT
RSLLOOP:   LDB    SHCNT
        BEQ    RSDONE
        LSR    ,U
        ROR    1,U
        DEC    SHCNT
        BRA    RSLLOOP
RSDONE:   RTS

CELLSW:   LDD    ,U
        ASLB
        ROLA
        STD    ,U
        RTS

CELLPLUS: LDD    ,U
        ADDD   #2
        STD    ,U
        RTS

CHARSW:   RTS

CHARPLUS: LDD    ,U
        ADDD   #1
        STD    ,U
        RTS

```

```
ALIGNW: RTS
ALIGNEDW: RTS
```

8.18 Comparison

```
; =====
; SECTION 17: COMPARISON
; =====
EQUALW: PULU D
        CMPD ,U
        BEQ EQTRUE
        LDD #FALSEV
        STD ,U
        RTS
EQTRUE: LDD #TRUEV
        STD ,U
        RTS

LESSW:  PULU D
        STD MSCR
        LDD ,U
        CMPD MSCR
        BLT LTTRUE
        LDD #FALSEV
        STD ,U
        RTS
LTTRUE: LDD #TRUEV
        STD ,U
        RTS

GREATERW: PULU D
          STD MSCR
          LDD ,U
          CMPD MSCR
          BGT GTTRUE
          LDD #FALSEV
          STD ,U
          RTS
GTTRUE:  LDD #TRUEV
          STD ,U
          RTS

ZEROEQ:  LDD ,U
          BEQ ZEQTRUE
          LDD #FALSEV
          STD ,U
          RTS
ZEQTRUE: LDD #TRUEV
          STD ,U
          RTS

ZEROLT:  LDD ,U
          BMI ZLTTRUE
          LDD #FALSEV
          STD ,U
          RTS
ZLTTRUE: LDD #TRUEV
          STD ,U
          RTS

ULESSW:  PULU D
          STD MSCR
          LDD ,U
          CMPD MSCR
          BLO ULTRUE
          LDD #FALSEV
          STD ,U
```

```

      RTS
ULTRUE: LDD #TRUEV
        STD ,U
        RTS

NOTEQUAL: JSR EQUALW
          LDD ,U
          COMA
          COMB
          STD ,U
          RTS

ZERONE: JSR ZEROEQ
        LDD ,U
        COMA
        COMB
        STD ,U
        RTS

ZEROGT: LDD ,U
        BEQ ZGTFALSE
        BMI ZGTFALSE
        LDD #TRUEV
        STD ,U
        RTS
ZGTFALSE: LDD #FALSEV
          STD ,U
          RTS

UGREATER: PULU D
          STD MSCR
          LDD ,U
          CMPD MSCR
          BLO UGFALSE
          BEQ UGFALSE
          LDD #TRUEV
          STD ,U
          RTS
UGFALSE: LDD #FALSEV
          STD ,U
          RTS

WITHINW: PULU D
          STD MSCR
          PULU D
          STD MSCR2
          LDD ,U
          SUBD MSCR2
          STD MSCR3
          LDD MSCR
          SUBD MSCR2
          STD MSCR
          LDD MSCR3
          CMPD MSCR
          BLO WITHTRUE
          LDD #FALSEV
          STD ,U
          RTS
WITHTRUE: LDD #TRUEV
          STD ,U
          RTS

DEQUAL: PULU D
          STD MSCR
          PULU D
          STD MSCR2
          PULU D
          STD MSCR3
          PULU D
          CMPD MSCR2
          BNE DEQFALSE

```

```

        LDD    MSCR3
        CMPD   MSCR
        BNE    DEQFALSE
        LDD    #TRUEV
        PSHU   D
        RTS
DEQFALSE: LDD    #FALSEV
        PSHU   D
        RTS

DLESSW:  PULU   D
        STD    MSCR
        PULU   D
        STD    MSCR2
        PULU   D
        STD    MSCR3
        PULU   D
        STD    MSCR4
        LDD    MSCR3
        CMPD   MSCR
        BLT    DLTRUE
        BGT    DLFALSE
        LDD    MSCR4
        CMPD   MSCR2
        BLO    DLTRUE
DLFALSE: LDD    #FALSEV
        PSHU   D
        RTS
DLTRUE:  LDD    #TRUEV
        PSHU   D
        RTS

DULESSW: PULU   D
        STD    MSCR
        PULU   D
        STD    MSCR2
        PULU   D
        STD    MSCR3
        PULU   D
        STD    MSCR4
        LDD    MSCR3
        CMPD   MSCR
        BLO    DULTRUE
        BHI    DULFALSE
        LDD    MSCR4
        CMPD   MSCR2
        BLO    DULTRUE
DULFALSE: LDD    #FALSEV
        PSHU   D
        RTS
DULTRUE: LDD    #TRUEV
        PSHU   D
        RTS

```

8.19 Memory

```

; =====
; SECTION 18: MEMORY (fetch/store, block ops)
; =====
STOREW:  PULU   X
        PULU   D
        STD    ,X
        RTS

CFETCH:  PULU   X
        LDB    ,X
        CLRA
        PSHU   D

```

```

      RTS

CSTOREW: PULU X
          PULU D
          STB ,X
          RTS

PLUSSTORE: PULU X
            PULU D
            ADDD ,X
            STD ,X
            RTS

DFETCH:  PULU X
          LDD 2,X
          PSHU D
          LDD ,X
          PSHU D
          RTS

DSTORE:  PULU X
          PULU D
          STD 2,X
          PULU D
          STD ,X
          RTS

CMOVEW:  PULU D
          STD MVCNT
          PULU D
          STD MVDST
          PULU D
          STD MVSRC
          LDX MVSRC
          LDY MVDST
CMVLOOP: LDD MVCNT
          BEQ CMDONE
          LDA ,X+
          STA ,Y+
          SUBD #1
          STD MVCNT
          BRA CMVLOOP

CMDONE:  RTS

CMOVEGT: PULU D
          STD MVCNT
          PULU D
          STD MVDST
          PULU D
          STD MVSRC
          LDD MVCNT
          BEQ CGDONE
          LDX MVSRC
          LEAX D,X
          LEAX -1,X
          LDY MVDST
          LEAY D,Y
          LEAY -1,Y
CGLLOOP: LDA ,X
          STA ,Y
          LEAX -1,X
          LEAY -1,Y
          LDD MVCNT
          SUBD #1
          STD MVCNT
          BNE CGLLOOP

CGDONE:  RTS

MOVEW:   PULU D
          STD MVCNT
          PULU D

```

```

        STD     MVDST
        PULU    D
        STD     MVSRC
        LDD     MVDST
        CMPD    MVSRC
        BLS     MVLOW
        LDD     MVSRC
        PSHU    D
        LDD     MVDST
        PSHU    D
        LDD     MVCNT
        PSHU    D
        JMP     CMOVEGT
MVLOW:   LDD     MVSRC
        PSHU    D
        LDD     MVDST
        PSHU    D
        LDD     MVCNT
        PSHU    D
        JMP     CMOVEV

FILLW:   PULU    D
        STB     FILLCHR
        PULU    D
        STD     FILLCNT
        PULU    D
        STD     FILLADDR
        LDX     FILLADDR
FILLLOOP: LDD     FILLCNT
        BEQ     FDONE
        LDA     FILLCHR
        STA     ,X+
        SUBD    #1
        STD     FILLCNT
        BRA     FILLLOOP
FDONE:   RTS

ERASEW:  LDD     #0
        PSHU    D
        JMP     FILLW

```

8.20 String Words

```

; =====
; SECTION 19: STRING WORDS
; =====
DOSTR:   PULS    X
        LDB     ,X
        LEAX    1,X
        PSHU    X
        CLRA
        PSHU    D
        LEAX    B,X
        PSHS    X
        RTS

SQUOTE:  LDD     #34
        PSHU    D
        JSR     WORD
        PULU    X
        LDA     ,X
        STA     SCNT
        LEAX    1,X
        STX     SPTR
        LDD     STATE
        BEQ     SQINTERP

        LDD     #DOSTR

```

```

        PSHU D
        JSR CCALL
        LDX CODEHERE
        LDA SCNT
        STA ,X+
        LDY SPTR
        LDB SCNT
        BEQ SQEND
SQCPY:   LDA ,Y+
        STA ,X+
        DECB
        BNE SQCPY
SQEND:   STX CODEHERE
        RTS

SQINTERP: LDY SPTR
          LDB SCNT
          LDX #SIBUF
          BEQ SQIEND
SQICPY:   LDA ,Y+
          STA ,X+
          DECB
          BNE SQICPY
SQIEND:   LDX #SIBUF
          PSHU X
          CLRA
          LDB SCNT
          PSHU D
          RTS

DOTSTR:   PULS X
          LDB ,X
          LEAX 1,X
          STX SPTR
          CLRA
          STD SCNT
          LEAX B,X
          PSHS X
          LDX SPTR
          PSHU X
          LDD SCNT
          PSHU D
          JSR TYPE
          RTS

DOTQUOTE: LDD #34
          PSHU D
          JSR WORD
          PULU X
          LDA ,X
          STA SCNT
          LEAX 1,X
          STX SPTR
          LDD #DOTSTR
          PSHU D
          JSR CCALL
          LDX CODEHERE
          LDA SCNT
          STA ,X+
          LDY SPTR
          LDB SCNT
          BEQ DQEND
DQCPY:   LDA ,Y+
          STA ,X+
          DECB
          BNE DQCPY
DQEND:   STX CODEHERE
          RTS

TYPE:     PULU D
          STD TYPECNT

```

```

        PULU D
        STD TYPEADDR
TYLOOP: LDD TYPECNT
        BEQ TYDONE
        LDX TYPEADDR
        LDA ,X+
        STX TYPEADDR
        TFR A,B
        CLRA
        PSHU D
        JSR EMIT
        LDD TYPECNT
        SUBD #1
        STD TYPECNT
        BRA TYLOOP
TYDONE: RTS

COUNT: PULU X
        LDB ,X
        CLRA
        STD MSCR
        LEAX 1,X
        PSHU X
        LDD MSCR
        PSHU D
        RTS

CHARW:  LDD #32
        PSHU D
        JSR WORD
        PULU X
        LDB 1,X
        CLRA
        PSHU D
        RTS

BRACKCHAR: LDD STATE
           BNE BCSTOK
           LDD #-14
           PSHU D
           JSR THROW
BCSTOK:   LDD #32
           PSHU D
           JSR WORD
           PULU X
           LDB 1,X
           CLRA
           PSHU D
           JSR LITERALW
           RTS

PARSEW:  PULU D
        STB PDELIM
        LDD TOIN
        LDX SRCADDR
        LEAX D,X
        STX PSTART
        LDD SRCLEN
        SUBD TOIN
        TFR D,Y
        LDD #0
        STD PLEN
PSCAN:   CMPY #0
        BEQ PDONE
        LDA ,X
        CMPA PDELIM
        BEQ PFOUND
        LEAX 1,X
        LEAY -1,Y
        LDD PLEN
        ADDD #1

```

```

        STD  PLEN
        BRA  PSCAN
PFOUND:  LEAX 1,X
        LEAY -1,Y
PDONE:   TFR  X,D
        SUBD SRCADDR
        STD  TOIN
        LDX  PSTART
        PSHU X
        LDD  PLEN
        PSHU D
        RTS

PARSENAME: LDD  TOIN
          LDX  SRCADDR
          LEAX D,X
          LDD  SRCLEN
          SUBD TOIN
          TFR  D,Y
PNSKIP:   CMPY #0
          BEQ  PNEMPTY
          LDA  ,X
          CMPA #32
          BNE  PNSTART
          LEAX 1,X
          LEAY -1,Y
          BRA  PNSKIP
PNSTART:  STX  PSTART
          LDD  #0
          STD  PLEN
PNSCAN:   CMPY #0
          BEQ  PNDONE
          LDA  ,X
          CMPA #32
          BEQ  PNFOUND
          LEAX 1,X
          LEAY -1,Y
          LDD  PLEN
          ADDD #1
          STD  PLEN
          BRA  PNSCAN
PNFOUND:  LEAX 1,X
          LEAY -1,Y
PNDONE:   TFR  X,D
          SUBD SRCADDR
          STD  TOIN
          LDX  PSTART
          PSHU X
          LDD  PLEN
          PSHU D
          RTS
PNEMPTY:  LDD  SRCLEN
          LDX  SRCADDR
          LEAX D,X
          STD  TOIN
          PSHU X
          LDD  #0
          PSHU D
          RTS

SLASHSTRING: PULU D
            STD  MSCR
            PULU D
            SUBD MSCR
            STD  MSCR2
            PULU D
            ADDD MSCR
            PSHU D
            LDD  MSCR2
            PSHU D
            RTS

```

```

DASHTRAILING: LDD ,U
               STD PLEN
DTLOOP:       LDD PLEN
               BEQ DTDONE
               LDX 2,U
               LEAX D,X
               LEAX -1,X
               LDA ,X
               CMPA #32
               BNE DTDONE
               LDD PLEN
               SUBD #1
               STD PLEN
               BRA DTLOOP
DTDONE:       LDD PLEN
               STD ,U
               RTS

COMPAREW:     PULU D
               STD CMPL2
               PULU D
               STD CMPA2
               PULU D
               STD CMPL1
               PULU D
               STD CMPA1
               LDD CMPL1
               CMPD CMPL2
               BLS CMMINIS1
               LDD CMPL2
               BRA CMMINSET
CMMINIS1:     LDD CMPL1
CMMINSET:     STD CMPMIN
               LDX CMPA1
               LDY CMPA2
CMPLOOP:      LDD CMPMIN
               BEQ CMTIEBREAK
               LDA ,X+
               CMPA ,Y
               BLO CMLT
               BHI CMGT
               LEAY 1,Y
               LDD CMPMIN
               SUBD #1
               STD CMPMIN
               BRA CMPLOOP
CMTIEBREAK:   LDD CMPL1
               CMPD CMPL2
               BLO CMLT
               BHI CMGT
               LDD #0
               PSHU D
               RTS
CMLT:         LDD #-1
               PSHU D
               RTS
CMGT:         LDD #1
               PSHU D
               RTS

SEARCHW:      PULU D
               STD SRCH2L
               PULU D
               STD SRCH2
               PULU D
               STD SRCH1L
               PULU D
               STD SRCH1
               LDD SRCH2L
               BEQ SRCHNOTFOUND

```

```

LDD SRCH1L
SUBD SRCH2L
BLT SRCHNOTFOUND
ADDD #1
STD SRCHPOS
LDD #0
STD SRCHI
SPOSLOOP: LDD SRCHI
CMPD SRCHPOS
BEQ SRCHNOTFOUND
LDX SRCH1
LDD SRCHI
LEAX D,X
LDY SRCH2
LDD SRCH2L
STD MSCR3
SMATCH: LDD MSCR3
BEQ SFOUND
LDA ,X+
CMPA ,Y+
BNE SNOMATCH
LDD MSCR3
SUBD #1
STD MSCR3
BRA SMATCH
SNOMATCH: LDD SRCHI
ADDD #1
STD SRCHI
BRA SPOSLOOP
SFOUND: LDX SRCH1
LDD SRCHI
LEAX D,X
PSHU X
LDD SRCH2L
PSHU D
LDD #TRUEV
PSHU D
RTS
SRCHNOTFOUND: LDD SRCH1
PSHU D
LDD SRCH1L
PSHU D
LDD #FALSEV
PSHU D
RTS

SNAMEW: PULU D
STD SNTARGET
LDD LATEST
STD SNXT
SNLOOP: LDD SNXT
BEQ SNNOTFOUND
LDX SNXT
LDA ,X
STA HDRFLAGS
LEAX 1,X
LDB HDRFLAGS
ANDB #$1F
CLRA
LEAX D,X
LEAX 2,X
LDD ,X
CMPD SNTARGET
BEQ SNFOUND
LDX SNXT
LEAX 1,X
LDB HDRFLAGS
ANDB #$1F
CLRA
LEAX D,X
LDD ,X

```

```

        STD     SNXT
        BRA     SNLOOP
SNFOUND: LDX     SNXT
        LEAX    1,X
        PSHU    X
        LDX     SNXT
        LDA     ,X
        ANDA    #$1F
        CLRB
        TFR     A,B
        CLRA
        PSHU    D
        RTS
SNNOTFOUND: LDD #0
        PSHU    D
        PSHU    D
        RTS

UNESCAPEW: PULU D
        STD     UESRCLEN
        LDD     ,U
        STD     UEADDR
        STD     UEDST
        LDD     #0
        STD     UEOUTLEN
        LDX     UEADDR
        LDY     UEDST
UELOOP:  LDD     UESRCLEN
        BEQ     UEDONE
        LDA     ,X+
        CMPA    #'\'
        BNE     UEPLAIN
        LDD     UESRCLEN
        SUBD    #1
        STD     UESRCLEN
        BEQ     UEPLAIN
        LDA     ,X+
        CMPA    #'n'
        BNE     UECKT
        LDA     #10
        BRA     UEEMIT
UECKT:   CMPA    #'t'
        BNE     UECKBS
        LDA     #9
        BRA     UEEMIT
UECKBS:  CMPA    #'\'
        BEQ     UEEMIT
        CMPA    #'"'
        BEQ     UEEMIT
        PSHS    A
        LDA     #'\'
        STA     ,Y+
        LDD     UEOUTLEN
        ADDD    #1
        STD     UEOUTLEN
        PULS    A
UEEMIT:  STA     ,Y+
        LDD     UEOUTLEN
        ADDD    #1
        STD     UEOUTLEN
        LDD     UESRCLEN
        SUBD    #1
        STD     UESRCLEN
        BRA     UELOOP
UEPLAIN: STA     ,Y+
        LDD     UEOUTLEN
        ADDD    #1
        STD     UEOUTLEN
        LDD     UESRCLEN
        SUBD    #1
        STD     UESRCLEN

```

```

        BRA   UELOOP
UEDONE:  LDD   UEOUTLEN
        STD   ,U
        RTS

; REPLACES/SUBSTITUTE - single-slot simplified version, per
; the explicit scoping-down discussed in the source conversation
REPLACESW: PULU D
        STD   REPLNLEN
        PULU D
        STD   REPLNAME
        PULU D
        STD   REPLVLEN
        PULU D
        STD   REPLVAL
        RTS

SUBCOPY: STD   SUBCOPYCNT
        STX   SUBCOPYSRC
SUBCPLP: LDD   SUBCOPYCNT
        BEQ   SUBCPDONE
        LDD   SUBOUTLEN
        CMPD  SUBDESTCAP
        LBHS  SUBOVERFLOW      ; was BHS - out of short-branch range
        LDX   SUBCOPYSRC
        LDA   ,X+
        STX   SUBCOPYSRC
        LDY   SUBWPTR
        STA   ,Y+
        STY   SUBWPTR
        LDD   SUBOUTLEN
        ADDD  #1
        STD   SUBOUTLEN
        LDD   SUBCOPYCNT
        SUBD  #1
        STD   SUBCOPYCNT
        BRA   SUBCPLP
SUBCPDONE: RTS

SUBSTITUTEW: PULU D
        STD   SUBDESTCAP
        PULU D
        STD   SUBDESTADR
        PULU D
        STD   SUBSRCLEN
        PULU D
        STD   SUBSRCADR

        LDD   SUBSRCADR
        PSHU D
        LDD   SUBSRCLEN
        PSHU D
        LDD   REPLNAME
        PSHU D
        LDD   REPLNLEN
        PSHU D
        JSR   SEARCHW
        PULU D
        CMPD  #0
        BEQ   SUBNOTFOUND
        PULU D
        PULU D
        STD   MSCR4

        LDY   SUBDESTADR
        STY   SUBWPTR
        LDD   #0
        STD   SUBOUTLEN

        LDD   MSCR4
        SUBD  SUBSRCADR

```

```

        STD  MSCR3
        LDX  SUBSRCADR
        LDD  MSCR3
        JSR  SUBCOPY

        LDX  REPLVAL
        LDD  REPLVLEN
        JSR  SUBCOPY

        LDD  MSCR4
        ADDD REPLNLEN
        STD  MSCR2
        LDD  SUBSRCLEN
        SUBD MSCR3
        SUBD REPLNLEN
        STD  MSCR
        LDX  MSCR2
        LDD  MSCR
        JSR  SUBCOPY

        LDX  SUBDESTADR
        PSHU X
        LDD  SUBOUTLEN
        PSHU D
        RTS

SUBNOTFOUND: LDY SUBDESTADR
              STY SUBWPTR
              LDD #0
              STD SUBOUTLEN
              LDX SUBSRCADR
              LDD SUBSRCLEN
              JSR SUBCOPY
              LDX SUBDESTADR
              PSHU X
              LDD SUBOUTLEN
              PSHU D
              RTS

SUBOVERFLOW: LDD #-1
              PSHU D
              JSR THROW

```

8.21 Numeric Output

```

; =====
; SECTION 20: NUMERIC OUTPUT (pictured + direct)
; =====
LTNUM:   JSR   PADW
         PULU  D
         STD   HLD
         RTS

HOLD:    PULU  D
         LDX   HLD
         LEAX  -1,X
         STX   HLD
         STB   ,X
         RTS

HOLDS:   PULU  D
         STD   HSLEN
         PULU  D
         STD   HSADDR
HSLLOOP: LDD   HSLEN
         BEQ   HSDONE
         SUBD  #1
         STD   HSLEN

```

```

        LDX    HSADDR
        LDD    HSLLEN
        LEAX   D,X
        LDA    ,X
        TFR    A,B
        CLRA
        PSHU   D
        JSR    HOLD
        BRA    HSLLOOP
HSDONE:  RTS

NUMSIGN: PULU   D
        STD    UDHI
        PULU   D
        STD    UDLO
        JSR    UDDIGIT
        LDA    REM
        CMPA   #10
        BLO    NDIGIT
        ADDA   #'A' -10
        BRA    NHOLD
NDIGIT:  ADDA   #'0'
NHOLD:   TFR    A,B
        CLRA
        PSHU   D
        JSR    HOLD
        LDD    UDLO
        PSHU   D
        LDD    UDHI
        PSHU   D
        RTS

UDDIGIT: CLR    REM
        LDB    #32
        STB    DCNT
UDDLLOOP: ASL    UDLO+1
        ROL    UDLO
        ROL    UDHI+1
        ROL    UDHI
        ROL    REM
        LDA    REM
        CMPA   BASE+1
        BLO    UDNEXT
        SUBA   BASE+1
        STA    REM
        INC    UDLO+1
UDNEXT:  DEC    DCNT
        BNE    UDDLLOOP
        RTS

NUMSIGNS: JSR    NUMSIGN
        LDD    UDHI
        BNE    NUMSIGNS
        LDD    UDLO
        BNE    NUMSIGNS
        RTS

SIGN:    PULU   D
        BPL    SIGNDONE
        LDD    #'-'
        PSHU   D
        JSR    HOLD
SIGNDONE: RTS

NUMGT:   PULU   D
        PULU   D
        LDX    HLD
        PSHU   X
        JSR    PADW
        PULU   D
        SUBD   HLD

```

```

        PSHU D
        RTS

DOT:    PULU D
        STD SAVEN
        BPL DABSOK
        COMA
        COMB
        ADDD #1
DABSOK: PSHU D
        LDD #0
        PSHU D
        JSR LTNUM
        JSR NUMSIGNS
        LDD SAVEN
        PSHU D
        JSR SIGN
        JSR NUMGT
        JSR TYPE
        LDD #32
        PSHU D
        JSR EMIT
        RTS

UDOT:   PULU D
        PSHU D
        LDD #0
        PSHU D
        JSR LTNUM
        JSR NUMSIGNS
        JSR NUMGT
        JSR TYPE
        LDD #32
        PSHU D
        JSR EMIT
        RTS

DOTR:   PULU D
        STD DRWIDTH
        PULU D
        STD SAVEN
        BPL DRABSOK
        COMA
        COMB
        ADDD #1
DRABSOK: PSHU D
        LDD #0
        PSHU D
        JSR LTNUM
        JSR NUMSIGNS
        LDD SAVEN
        PSHU D
        JSR SIGN
        JSR NUMGT
        PULU D
        STD DRLEN
        PULU D
        STD DRADDR
        LDD DRWIDTH
        SUBD DRLEN
        BLE DRNOPAD
        STD DRPAD
DRPADLP: LDD DRPAD
        BEQ DRNOPAD
        SUBD #1
        STD DRPAD
        LDD #32
        PSHU D
        JSR EMIT
        BRA DRPADLP
DRNOPAD: LDX DRADDR

```

```

        PSHU X
        LDD DRLEN
        PSHU D
        JSR TYPE
        RTS

UDOTR:  PULU D
        STD DRWIDTH
        PULU D
        PSHU D
        LDD #0
        PSHU D
        JSR LTNUM
        JSR NUMSIGNS
        JSR NUMGT
        PULU D
        STD DRLEN
        PULU D
        STD DRADDR
        LDD DRWIDTH
        SUBD DRLEN
        BLE UDRNOPAD
        STD DRPAD
UDRPADLP: LDD DRPAD
        BEQ UDRNOPAD
        SUBD #1
        STD DRPAD
        LDD #32
        PSHU D
        JSR EMIT
        BRA UDRPADLP
UDRNOPAD: LDX DRADDR
        PSHU X
        LDD DRLEN
        PSHU D
        JSR TYPE
        RTS

QMARK:  PULU X
        LDD ,X
        PSHU D
        JSR DOT
        RTS

DDOT:   PULU D
        STD PRODHI
        PULU D
        STD PRODLO
        LDD PRODHI
        STD SAVEN
        BPL DDPOS
        JSR MNEG32
DDPOS:  LDD PRODLO
        PSHU D
        LDD PRODHI
        PSHU D
        JSR LTNUM
        JSR NUMSIGNS
        LDD SAVEN
        PSHU D
        JSR SIGN
        JSR NUMGT
        JSR TYPE
        LDD #32
        PSHU D
        JSR EMIT
        RTS

DDOTR:  PULU D
        STD DRWIDTH
        PULU D

```

```

        STD  PRODHI
        PULU D
        STD  PRODLO
        LDD  PRODHI
        STD  SAVEN
        BPL  DDRPOS
        JSR  MNEG32
DDRPOS:  LDD  PRODLO
        PSHU D
        LDD  PRODHI
        PSHU D
        JSR  LTNUM
        JSR  NUMSIGNS
        LDD  SAVEN
        PSHU D
        JSR  SIGN
        JSR  NUMGT
        PULU D
        STD  DRLEN
        PULU D
        STD  DRADDR
        LDD  DRWIDTH
        SUBD DRLEN
        BLE  DRDNOPAD
        STD  DRPAD
DRDPADLP: LDD  DRPAD
        BEQ  DRDNOPAD
        SUBD #1
        STD  DRPAD
        LDD  #32
        PSHU D
        JSR  EMIT
        BRA  DRDPADLP
DRDNOPAD: LDX  DRADDR
        PSHU X
        LDD  DRLEN
        PSHU D
        JSR  TYPE
        RTS

```

8.22 Base / Radix Control

```

; =====
; SECTION 21: BASE / RADIX CONTROL
; =====
BASEW:  LDD  #BASE
        PSHU D
        RTS

DECIMAL: LDD  #10
        STD  BASE
        RTS

HEXW:   LDD  #16
        STD  BASE
        RTS

BINARYW: LDD  #2
        STD  BASE
        RTS

```

8.23 Output Formatting

```

; =====
; SECTION 22: OUTPUT FORMATTING (CR/SPACE/SPACES)
; =====

```

```

CRW:    LDD    #13
        PSHU   D
        JSR    EMIT
        LDD    #10
        PSHU   D
        JSR    EMIT
        RTS

SPACEW: LDD    #32
        PSHU   D
        JSR    EMIT
        RTS

SPACESW: PULU   D
        STD    SHCNT2
SLOOP:  LDD    SHCNT2
        BLE    SPDONE
        LDD    #32
        PSHU   D
        JSR    EMIT
        LDD    SHCNT2
        SUBD   #1
        STD    SHCNT2
        BRA    SLOOP
SPDONE: RTS

```

8.24 Comment Words

```

; =====
; SECTION 23: COMMENT WORDS
; =====
LPAREN: LDD    #' ) '
        PSHU   D
        JSR    WORD
        RTS

BACKSLASH: LDD  SRCLEN
           STD  TOIN
           RTS

```

8.25 Environmental Query / SOURCE / REFILL / EVALUATE

```

; =====
; SECTION 24: ENVIRONMENTAL QUERY / SOURCE / REFILL / EVALUATE
; =====
SOURCEW: LDD    SRCADDR
        PSHU   D
        LDD    SRCLEN
        PSHU   D
        RTS

SOURCEID: LDD    SRCID
        PSHU   D
        RTS

REFILLW: LDD    SRCID
        BEQ    RFTERM
        LDD    #FALSEV
        PSHU   D
        RTS

RFTERM:  JSR    QUERY
        LDD    #TRUEV
        PSHU   D
        RTS

EVALUATEW: LDD  SRCADDR

```

```

STD EVSAVEA
LDD SRCLEN
STD EVSAVEL
LDD SRCID
STD EVSAVEI
LDD TOIN
STD EVSAVET
PULU D
STD SRCLEN
PULU D
STD SRCADDR
LDD #-1
STD SRCID
LDD #0
STD TOIN
JSR INTERPRET
LDD EVSAVEA
STD SRCADDR
LDD EVSAVEL
STD SRCLEN
LDD EVSAVEI
STD SRCID
LDD EVSAVET
STD TOIN
RTS

; ENVIRONMENT? - dispatcher complete; table has only the
; entries that could be derived without fabricating unfixed
; capacities (/HOLD, /PAD were explicitly left out - see the
; source conversation's ENVTABLE discussion). MAX-D/MAX-UD
; and WORDLISTS/FLOORED need dispatcher extensions not yet
; built (single-cell-only ENVFOUND path).
ENVQUERY: PULU D
STD ENVLEN
PULU D
STD ENVADDR
LDX #ENVTABLE
ENVLOOP: LDD ,X
CMPD #0
BEQ ENVNOTFOUND
PSHU D
LDD 2,X
PSHU D
LDD ENVADDR
PSHU D
LDD ENVLEN
PSHU D
JSR COMPAREW
PULU D
CMPD #0
BEQ ENVFOUND
LEAX 6,X
BRA ENVLOOP
ENVFOUND: LDD 4,X
PSHU D
LDD #TRUEV
PSHU D
RTS
ENVNOTFOUND: LDD #FALSEV
PSHU D
RTS

ENVTABLE:
FDB EN1, EN1L, 31
FDB EN2, EN2L, 32767
FDB EN3, EN3L, 65535
FDB EN6, EN6L, 8
FDB 0
EN1: FCC "/COUNTED-STRING"
EN1L EQU *-EN1
EN2: FCC "MAX-N"
```

```

EN2L    EQU    *-EN2
EN3:    FCC    "MAX-U"
EN3L    EQU    *-EN3
EN6:    FCC    "ADDRESS-UNIT-BITS"
EN6L    EQU    *-EN6

```

8.26 Tools Word Set

```

; =====
; SECTION 25: TOOLS WORD SET (.S / WORDS / DUMP)
; =====
DOTS:    TFR    U,D
          STD    DSPTMP
DSL00P:  LDD    DSPTMP
          CMPD   #SP0
          BEQ    DSDONE
          LDX    DSPTMP
          LDD    ,X
          PSHU   D
          JSR    DOT
          LDD    DSPTMP
          ADDD   #2
          STD    DSPTMP
          BRA    DSL00P
DSDONE:  RTS

WORDSW:  LDD    LATEST
          STD    WWALK
WWLOOP:  LDD    WWALK
          BEQ    WWDONE
          LDX    WWALK
          LDA    ,X
          STA    HDRFLAGS
          LEAX   1,X
          PSHU   X
          LDB    HDRFLAGS
          ANDB   #$1F
          CLRA
          PSHU   D
          JSR    TYPE
          JSR    SPACEW
          LDX    WWALK
          LEAX   1,X
          LDB    HDRFLAGS
          ANDB   #$1F
          CLRA
          LEAX   D,X
          LDD    ,X
          STD    WWALK
          BRA    WWLOOP
WWDONE:  JSR    CRW
          RTS

HEXDIGIT: PULU   D
          CMPD   #10
          BLO    HDDIGIT
          ADDD   #'A'-10
          BRA    HDEMIT
HDDIGIT:  ADDD   #'0'
HDEMIT:  PSHU   D
          JSR    EMIT
          RTS

HEXBYTE: PULU   D
          STB    MSCR
          LDB    MSCR+1
          LSRB
          LSRB

```

```

        LSRB
        LSRB
        CLRA
        PSHU D
        JSR  HEXDIGIT
        LDB  MSCR+1
        ANDB #$0F
        CLRA
        PSHU D
        JSR  HEXDIGIT
        RTS

; DUMP - includes the partial-final-line ASCII fix
DUMPW:  PULU D
        STD  DUMPCNT
        PULU D
        STD  DUMPADDR
DULINE: LDD  DUMPCNT
        LBEQ DUDONE          ; was BEQ - out of short-branch range
        LDD  DUMPADDR
        STD  HEXBUF
        CLR  DUMPCOL
        LDA  #16
        STA  DUVALID
DUHEX:  LDB  DUMPCOL
        CMPB #16
        BEQ  DUASCII
        LDD  DUMPCNT
        BNE  DUHEXBYTE
        LDA  DUMPCOL
        STA  DUVALID
        BRA  DUHEXPAD
DUHEXBYTE: LDX DUMPADDR
        LDB  ,X
        CLRA
        PSHU D
        JSR  HEXBYTE
        LDD  #32
        PSHU D
        JSR  EMIT
        LDX DUMPADDR
        LEAX 1,X
        STX DUMPADDR
        LDD DUMPCNT
        SUBD #1
        STD DUMPCNT
        INC DUMPCOL
        BRA DUHEX
DUHEXPAD: LDD #32
        PSHU D
        JSR  EMIT
        PSHU D
        JSR  EMIT
        PSHU D
        JSR  EMIT
        INC DUMPCOL
        LDB DUMPCOL
        CMPB #16
        BNE DUHEXPAD
        BRA DUASCII
DUASCII: LDD #32
        PSHU D
        JSR  EMIT
        CLR DUMPCOL
DUACHAR: LDB DUMPCOL
        CMPB #16
        BEQ  DULEND
        CMPB DUVALID
        BHS  DUABLANK
        LDX  HEXBUF
        LDB  DUMPCOL

```

```

        CLRA
        LEAX D,X
        LDA ,X
        CMPA #32
        BLO DUDOT
        CMPA #127
        BHS DUDOT
        BRA DUPRINT
DUDOT:  LDA #'.'
DUPRINT: TFR A,B
        CLRA
        PSHU D
        JSR  EMIT
        INC  DUMPCOL
        BRA  DUACHAR
DUABLANK: LDD #32
        PSHU D
        JSR  EMIT
        INC  DUMPCOL
        BRA  DUACHAR
DULEND:  JSR  CRW
        LDD  DUMPCNT
        LBNE DULINE          ; was BNE - out of short-branch range
DUDONE:  RTS

```

8.27 ABORT / QUIT Hand-Built Headers

```

; =====
; SECTION 26: TRUE / FALSE (CONSTANT-pattern ROM words). This
; section used to also hold ABORT/QUIT's hand-built dictionary
; headers (ABORTHDR/QUITHDR) - moved into BASEDICT and renamed to
; H_ABORT/H_QUIT (see the end of SECTION 27) so every header in
; this ROM lives in one contiguous block, rather than header data
; sitting apart from the rest of the dictionary inside BASECODE.
; =====
; -----
; TRUE / FALSE - CONSTANT TRUE -1 / CONSTANT FALSE 0. The first
; CONSTANT-pattern ROM-resident words in this system: every
; other ROM word's CFA is a plain code label (a real routine),
; but a CONSTANT's CFA is the DODOES trampoline pattern (JSR
; DODOES + FDB ATSIGN + FDB <value-cell-address>), matching
; exactly what interactive CONSTANT compiles at runtime - the
; only difference is the value-cell address is a fixed, known-
; at-assembly-time label here (TRUEVAL/FALSEVAL) rather than a
; CODEHERE snapshot captured dynamically. DODOES pushes the
; value stored at the PFA field (the address in TRUEVAL/
; FALSEVAL); ATSIGN then fetches through that address, yielding
; the actual -1 / 0 - the same indirection interactive CONSTANT
; relies on, just with fixed addresses instead of a runtime one.
; -----
TRUEBODY: JSR  DODOES
          FDB  ATSIGN
          FDB  TRUEVAL
TRUEVAL:  FDB  -1

FALSEBODY: JSR  DODOES
          FDB  ATSIGN
          FDB  FALSEVAL
FALSEVAL: FDB  0

; Verify no collision with init code,
; value should match ORG INITCODE.
BASECODEEND EQU *
BASECODESIZE EQU  BASECODEEND-BASECODE

; Prevent the assembler from extinguishing the gap between the
; BASECODE block and the INITCODE block when it generates the
; .bin file.

```

```
BASEND:
    FILL $FF,INITCODE-BASEND
```

8.28 Forth Dictionary (ROM Base Dictionary Headers)

```
; =====
; SECTION 27: FORTH DICTIONARY (ROM base dictionary headers)
; Every primitive word in the Glossary gets a real header here,
; chained via LINK, living in BASEDICT ($D83F-$E006, an exact fit
; for this dictionary's 1992 bytes - was $E000-$E7FF). CFA points
; directly at each primitive's own code label for almost every
; entry - these are raw code entries, not DODOES trampolines, so
; CFA = the label itself. The two exceptions are TRUE and FALSE
; (added in a later pass, chain's newest entries before H_ABORT/
; H_QUIT below): their CFA is the DODOES-trampoline pattern
; instead, matching what CONSTANT would compile interactively -
; see TRUEBODY/FALSEBODY, section 26, for why and how.
;
; DOES> is included (H_DOESGT) - added in a follow-up pass after
; the original 214-entry generation flagged it as missing, then
; moved again so it sits immediately after CREATE in the chain
; (H_CREATE -> H_DOESGT -> H_VARIABLE) rather than at the chain's
; newest end, since the two words are tightly coupled and read
; better adjacent. SETDOES (the runtime it compiles a call to)
; lives beside DODOES/DOESRT0; DOESGT is DOES>'s code label,
; since a literal ">" is not valid in a 6809 assembler label.
;
; H_ABORT and H_QUIT (formerly ABORTHDR/QUITHDR, renamed to match
; this file's H_ naming convention) are hand-built, not produced
; by the same generation pass as the other 217 entries - see the
; note where they're defined, at the end of this section, for why
; HEADER/CREATE couldn't be used directly for these two. They used
; to live physically apart from the rest of the dictionary, inside
; BASECODE rather than BASEDICT, despite being header data rather
; than code - moved here so every header lives in one place and
; the chain (H_QUIT -> H_ABORT -> (newest entry below) -> ... ->
; (oldest entry) -> 0) is visible in one block. BASELATEST remains
; H_QUIT, the chain's true overall head.
;
; H_ABORT's LINK field, a placeholder 0 since it was first built,
; is resolved here: it now points to this chain's newest entry.
;
; Names containing a literal double-quote (S", .", ABORT") have
; that character split into a standalone FCB $22 rather than
; escaped inside FCC - see emit_name's comment for why.
; =====

    ORG    BASEDICT        ; BASEDICT is $D83F

H_KEY:
    FCB    $03
    FCC    "KEY"
    FDB    0
    FDB    KEY

H_KEYQ:
    FCB    $04
    FCC    "KEY?"
    FDB    H_KEY
    FDB    KEYQ

H_EMIT:
    FCB    $04
    FCC    "EMIT"
    FDB    H_KEYQ
    FDB    EMIT

H_ACCEPT:
    FCB    $06
    FCC    "ACCEPT"
    FDB    H_EMIT
```

```

      FDB  ACCEPT
H_EXPECTW:
      FCB  $06
      FCC  "EXPECT"
      FDB  H_ACCEPT
      FDB  EXPECTW
H_QUERY:
      FCB  $05
      FCC  "QUERY"
      FDB  H_EXPECTW
      FDB  QUERY
H_TYPE:
      FCB  $04
      FCC  "TYPE"
      FDB  H_QUERY
      FDB  TYPE
H_CRW:
      FCB  $02
      FCC  "CR"
      FDB  H_TYPE
      FDB  CRW
H_SPACEW:
      FCB  $05
      FCC  "SPACE"
      FDB  H_CRW
      FDB  SPACEW
H_SPACESW:
      FCB  $06
      FCC  "SPACES"
      FDB  H_SPACEW
      FDB  SPACESW
H_DUP:
      FCB  $03
      FCC  "DUP"
      FDB  H_SPACESW
      FDB  DUP
H_DROP:
      FCB  $04
      FCC  "DROP"
      FDB  H_DUP
      FDB  DROP
H_SWAP:
      FCB  $04
      FCC  "SWAP"
      FDB  H_DROP
      FDB  SWAP
H_OVER:
      FCB  $04
      FCC  "OVER"
      FDB  H_SWAP
      FDB  OVER
H_ROT:
      FCB  $03
      FCC  "ROT"
      FDB  H_OVER
      FDB  ROT
H_QDUP:
      FCB  $04
      FCC  "?DUP"
      FDB  H_ROT
      FDB  QDUP
H_DEPTH:
      FCB  $05
      FCC  "DEPTH"
      FDB  H_QDUP
      FDB  DEPTH
H_DDUP:
      FCB  $04
      FCC  "2DUP"
      FDB  H_DEPTH
      FDB  DDUP

```

```

H_DDROD:
    FCB    $05
    FCC    "2DROD"
    FDB    H_DDUP
    FDB    DDROD

H_DSWAP:
    FCB    $05
    FCC    "2SWAP"
    FDB    H_DDROD
    FDB    DSWAP

H_DOVER:
    FCB    $05
    FCC    "2OVER"
    FDB    H_DSWAP
    FDB    DOVER

H_NIP:
    FCB    $03
    FCC    "NIP"
    FDB    H_DOVER
    FDB    NIP

H_TUCK:
    FCB    $04
    FCC    "TUCK"
    FDB    H_NIP
    FDB    TUCK

H_PICK:
    FCB    $04
    FCC    "PICK"
    FDB    H_TUCK
    FDB    PICK

H_ROLL:
    FCB    $04
    FCC    "ROLL"
    FDB    H_PICK
    FDB    ROLL

H_DR0T:
    FCB    $04
    FCC    "2R0T"
    FDB    H_ROLL
    FDB    DR0T

H_TOR:
    FCB    $02
    FCC    ">R"
    FDB    H_DR0T
    FDB    TOR

H_FROMR:
    FCB    $02
    FCC    "R>"
    FDB    H_TOR
    FDB    FROMR

H_RFETCH:
    FCB    $02
    FCC    "R@"
    FDB    H_FROMR
    FDB    RFETCH

H_TW0TOR:
    FCB    $03
    FCC    "2>R"
    FDB    H_RFETCH
    FDB    TW0TOR

H_TW0FROMR:
    FCB    $03
    FCC    "2R>"
    FDB    H_TW0TOR
    FDB    TW0FROMR

H_TW0RFETCH:
    FCB    $03
    FCC    "2R@"
    FDB    H_TW0FROMR
    FDB    TW0RFETCH

H_PLUS:

```

```

        FCB    $01
        FCC    "+"
        FDB    H_TWOFETCH
        FDB    PLUS
H_MINUS:
        FCB    $01
        FCC    "-"
        FDB    H_PLUS
        FDB    MINUS
H_STAR:
        FCB    $01
        FCC    "*"
        FDB    H_MINUS
        FDB    STAR
H_SLASH:
        FCB    $01
        FCC    "/"
        FDB    H_STAR
        FDB    SLASH
H_MODW:
        FCB    $03
        FCC    "MOD"
        FDB    H_SLASH
        FDB    MODW
H_SLASHMOD:
        FCB    $04
        FCC    "/MOD"
        FDB    H_MODW
        FDB    SLASHMOD
H_NEGATE:
        FCB    $06
        FCC    "NEGATE"
        FDB    H_SLASHMOD
        FDB    NEGATE
H_ABSW:
        FCB    $03
        FCC    "ABS"
        FDB    H_NEGATE
        FDB    ABSW
H_MIN:
        FCB    $03
        FCC    "MIN"
        FDB    H_ABSW
        FDB    MIN
H_MAX:
        FCB    $03
        FCC    "MAX"
        FDB    H_MIN
        FDB    MAX
H_ONEPLUS:
        FCB    $02
        FCC    "1+"
        FDB    H_MAX
        FDB    ONEPLUS
H_ONEMINUS:
        FCB    $02
        FCC    "1-"
        FDB    H_ONEPLUS
        FDB    ONEMINUS
H_TWOPLUS:
        FCB    $02
        FCC    "2+"
        FDB    H_ONEMINUS
        FDB    TWOPLUS
H_TWOSTAR:
        FCB    $02
        FCC    "2*"
        FDB    H_TWOPLUS
        FDB    TWOSTAR
H_TWOSLASH:
        FCB    $02

```

```

        FCC    "2/"
        FDB    H_TWOSTAR
        FDB    TWOSLASH
H_STARSLASH:
        FCB    $02
        FCC    "*/"
        FDB    H_TWOSLASH
        FDB    STARSLASH
H_STARSLASHMOD:
        FCB    $05
        FCC    "*/MOD"
        FDB    H_STARSLASH
        FDB    STARSLASHMOD
H_UMSTAR:
        FCB    $03
        FCC    "UM*"
        FDB    H_STARSLASHMOD
        FDB    UMSTAR
H_UMSLASHMOD:
        FCB    $06
        FCC    "UM/MOD"
        FDB    H_UMSTAR
        FDB    UMSLASHMOD
H_MSTAR:
        FCB    $02
        FCC    "M*"
        FDB    H_UMSLASHMOD
        FDB    MSTAR
H_FMSLASHMOD:
        FCB    $06
        FCC    "FM/MOD"
        FDB    H_MSTAR
        FDB    FMSLASHMOD
H_SMSLASHREM:
        FCB    $06
        FCC    "SM/REM"
        FDB    H_FMSLASHMOD
        FDB    SMSLASHREM
H_DPLUS:
        FCB    $02
        FCC    "D+"
        FDB    H_SMSLASHREM
        FDB    DPLUS
H_DMINUS:
        FCB    $02
        FCC    "D-"
        FDB    H_DPLUS
        FDB    DMINUS
H_DNEGATEW:
        FCB    $07
        FCC    "DNEGATE"
        FDB    H_DMINUS
        FDB    DNEGATEW
H_DABSW:
        FCB    $04
        FCC    "DABS"
        FDB    H_DNEGATEW
        FDB    DABSW
H_MPLUS:
        FCB    $02
        FCC    "M+"
        FDB    H_DABSW
        FDB    MPLUS
H_STOD:
        FCB    $03
        FCC    "S>D"
        FDB    H_MPLUS
        FDB    STOD
H_DTOS:
        FCB    $03
        FCC    "D>S"

```

```

        FDB H_STOD
        FDB DTOS
H_DMAXW:
        FCB $04
        FCC "DMAX"
        FDB H_DTOS
        FDB DMAXW
H_DMINW:
        FCB $04
        FCC "DMIN"
        FDB H_DMAXW
        FDB DMINW
H_ANDW:
        FCB $03
        FCC "AND"
        FDB H_DMINW
        FDB ANDW
H_ORW:
        FCB $02
        FCC "OR"
        FDB H_ANDW
        FDB ORW
H_XORW:
        FCB $03
        FCC "XOR"
        FDB H_ORW
        FDB XORW
H_INVERT:
        FCB $06
        FCC "INVERT"
        FDB H_XORW
        FDB INVERT
H_LSHIFT:
        FCB $06
        FCC "LSHIFT"
        FDB H_INVERT
        FDB LSHIFT
H_RSHIFT:
        FCB $06
        FCC "RSHIFT"
        FDB H_LSHIFT
        FDB RSHIFT
H_CELLSW:
        FCB $05
        FCC "CELLS"
        FDB H_RSHIFT
        FDB CELLSW
H_CELLPLUS:
        FCB $05
        FCC "CELL+"
        FDB H_CELLSW
        FDB CELLPLUS
H_CHARSW:
        FCB $05
        FCC "CHARS"
        FDB H_CELLPLUS
        FDB CHARSW
H_CHARPLUS:
        FCB $05
        FCC "CHAR+"
        FDB H_CHARSW
        FDB CHARPLUS
H_ALIGNW:
        FCB $05
        FCC "ALIGN"
        FDB H_CHARPLUS
        FDB ALIGNW
H_ALIGNEDW:
        FCB $07
        FCC "ALIGNED"
        FDB H_ALIGNW

```

```

      FDB    ALIGNEDW
H_EQUALW:
      FCB    $01
      FCC    "="
      FDB    H_ALIGNEDW
      FDB    EQUALW
H_LESSW:
      FCB    $01
      FCC    "<"
      FDB    H_EQUALW
      FDB    LESSW
H_GREATERW:
      FCB    $01
      FCC    ">"
      FDB    H_LESSW
      FDB    GREATERW
H_ZEROEQ:
      FCB    $02
      FCC    "0="
      FDB    H_GREATERW
      FDB    ZEROEQ
H_ZEROLT:
      FCB    $02
      FCC    "0<"
      FDB    H_ZEROEQ
      FDB    ZEROLT
H_ULESSW:
      FCB    $02
      FCC    "U<"
      FDB    H_ZEROLT
      FDB    ULESSW
H_NOTEQUAL:
      FCB    $02
      FCC    "<>"
      FDB    H_ULESSW
      FDB    NOTEQUAL
H_ZERONE:
      FCB    $03
      FCC    "0<>"
      FDB    H_NOTEQUAL
      FDB    ZERONE
H_ZEROGT:
      FCB    $02
      FCC    "0>"
      FDB    H_ZERONE
      FDB    ZEROGT
H_UGREATER:
      FCB    $02
      FCC    "U>"
      FDB    H_ZEROGT
      FDB    UGREATER
H_WITHINW:
      FCB    $06
      FCC    "WITHIN"
      FDB    H_UGREATER
      FDB    WITHINW
H_DEQUAL:
      FCB    $02
      FCC    "D="
      FDB    H_WITHINW
      FDB    DEQUAL
H_DLESSW:
      FCB    $02
      FCC    "D<"
      FDB    H_DEQUAL
      FDB    DLESSW
H_DULESSW:
      FCB    $03
      FCC    "DU<"
      FDB    H_DLESSW
      FDB    DULESSW

```

```

H_IF:
    FCB    $82
    FCC    "IF"
    FDB    H_DULESSW
    FDB    IF
H_THEN:
    FCB    $84
    FCC    "THEN"
    FDB    H_IF
    FDB    THEN
H_ELSE:
    FCB    $84
    FCC    "ELSE"
    FDB    H_THEN
    FDB    ELSE
H_BEGIN:
    FCB    $85
    FCC    "BEGIN"
    FDB    H_ELSE
    FDB    BEGIN
H_UNTIL:
    FCB    $85
    FCC    "UNTIL"
    FDB    H_BEGIN
    FDB    UNTIL
H_AGAIN:
    FCB    $85
    FCC    "AGAIN"
    FDB    H_UNTIL
    FDB    AGAIN
H_WHILE:
    FCB    $85
    FCC    "WHILE"
    FDB    H_AGAIN
    FDB    WHILE
H_REPEAT:
    FCB    $86
    FCC    "REPEAT"
    FDB    H_WHILE
    FDB    REPEAT
H_RECURSE:
    FCB    $87
    FCC    "RECURSE"
    FDB    H_REPEAT
    FDB    RECURSE
H_DO:
    FCB    $82
    FCC    "DO"
    FDB    H_RECURSE
    FDB    DO
H_QDO:
    FCB    $83
    FCC    "?DO"
    FDB    H_DO
    FDB    QDO
H_LOOP:
    FCB    $84
    FCC    "LOOP"
    FDB    H_QDO
    FDB    LOOP
H_PLUSLOOP:
    FCB    $85
    FCC    "+LOOP"
    FDB    H_LOOP
    FDB    PLUSLOOP
H_IWORD:
    FCB    $01
    FCC    "I"
    FDB    H_PLUSLOOP
    FDB    IWORD
H_JWORD:

```

```

        FCB    $01
        FCC    "J"
        FDB    H_IWORD
        FDB    JWORD
H_LEAVE:
        FCB    $05
        FCC    "LEAVE"
        FDB    H_JWORD
        FDB    LEAVE
H_UNLOOP:
        FCB    $06
        FCC    "UNLOOP"
        FDB    H_LEAVE
        FDB    UNLOOP
H_EXIT:
        FCB    $84
        FCC    "EXIT"
        FDB    H_UNLOOP
        FDB    EXIT
H_CASEW:
        FCB    $84
        FCC    "CASE"
        FDB    H_EXIT
        FDB    CASEW
H_OF:
        FCB    $82
        FCC    "OF"
        FDB    H_CASEW
        FDB    OF
H_ENDOF:
        FCB    $85
        FCC    "ENDOF"
        FDB    H_OF
        FDB    ENDOF
H_ENDCASE:
        FCB    $87
        FCC    "ENDCASE"
        FDB    H_ENDOF
        FDB    ENDCASE
H_COLON:
        FCB    $01
        FCC    ":"
        FDB    H_ENDCASE
        FDB    COLON
H_SEMI:
        FCB    $81
        FCC    ";"
        FDB    H_COLON
        FDB    SEMI
H_CREATE:
        FCB    $06
        FCC    "CREATE"
        FDB    H_SEMI
        FDB    CREATE
H_DOESGT:
        FCB    $85                ; $80 IMMEDIATE | 5 (length of "DOES>")
        FCC    "DOES>"
        FDB    H_CREATE
        FDB    DOESGT
H_VARIABLE:
        FCB    $08
        FCC    "VARIABLE"
        FDB    H_DOESGT
        FDB    VARIABLE
H_CONSTANT:
        FCB    $08
        FCC    "CONSTANT"
        FDB    H_VARIABLE
        FDB    CONSTANT
H_VALUEW:
        FCB    $05

```

```

        FCC    "VALUE"
        FDB    H_CONSTANT
        FDB    VALUEW
H_TOW:
        FCB    $82
        FCC    "TO"
        FDB    H_VALUEW
        FDB    TOW
H_TWOVARIABLE:
        FCB    $09
        FCC    "2VARIABLE"
        FDB    H_TOW
        FDB    TWOVARIABLE
H_TWOCONSTANT:
        FCB    $09
        FCC    "2CONSTANT"
        FDB    H_TWOVARIABLE
        FDB    TWOCONSTANT
H_BUFFERCOLON:
        FCB    $07
        FCC    "BUFFER:"
        FDB    H_TWOCONSTANT
        FDB    BUFFERCOLON
H_DEFERW:
        FCB    $05
        FCC    "DEFER"
        FDB    H_BUFFERCOLON
        FDB    DEFERW
H_DEFERFETCH:
        FCB    $06
        FCC    "DEFER@"
        FDB    H_DEFERW
        FDB    DEFERFETCH
H_DEFERSTORE:
        FCB    $06
        FCC    "DEFER!"
        FDB    H_DEFERFETCH
        FDB    DEFERSTORE
H_ISW:
        FCB    $82
        FCC    "IS"
        FDB    H_DEFERSTORE
        FDB    ISW
H_ACTIONOF:
        FCB    $89
        FCC    "ACTION-OF"
        FDB    H_ISW
        FDB    ACTIONOF
H_MARKERW:
        FCB    $06
        FCC    "MARKER"
        FDB    H_ACTIONOF
        FDB    MARKERW
H_IMMEDIATE:
        FCB    $09
        FCC    "IMMEDIATE"
        FDB    H_MARKERW
        FDB    IMMEDIATE
H_STATEW:
        FCB    $05
        FCC    "STATE"
        FDB    H_IMMEDIATE
        FDB    STATEW
H_LBRACKET:
        FCB    $81
        FCC    "["
        FDB    H_STATEW
        FDB    LBRACKET
H_RBRACKET:
        FCB    $81
        FCC    "]"

```

```

        FDB H_LBRACKET
        FDB RBRACKET
H_TICK:
        FCB $01
        FCC " '"
        FDB H_RBRACKET
        FDB TICK
H_COMPILECOMMA:
        FCB $08
        FCC "COMPILE, "
        FDB H_TICK
        FDB COMPILECOMMA
H_LITERALW:
        FCB $87
        FCC "LITERAL"
        FDB H_COMPILECOMMA
        FDB LITERALW
H_BRACKTICK:
        FCB $83
        FCC "[ ' ]"
        FDB H_LITERALW
        FDB BRACKTICK
H_POSTPONEW:
        FCB $88
        FCC "POSTPONE"
        FDB H_BRACKTICK
        FDB POSTPONEW
H_TOBODY:
        FCB $05
        FCC ">BODY"
        FDB H_POSTPONEW
        FDB TOBODY
H_EXECUTE:
        FCB $07
        FCC "EXECUTE"
        FDB H_TOBODY
        FDB EXECUTE
H_SLITERALW:
        FCB $88
        FCC "SLITERAL"
        FDB H_EXECUTE
        FDB SLITERALW
H_ABORTQUOTE:
        FCB $86
        FCC "ABORT"
        FCB $22 ; ' ' - split out of FCC, not escaped within it
        FDB H_SLITERALW
        FDB ABORTQUOTE
H_ATSIGN:
        FCB $01
        FCC "@"
        FDB H_ABORTQUOTE
        FDB ATSIGN
H_STOREW:
        FCB $01
        FCC "!"
        FDB H_ATSIGN
        FDB STOREW
H_CFETCH:
        FCB $02
        FCC "C@"
        FDB H_STOREW
        FDB CFETCH
H_CSTOREW:
        FCB $02
        FCC "C!"
        FDB H_CFETCH
        FDB CSTOREW
H_PLUSSTORE:
        FCB $02
        FCC "+!"

```

```

        FDB H_CSTOREW
        FDB PLUSSTORE
H_DFETCH:
        FCB $02
        FCC "2@"
        FDB H_PLUSSTORE
        FDB DFETCH
H_DSTORE:
        FCB $02
        FCC "2!"
        FDB H_DFETCH
        FDB DSTORE
H_COMMA:
        FCB $01
        FCC ", "
        FDB H_DSTORE
        FDB COMMA
H_CCOMMA:
        FCB $02
        FCC "C, "
        FDB H_COMMA
        FDB CCOMMA
H_ALLOT:
        FCB $05
        FCC "ALLOT"
        FDB H_CCOMMA
        FDB ALLOT
H_HEREW:
        FCB $04
        FCC "HERE"
        FDB H_ALLOT
        FDB HEREW
H_VCOMMA:
        FCB $02
        FCC "V, "
        FDB H_HEREW
        FDB VCOMMA
H_VCCOMMA:
        FCB $03
        FCC "VC, "
        FDB H_VCOMMA
        FDB VCCOMMA
H_VALLOT:
        FCB $06
        FCC "VALLOT"
        FDB H_VCCOMMA
        FDB VALLOT
H_VHEREW:
        FCB $05
        FCC "VHERE"
        FDB H_VALLOT
        FDB VHEREW
H_PADW:
        FCB $03
        FCC "PAD"
        FDB H_VHEREW
        FDB PADW
H_UNUSEDW:
        FCB $06
        FCC "UNUSED"
        FDB H_PADW
        FDB UNUSEDW
H_VUNUSEDW:
        FCB $07
        FCC "VUNUSED"
        FDB H_UNUSEDW
        FDB VUNUSEDW
H_MOVEW:
        FCB $04
        FCC "MOVE"
        FDB H_VUNUSEDW

```

```

      FDB    MOVEW
H_FILLW:
      FCB    $04
      FCC    "FILL"
      FDB    H_MOVEW
      FDB    FILLW
H_ERASEW:
      FCB    $05
      FCC    "ERASE"
      FDB    H_FILLW
      FDB    ERASEW
H_CMOVEW:
      FCB    $05
      FCC    "CMOVE"
      FDB    H_ERASEW
      FDB    CMOVEW
H_CMOVEGT:
      FCB    $06
      FCC    "CMOVE>"
      FDB    H_CMOVEW
      FDB    CMOVEGT
H_COUNT:
      FCB    $05
      FCC    "COUNT"
      FDB    H_CMOVEGT
      FDB    COUNT
H_WORD:
      FCB    $04
      FCC    "WORD"
      FDB    H_COUNT
      FDB    WORD
H_CHARW:
      FCB    $04
      FCC    "CHAR"
      FDB    H_WORD
      FDB    CHARW
H_BRACKCHAR:
      FCB    $86
      FCC    "[CHAR]"
      FDB    H_CHARW
      FDB    BRACKCHAR
H_PARSEW:
      FCB    $05
      FCC    "PARSE"
      FDB    H_BRACKCHAR
      FDB    PARSEW
H_PARSENAME:
      FCB    $0A
      FCC    "PARSE-NAME"
      FDB    H_PARSEW
      FDB    PARSENAME
H_SQUOTE:
      FCB    $82
      FCC    "S"
      FCB    $22          ; '"' - split out of FCC, not escaped within it
      FDB    H_PARSENAME
      FDB    SQUOTE
H_DOTQUOTE:
      FCB    $82
      FCC    " ."
      FCB    $22          ; '"' - split out of FCC, not escaped within it
      FDB    H_SQUOTE
      FDB    DOTQUOTE
H_COMPAREW:
      FCB    $07
      FCC    "COMPARE"
      FDB    H_DOTQUOTE
      FDB    COMPAREW
H_SEARCHW:
      FCB    $06
      FCC    "SEARCH"

```

```

        FDB  H_COMPAREW
        FDB  SEARCHW
H_DASHTRAILING:
        FCB  $09
        FCC  "-TRAILING"
        FDB  H_SEARCHW
        FDB  DASHTRAILING
H_SLASHSTRING:
        FCB  $07
        FCC  "/STRING"
        FDB  H_DASHTRAILING
        FDB  SLASHSTRING
H_REPLACESW:
        FCB  $08
        FCC  "REPLACES"
        FDB  H_SLASHSTRING
        FDB  REPLACESW
H_SUBSTITUTEW:
        FCB  $0A
        FCC  "SUBSTITUTE"
        FDB  H_REPLACESW
        FDB  SUBSTITUTEW
H_SNAMEW:
        FCB  $05
        FCC  "SNAME"
        FDB  H_SUBSTITUTEW
        FDB  SNAMEW
H_UNESCAPEW:
        FCB  $08
        FCC  "UNESCAPE"
        FDB  H_SNAMEW
        FDB  UNESCAPEW
H_LTNUM:
        FCB  $02
        FCC  "<#"
        FDB  H_UNESCAPEW
        FDB  LTNUM
H_NUMSIGN:
        FCB  $01
        FCC  "#"
        FDB  H_LTNUM
        FDB  NUMSIGN
H_NUMSIGNS:
        FCB  $02
        FCC  "#S"
        FDB  H_NUMSIGN
        FDB  NUMSIGNS
H_NUMGT:
        FCB  $02
        FCC  "#>"
        FDB  H_NUMSIGNS
        FDB  NUMGT
H_HOLD:
        FCB  $04
        FCC  "HOLD"
        FDB  H_NUMGT
        FDB  HOLD
H_HOLDS:
        FCB  $05
        FCC  "HOLDS"
        FDB  H_HOLD
        FDB  HOLDS
H_SIGN:
        FCB  $04
        FCC  "SIGN"
        FDB  H_HOLDS
        FDB  SIGN
H_DOT:
        FCB  $01
        FCC  "."
        FDB  H_SIGN

```

```

      FDB    DOT
H_UDOT:
      FCB    $02
      FCC    "U. "
      FDB    H_DOT
      FDB    UDOT
H_DOTR:
      FCB    $02
      FCC    ".R"
      FDB    H_UDOT
      FDB    DOTR
H_UDOTR:
      FCB    $03
      FCC    "U.R"
      FDB    H_DOTR
      FDB    UDOTR
H_QMARK:
      FCB    $01
      FCC    "?"
      FDB    H_UDOTR
      FDB    QMARK
H_DDOT:
      FCB    $02
      FCC    "D. "
      FDB    H_QMARK
      FDB    DDOT
H_DDOTR:
      FCB    $03
      FCC    "D.R"
      FDB    H_DDOT
      FDB    DDOTR
H_BASEW:
      FCB    $04
      FCC    "BASE"
      FDB    H_DDOTR
      FDB    BASEW
H_DECIMAL:
      FCB    $07
      FCC    "DECIMAL "
      FDB    H_BASEW
      FDB    DECIMAL
H_HEXW:
      FCB    $03
      FCC    "HEX"
      FDB    H_DECIMAL
      FDB    HEXW
H_BINARYW:
      FCB    $06
      FCC    "BINARY"
      FDB    H_HEXW
      FDB    BINARYW
H_CATCH:
      FCB    $05
      FCC    "CATCH"
      FDB    H_BINARYW
      FDB    CATCH
H_THROW:
      FCB    $05
      FCC    "THROW"
      FDB    H_CATCH
      FDB    THROW
H_LPAREN:
      FCB    $81
      FCC    "("
      FDB    H_THROW
      FDB    LPAREN
H_BACKSLASH:
      FCB    $81
      FCC    "\"
      FDB    H_LPAREN
      FDB    BACKSLASH

```

```

H_ENVQUERY:
    FCB    $0C
    FCC    "ENVIRONMENT?"
    FDB    H_BACKSLASH
    FDB    ENVQUERY
H_SOURCEW:
    FCB    $06
    FCC    "SOURCE"
    FDB    H_ENVQUERY
    FDB    SOURCEW
H_SOURCEID:
    FCB    $09
    FCC    "SOURCE-ID"
    FDB    H_SOURCEW
    FDB    SOURCEID
H_REFILLW:
    FCB    $06
    FCC    "REFILL"
    FDB    H_SOURCEID
    FDB    REFILLW
H_EVALUATEW:
    FCB    $08
    FCC    "EVALUATE"
    FDB    H_REFILLW
    FDB    EVALUATEW
H_TIBW:
    FCB    $03
    FCC    "TIB"
    FDB    H_EVALUATEW
    FDB    TIBW
H_NTIBW:
    FCB    $04
    FCC    "#TIB"
    FDB    H_TIBW
    FDB    NTIBW
H_TOINW:
    FCB    $03
    FCC    ">IN"
    FDB    H_NTIBW
    FDB    TOINW
H_SPANW:
    FCB    $04
    FCC    "SPAN"
    FDB    H_TOINW
    FDB    SPANW
H_BLW:
    FCB    $02
    FCC    "BL"
    FDB    H_SPANW
    FDB    BLW
H_DOTS:
    FCB    $02
    FCC    ".S"
    FDB    H_BLW
    FDB    DOTS
H_WORDSW:
    FCB    $05
    FCC    "WORDS"
    FDB    H_DOTS
    FDB    WORDSW
H_DUMPW:
    FCB    $04
    FCC    "DUMP"
    FDB    H_WORDSW
    FDB    DUMPW
H_TRUE:
    FCB    $04
    FCC    "TRUE"
    FDB    H_DUMPW
    FDB    TRUEBODY

```

```

H_FALSE:
    FCB    $05
    FCC    "FALSE"
    FDB    H_TRUE
    FDB    FALSEBODY

; H_ABORT and H_QUIT are hand-built, not produced by the same
; generation pass as the 217 entries above - ABORT and QUIT need
; to be findable at the prompt, but HEADER/CREATE couldn't be used
; directly for these two (see the source conversation this file
; was derived from for why). Moved here from their own separate
; section so every header in this ROM lives in one contiguous
; block, with the chain visible end to end: H_QUIT -> H_ABORT ->
; H_FALSE (the newest of the 217 generated entries) -> ... ->
; H_KEY -> 0.
H_ABORT: FCB    5
    FCC    "ABORT"
    FDB    H_FALSE
    ; resolved - was placeholder 0, then H_DUMPW,
    ; then H_DOESGT, then H_DUMPW again once
    ; DOES> moved out of the chain's newest slot;
    ; now H_FALSE, the chain's newest entry
    FDB    ABORT

H_QUIT:  FCB    4
    FCC    "QUIT"
    FDB    H_ABORT
    FDB    QUIT

BASELATEST EQU  H_QUIT    ; the ROM dictionary's true head - referenced by
    ; COLD to initialize LATEST. Was QUITHDR before
    ; the H_QUIT rename; before that, undefined
    ; entirely - a real bug, not a placeholder,
    ; found and fixed several turns before this one.

; Verify no collision with base code.
; Value should match ORG BASECODE
BASEDICTEND EQU    *
BASEDICTSIZE EQU  BASEDICTEND-BASEDICT

```

9. Alphabetical Index

Every word implemented, alphabetically, with the page in the Glossary (Section 3) where its full entry appears. Page numbers are live Word fields — update fields (Ctrl+A, F9) if they show as blank or “?” after edits.

-	12
- TRAILING.....	25
,	23
;	19
:	19
!	22
?	27
?DO.....	18
?DUP.....	10
.	26
. "	25
. R.....	26
. S.....	29
'	21
(.....	28
[.....	21
[']	21
[CHAR]	24
]	21
@.....	22
*	12
* /	13
* /MOD.....	13
/	12
/MOD.....	12
/STRING.....	25
\	28
#.....	26
#>.....	26
#S.....	26
#TIB.....	28
+	12
+ !	22
+LOOP.....	18
<	16
<#.....	26

<>.....	16
=.....	16
>.....	16
>BODY.....	22
>IN.....	28
>R.....	11
0<.....	16
0<>.....	16
0=.....	16
0>.....	17
1 -	13
1+.....	13
2!.....	23
2@.....	22
2*.....	13
2/.....	13
2+.....	13
2>R.....	12
2CONSTANT.....	20
2DROP.....	11
2DUP.....	10
2OVER.....	11
2R@.....	12
2R>.....	12
2ROT.....	11
2SWAP.....	11
2VARIABLE.....	20
ABORT.....	10
ABORT".....	22
ABS.....	12
ACCEPT.....	9
ACTION-OF.....	21
AGAIN.....	18
ALIGN.....	16
ALIGNED.....	16
ALLOT.....	23
AND.....	15
BASE.....	27
BEGIN.....	17
BINARY.....	27
BL.....	29

BUFFER:	20
C,	23
C!	22
C@	22
CASE	19
CATCH	27
CELL+	15
CELLS	15
CHAR	24
CHAR+	16
CHARS	15
CMOVE	24
CMOVE>	24
COMPARE	25
COMPILE,	21
CONSTANT	20
COUNT	24
CR	9
CREATE	19
D-	14
D.	27
D.R.	27
D+	14
D<	17
D=	17
D>S	14
DABS	14
DECIMAL	27
DEFER	20
DEFER!	20
DEFER@	20
DEPTH	10
DMAX	15
DMIN	15
DNEGATE	14
DO	18
DOES>	19
DROP	10
DU<	17
DUMP	29
DUP	10

ELSE.....	17
EMIT.....	9
ENDCASE.....	19
ENDOF.....	19
ENVIRONMENT?.....	28
ERASE.....	24
EVALUATE.....	28
EXECUTE.....	22
EXIT.....	19
EXPECT.....	9
FILL.....	24
FM/MOD.....	14
HERE.....	23
HEX.....	27
HOLD.....	26
HOLDS.....	26
I.....	18
IF.....	17
IMMEDIATE.....	21
INVERT.....	15
IS.....	20
J.....	18
KEY.....	9
KEY?.....	9
LEAVE.....	19
LITERAL.....	21
LOOP.....	18
LSHIFT.....	15
M*.....	14
M+.....	14
MARKER.....	21
MAX.....	13
MIN.....	13
MOD.....	12
MOVE.....	24
NEGATE.....	12
NIP.....	11
OF.....	19
OR.....	15
OVER.....	10
PAD.....	23

PARSE.....	24
PARSE - NAME.....	25
PICK.....	11
POSTPONE.....	22
QUERY.....	9
QUIT.....	10
R@.....	11
R>.....	11
RECURSE.....	18
REFILL.....	28
REPEAT.....	18
REPLACES.....	25
ROLL.....	11
ROT.....	10
RSHIFT.....	15
S".....	25
S>D.....	14
SEARCH.....	25
SIGN.....	26
SLITERAL.....	22
SM/REM.....	14
SNAME.....	25
SOURCE.....	28
SOURCE - ID.....	28
SPACE.....	9
SPACES.....	9
SPAN.....	28
STATE.....	21
SUBSTITUTE.....	25
SWAP.....	10
THEN.....	17
THROW.....	27
TIB.....	28
TO.....	20
TUCK.....	11
TYPE.....	9
U.....	26
U . R.....	27
U<.....	16
U>.....	17
UM*.....	13

UM/MOD.....	14
UNESCAPE.....	25
UNLOOP.....	19
UNTIL.....	18
UNUSED.....	23
V,	23
VALLLOT.....	23
VALUE.....	20
VARIABLE.....	20
VC,	23
VHERE.....	23
VUNUSED.....	24
WHILE.....	18
WITHIN.....	17
WORD.....	24
WORDS.....	29
XOR.....	15